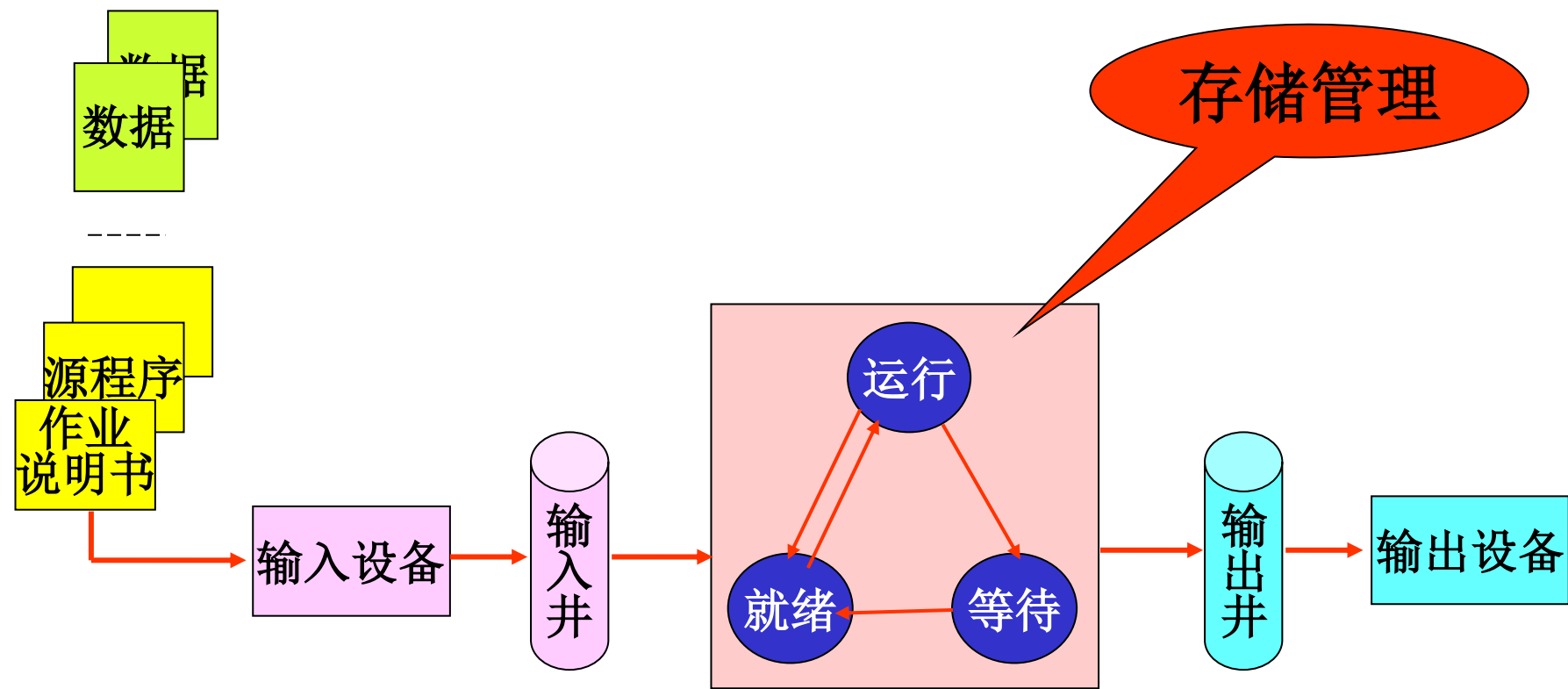
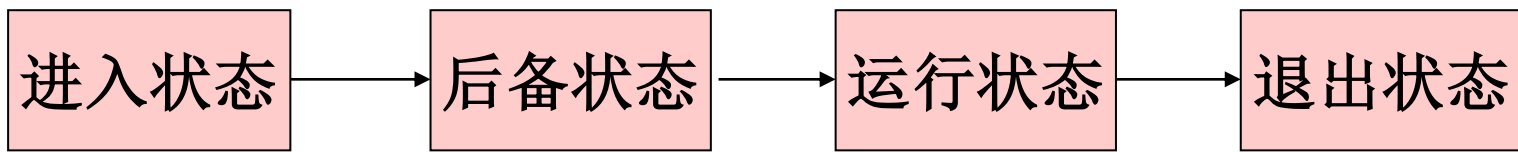




第八章

存储管理







第八章 存储管理



- **存储器**是计算机系统的重要资源。程序只有装载进内存，才能运行；**信息**需要存储在辅助存储设备上保存。存储管理直接影响系统性能。

几个问题：

1. 程序在内存是如何存放的？
2. 如何安全运行如此多的程序？
3. 如何多个进程不互相干扰？
4. 如何运行大程序？
5. 编程时考虑过代码或数据放在哪里吗，即地址是多少？
6.

进程 性能 应用历史记录 启动 用户 详细信息 服务					
名称	状态	1% CPU	17% 内存	0% 磁盘	0% 网络
> WPS Office (32 位) (8)		0.3%	409.5 MB	0 MB/秒	0 Mbps
> 360安全浏览器 (32 位) (8)		0.1%	82.3 MB	0 MB/秒	0 Mbps
Microsoft Edge		0%	41.1 MB	0 MB/秒	0 Mbps
360安全卫士 安全防护中心模块...		0.1%	37.2 MB	0 MB/秒	0.1 Mbps
桌面窗口管理器		0.1%	26.7 MB	0 MB/秒	0 Mbps
> Windows 资源管理器		0%	25.9 MB	0 MB/秒	0 Mbps
任务管理器		0.4%	23.0 MB	0 MB/秒	0 Mbps
Microsoft Edge		0%	20.3 MB	0.1 MB/秒	0 Mbps
Microsoft Windows Search ...		0%	13.6 MB	0 MB/秒	0 Mbps
> 360日历主程序 (32 位) (4)		0.1%	12.6 MB	0.1 MB/秒	0 Mbps
Microsoft Text Input Applicat...		0%	10.3 MB	0 MB/秒	0 Mbps
服务主机: Windows Manage...		0%	9.8 MB	0 MB/秒	0 Mbps
360主动防御服务模块 (32 位)		0%	8.8 MB	0 MB/秒	0 Mbps
服务主机: Diagnostic Policy S...		0%	8.8 MB	0 MB/秒	0 Mbps
MoUSO Core Worker Process		0%	8.1 MB	0 MB/秒	0 Mbps
桌面助手		0.1%	8.0 MB	0 MB/秒	0 Mbps
服务主机: DCOM 服务进程...		0%	8.0 MB	0 MB/秒	0 Mbps
服务主机: 远程过程调用 (2)		0.1%	7.9 MB	0 MB/秒	0 Mbps
服务主机: Windows Event Log		0%	7.6 MB	0.1 MB/秒	0 Mbps
360安全浏览器 组件 (32 位)		0%	6.8 MB	0 MB/秒	0 Mbps
Microsoft Edge		0%	6.7 MB	0.1 MB/秒	0 Mbps
服务主机: State Repository Se...		0%	6.6 MB	0 MB/秒	0 Mbps
CTF 加载程序		0%	6.1 MB	0 MB/秒	0 Mbps
WPS服务程序, 提供账号登录...		0.1%	6.1 MB	0 MB/秒	0 Mbps
Prome CEF SubProcess (32 位)		0%	5.9 MB	0 MB/秒	0 Mbps
WPS Office (32 位)		0%	5.9 MB	0 MB/秒	0 Mbps



第八章 存储管理



内容目录

1. 存储管理的功能及目标
2. 分区存储器
3. 覆盖与交换技术
4. 页式与段式存储管理
5. 虚拟存储器

目标

1. 熟悉存储管理的功能；
2. 掌握虚拟地址变换的过程；
3. 了解分区存储管理的原理，能够利用相应算法实现内存的管理与分配；
4. 了解覆盖和交换技术；
5. 了解页式和段式管理的基本原理；理解各自实现地址变换的过程
6. 实现虚拟存储器的基本原理，地址变换的机构和过程；
7. 能利用页面置换的算法实现页面置换



8.1 存储管理的功能及目标



内容:

1. 虚拟存储器
2. 地址变换
3. 程序链接
4. 内外存数据传输的控制
5. 内存的分配与回收
6. 内存信息的共享与保护

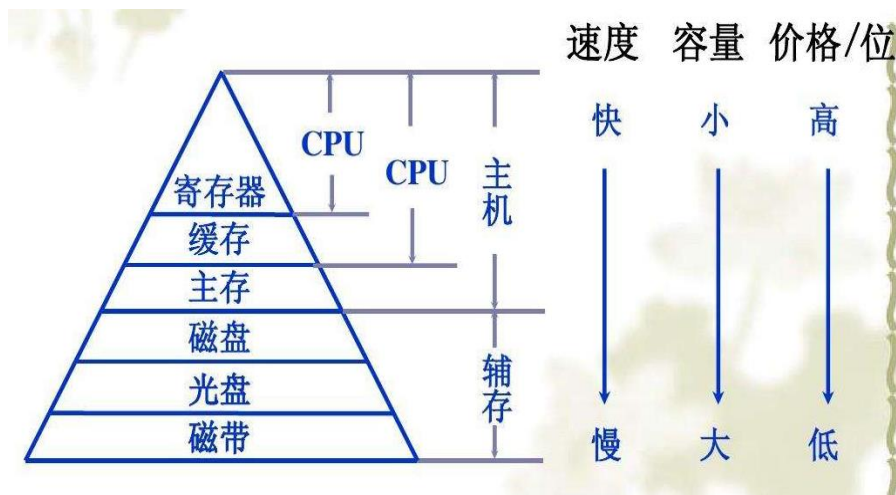


8.1 存储管理的功能及目标

8.1.1 虚拟存储器

(1) 存储器层次结构

- 存储器的功能是保存数据，存储器的发展方向是高速、大容量和小体积
 - 内存在访问速度方面的发展：DRAM、SDRAM、SRAM等
 - 硬盘技术在大容量方面的发展：接口标准、存储密度等
- 存储组织是指在存储技术和CPU寻址技术许可的范围内组织合理的存储结构
 - 依据是访问速度匹配关系、容量要求和价格
 - “寄存器-内存-外存”结构
 - “寄存器-缓存-内存-磁盘-光盘-磁带”结构





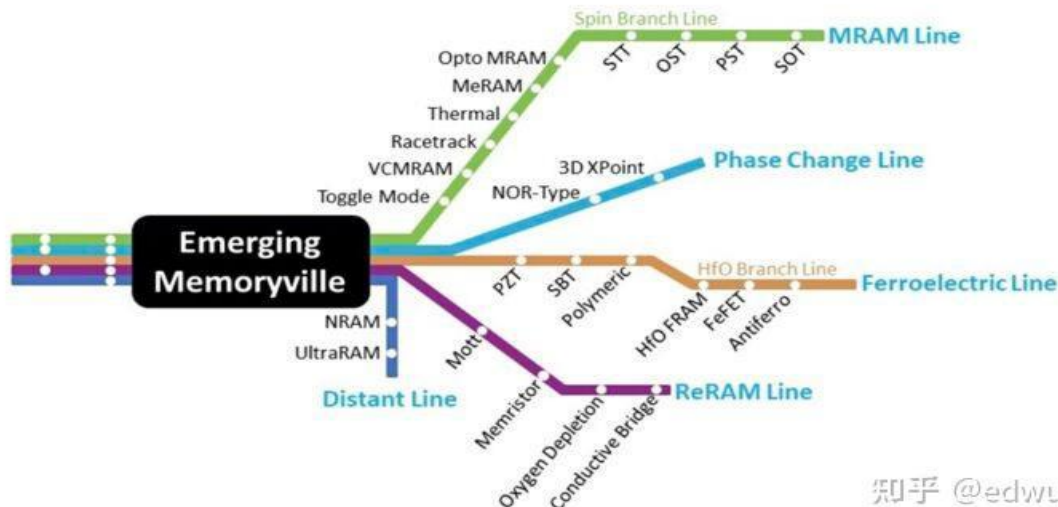
8.1 存储管理的功能及目标

8.1.1 虚拟存储器

(1) 存储器层次结构

● 新兴存储器技术

五种新兴存储技术的前景：磁阻随机存储器（MRAM）、相变存储器（PCM）、铁电随机存储器（FERAM）、电阻式随机存储器（ReRAM）和 NRAM/UltraRAM。这些通常是具有**动态随机存储器（DRAM）**数据访问速度的**非易失性存储器**，并且它们的路线图可以实现更高的**密度**，有望突破 **NAND 和 NOR** 存储的缩放极限，并且比不断刷新的 **DRAM 和 SRAM** 使用更少的电力。



知乎 @edwu



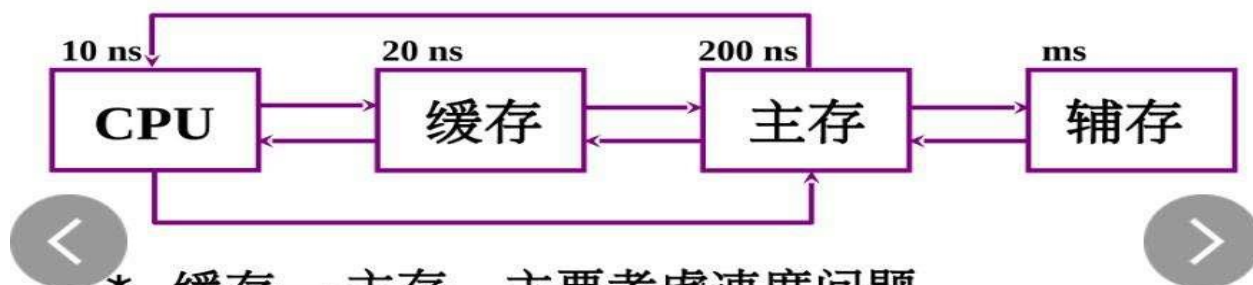
8.1 存储管理的功能及目标

8.1.1 虚拟存储器

(1) 存储器层次结构

● 微机中的存储层次组织

- 访问速度越慢，容量越大，价格越便宜
- 最佳状态应是各层次的存储器都处于均衡的繁忙状态（如：缓存命中率正好使主存读写保持繁忙）



* 缓存—主存 主要考虑速度问题

* 主存—辅存 主要考虑容量问题

* 局部性原理

(1) 时间局部性：现在正在访问的信息可能马上还会被访问到

(2) 空间局部性：现在正在访问的信息，与之相邻的信息可能马上也会被访问到



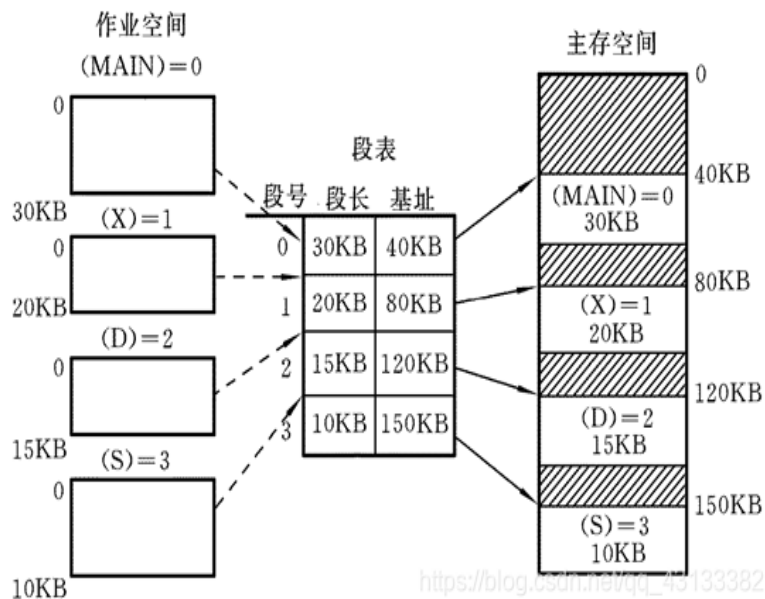
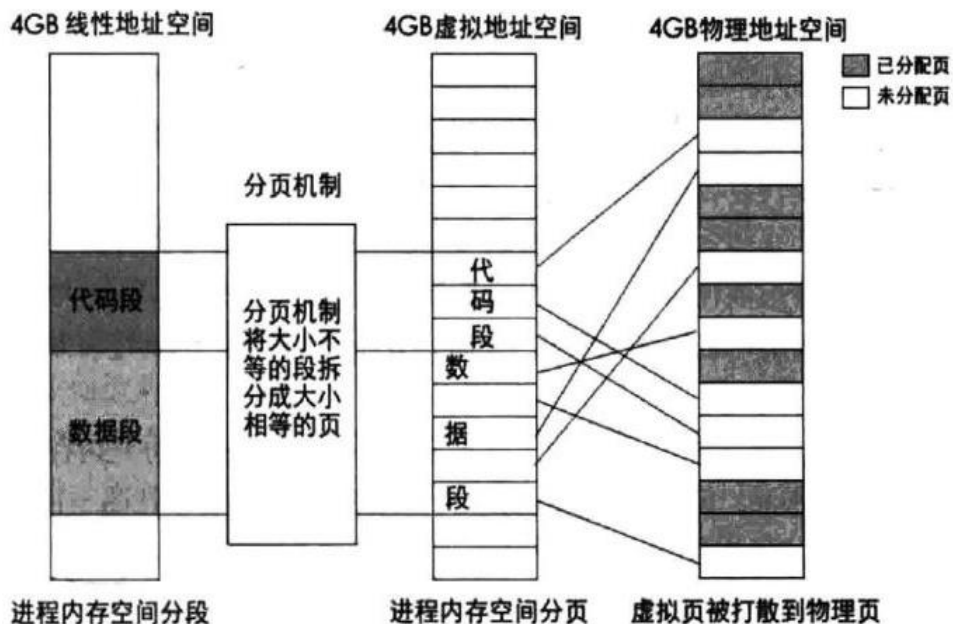
8.1 存储管理的功能及目标

8.1.1 虚拟存储器

(1) 存储器层次结构

主存储器 (Primary storage)

- 简称主存，或内存
- 主存可以抽象成什么？ 一个字节数组





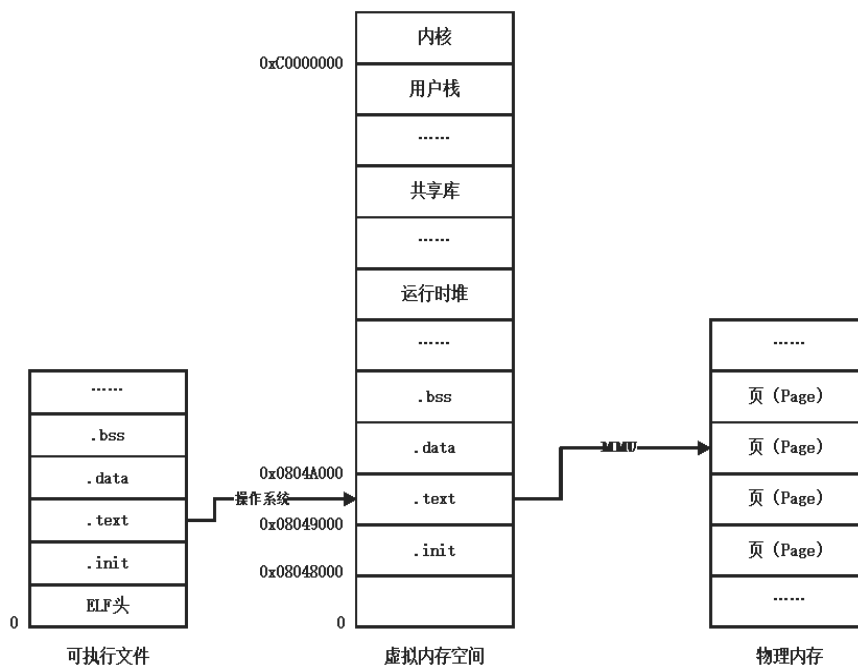
8.1 存储管理的功能及目标

8.1.1 虚拟存储器

(2) 虚拟存储器

- 进程的一部分数据和程序段在内存、另一部分在外存，
如何安排编译链接后的目标代码的地址？

- 按照物理存储器中的位置赋予实际物理地址？
- 编译链接到一个以0地址开始的线性或多维虚拟地址空间。可以是静态链接也可以是动态链接。其中的每条指令或数据的地址，称为虚拟地址（Virtual Address）。



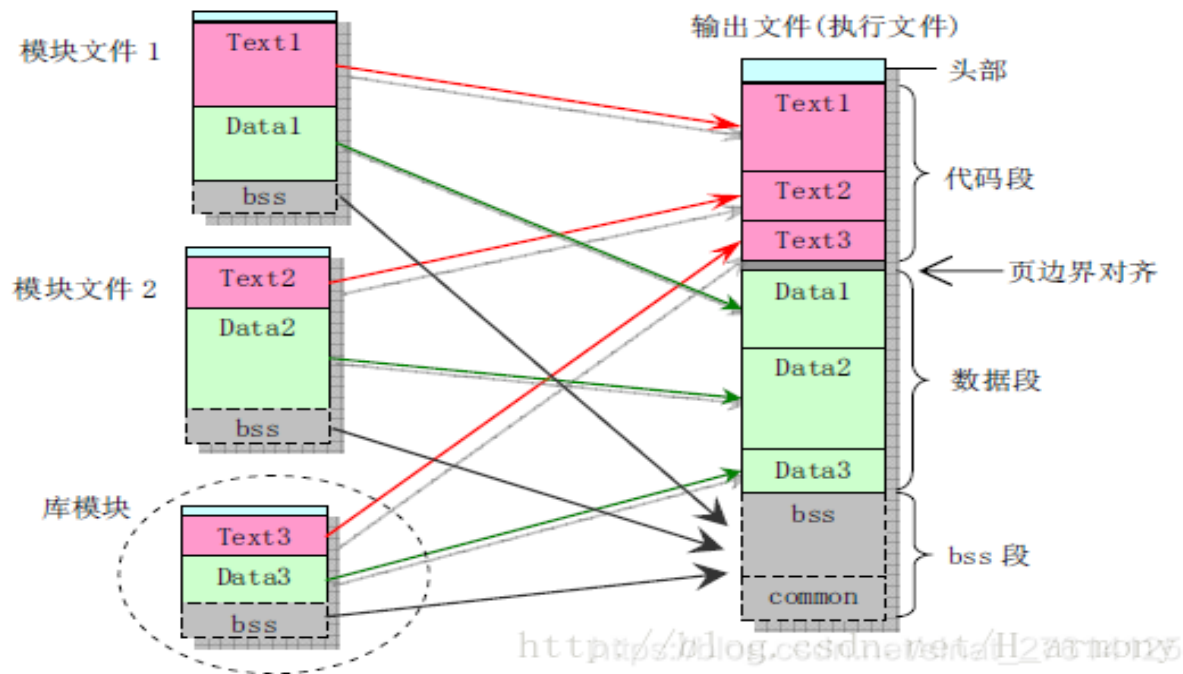


8.1 存储管理的功能及目标

8.1.1 虚拟存储器

(2) 虚拟存储器

➤ **链接：**是把相同的section合到一个模块段的过程





8.1 存储管理的功能及目标



8.1.1 虚拟存储器

(2) 虚拟存储器

- **虚拟 (Virtual) :** 一个物理实体映射为若干个对应的逻辑实体——分时或分空间。**虚拟**是OS管理系统资源的重要手段，可提高资源利用率。
 - **CPU**虚拟：每个用户（进程）的“虚拟处理机”
 - **存储器**虚拟：每个进程都占有的地址空间（指令+数据+堆+栈）——**虚拟存储器**
 - **显示设备**虚拟：多窗口、虚拟终端，虚设备等。

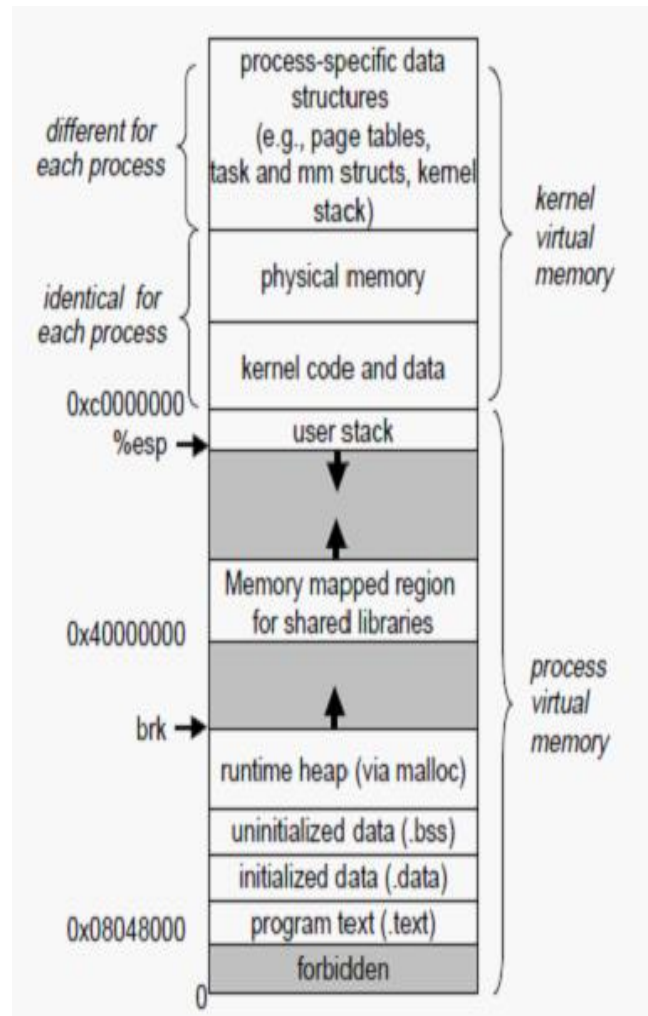


8.1 存储管理的功能及目标

8.1.1 虚拟存储器

(2) 虚拟存储器

- **虚地址空间**：一个程序被编译、链接后所限定的地址集合，称为**逻辑地址空间**，又称**虚拟存储器**，是一种实际上并不以物理形式存在的逻辑上的存储器。
- **目标程序**中指令和数据放置的位置，不考虑物理存储器的大小和信息存放的实际位置，只规定每个进程中互相关联的信息的**相对位置**——**相对地址**（逻辑地址、虚拟地址），是一个进程所访问的地址。



Virtual Memory of Linux Process



8.1 存储管理的功能及目标

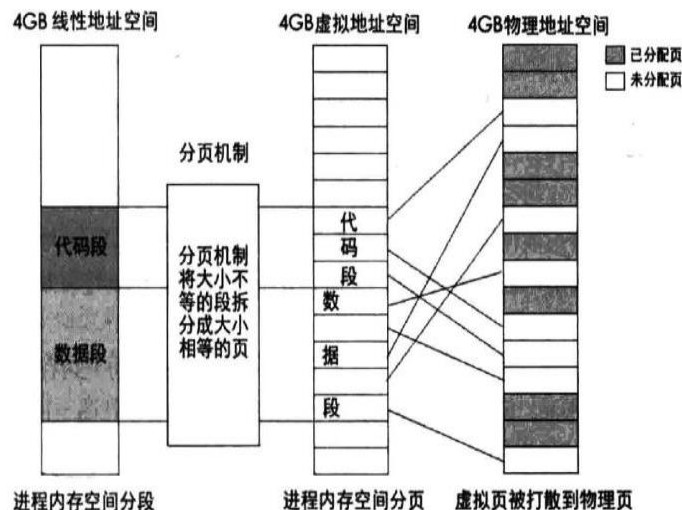
8.1.1 虚拟存储器

(2) 虚拟存储器

- **实地址空间**：进程在执行时实际存放其指令和数据的物理空间，是CPU能直接访问的内存的物理地址空间。

CPU可直接访问的内存地址——**实地址，或物理地址**

- 将进程所访问的**虚拟地址空间**与**实地址空间**区分开来，为进程的大小可**远大于**内存空间创造了条件



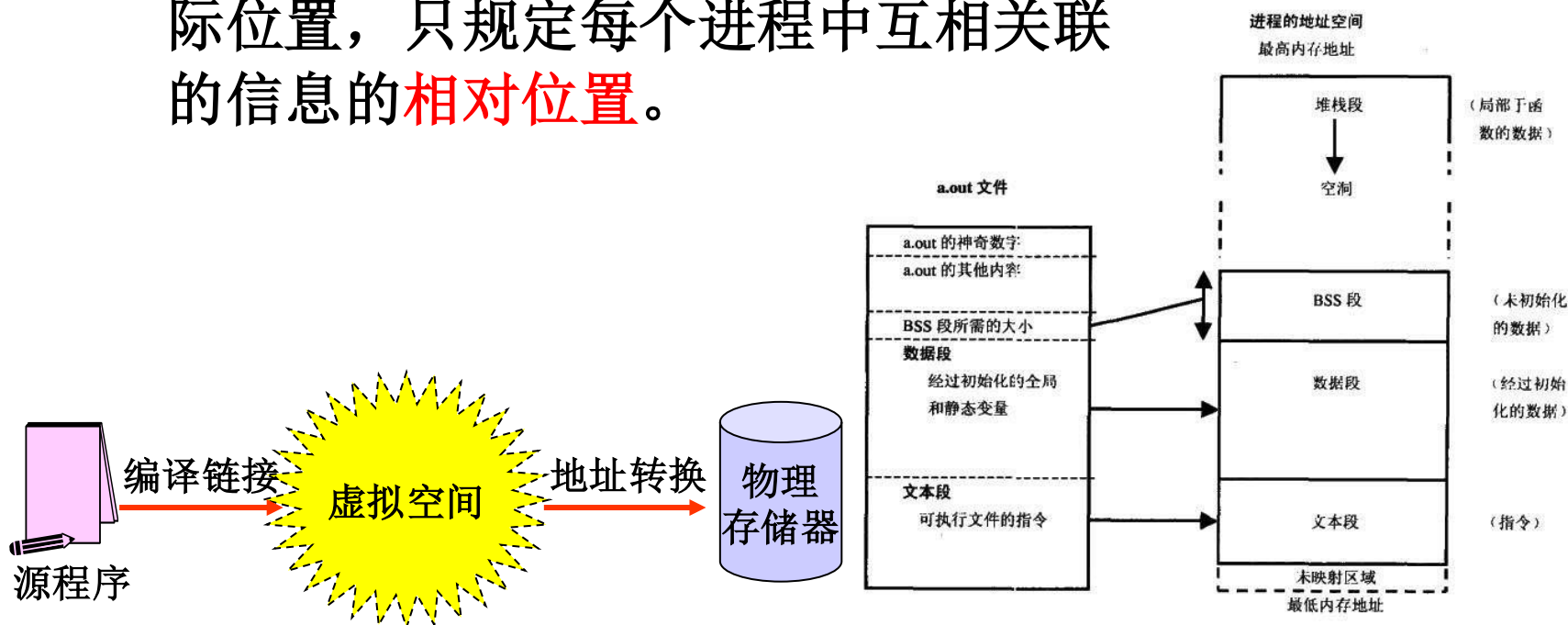


8.1 存储管理的功能及目标

8.1.1 虚拟存储器

(2) 虚拟存储器

- 进程中的目标代码、数据等的虚拟地址组成的虚拟空间称为**虚拟存储器** (virtual store or memory)
- 虚拟存储器**不考虑物理存储器的大小和信息存储的实际位置, 只规定每个进程中互相关联的信息的**相对位置**。





8.1 存储管理的功能及目标



8.1.1 虚拟存储器

(3) 虚拟存储器的基本原理

- **依据：**程序局部性原理
- **部分装入：**在程序装入时，不必将其全部读入内存，而只需将当前需要执行的部分读入到内存，就可让程序开始执行；
- **需要再调入：**在程序执行过程中，如果需执行的指令或访问的数据尚未在内存，则由处理器通知OS将相应部分调入内存，然后继续执行程序；
- **按需调出：**如果内存不足，OS可将内存中暂时不使用的部分调出保存在外存上，从而腾出空间存放将要装入的部分；
- **大小仍然受限：**通过物理内存和外存相结合，提供大范围的虚拟地址空间，可使用的总容量不超过物理内存和外存交换区容量之和，同时也受地址线宽度的限制。



8.1 存储管理的功能及目标



8.1.2 存储管理的功能

(1) 存储空间分配和回收

- 内存分配和回收算法、相应的**数据结构**：通过建表、查表、改表和回收登录内存使用情况，系统或用户申请内存时按选定分配算法确定分区等。

(2) 地址变换

- 可执行文件**生成中**的链接技术
- 程序**加载**（装入）时的重定位技术
- 进程**运行时**硬件和软件的地址变换技术和机构



8.1 存储管理的功能及目标

8.1.2 存储管理的功能

(3) 存储共享和保护

- 各道作业只在自己所属区域中运行，不破坏其他作业以及不被其他作业破坏；
- 代码和数据共享；
- 地址空间访问权限（读、写、执行）。

(4) 存储器扩充

- 使用虚存或自动复盖技术提供比实际内存更大的空间

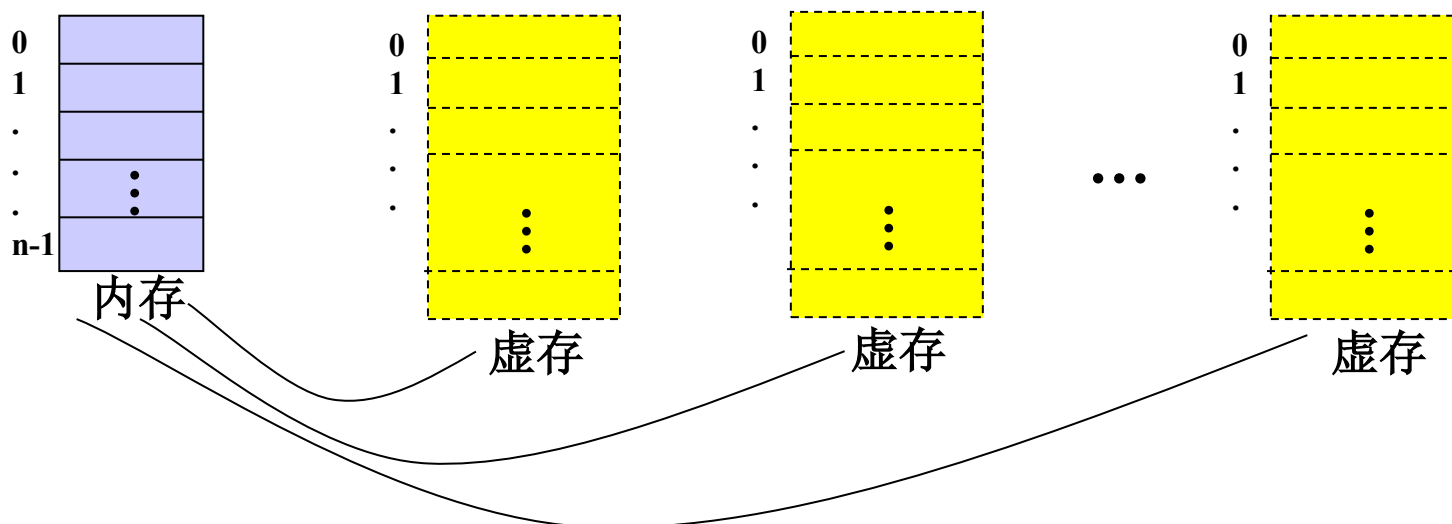
(5) 内外存数据传输控制

- 由应用程序控制：覆盖
- 由OS控制：交换（整个进程空间），虚拟存储的请求调入和预调入（部分进程空间）



8.1 存储管理的功能及目标

8.1.3 地址变换



问题:

- 如何划分虚拟空间
- 如何装入内存, 并将虚拟地址映射为内存地址

地址重定位, 包括静态地址重定位和动态地址重定位





8.1 存储管理的功能及目标

8.1.3 地址变换

(1) 逻辑地址、物理地址

- **逻辑地址**（相对地址，虚地址）：用户的程序经汇编或编译后形成**目标代码**，目标代码通常采用相对地址的形式
 - **其首地址为0**，其余指令中的地址都相对于首地址来编址
 - 不能用逻辑地址在内存中读取信息
- **物理地址**（绝对地址，实地址）：内存中存储单元的地址。物理地址可直接寻址



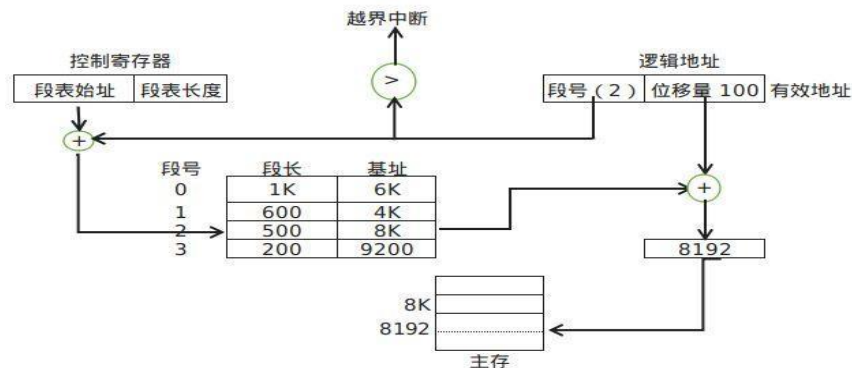
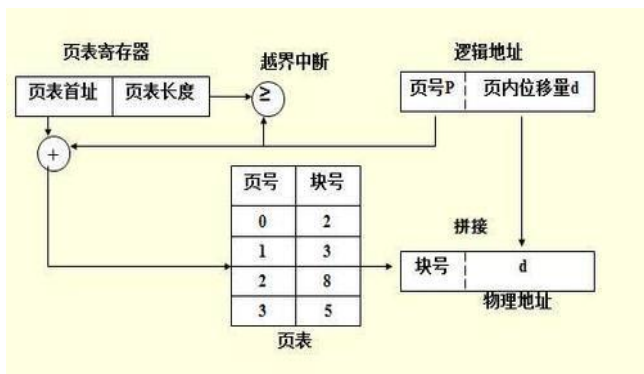
8.1 存储管理的功能及目标

8.1.3 地址变换

(1) 逻辑地址、物理地址（续）

➤ **地址映射**：将用户程序中的**逻辑地址**转换为运行时由机器直接寻址的**物理地址**的过程

🌍 **why?** 当程序装入内存时，OS要为该程序分配一个合适的内存空间，由于**程序的逻辑地址与分配到内存物理地址不一致**，而CPU执行指令时，是按物理地址进行的，所以要进行地址转换。



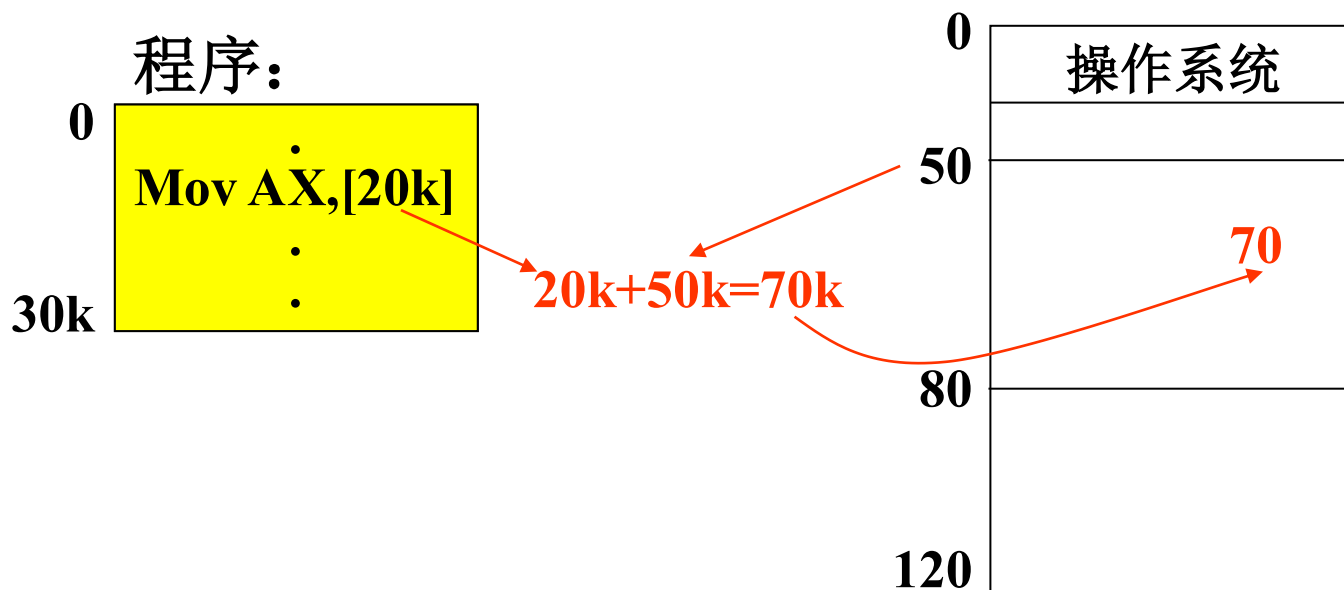


8.1 存储管理的功能及目标

8.1.3 地址变换

(2) 静态地址重定位 (Static Address Relocation)

- **静态重定位**是在目标程序装入内存时，由装入程序对目标程序中的指令和数据的地址进行修改，即把程序的逻辑地址都改成实际的内存地址。





8.1 存储管理的功能及目标



8.1.3 地址变换

(2) 静态地址重定位 (Static Address Relocation)

➤ 优点

- 无需硬件支持，可以装入有限多道程序

➤ 缺点

- 一个程序通常需要占用连续的内存空间，不易实现共享；
- 程序装入内存后不能移动；
- 无法实现虚拟存储器，因为受内存大小限制。

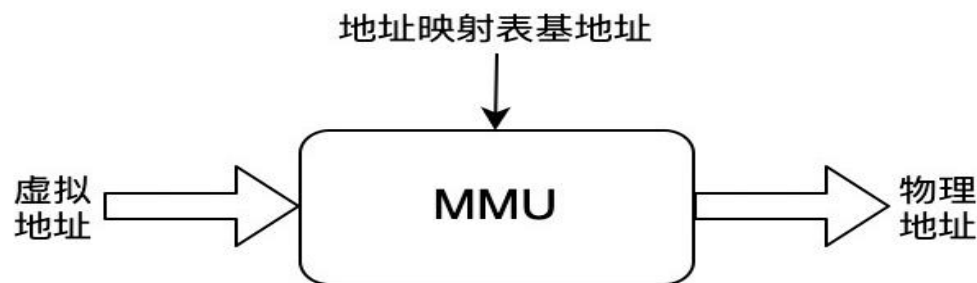


8.1 存储管理的功能及目标

8.1.3 地址变换

(3) 动态地址重定位 (Dynamic Address Relocation)

- 在程序运行过程中要访问数据或指令时再进行地址变换，即在逐条指令执行时完成地址映射。一般为了提高效率，此工作由硬件地址映射机制来完成。硬件支持，软硬件结合完成。
- 硬件上需要一对寄存器（地址映射表基址寄存器、虚拟地址寄存器）的支持，装入和执行时通过硬件地址变换机构，完成虚拟地址到实际内存地址的变换





8.1 存储管理的功能及目标



8.1.3 地址变换

(3) 动态地址重定位(dynamic Address Relocation)

● 优点

- OS可以将一个程序分散存放于不连续的内存空间，可以浮动程序，有利于实现共享（只要改变BR中地内容，便可将程序定位在新地内存空间中）；
- 可部分地、动态地分配内存，是虚拟存储器实现的基础。

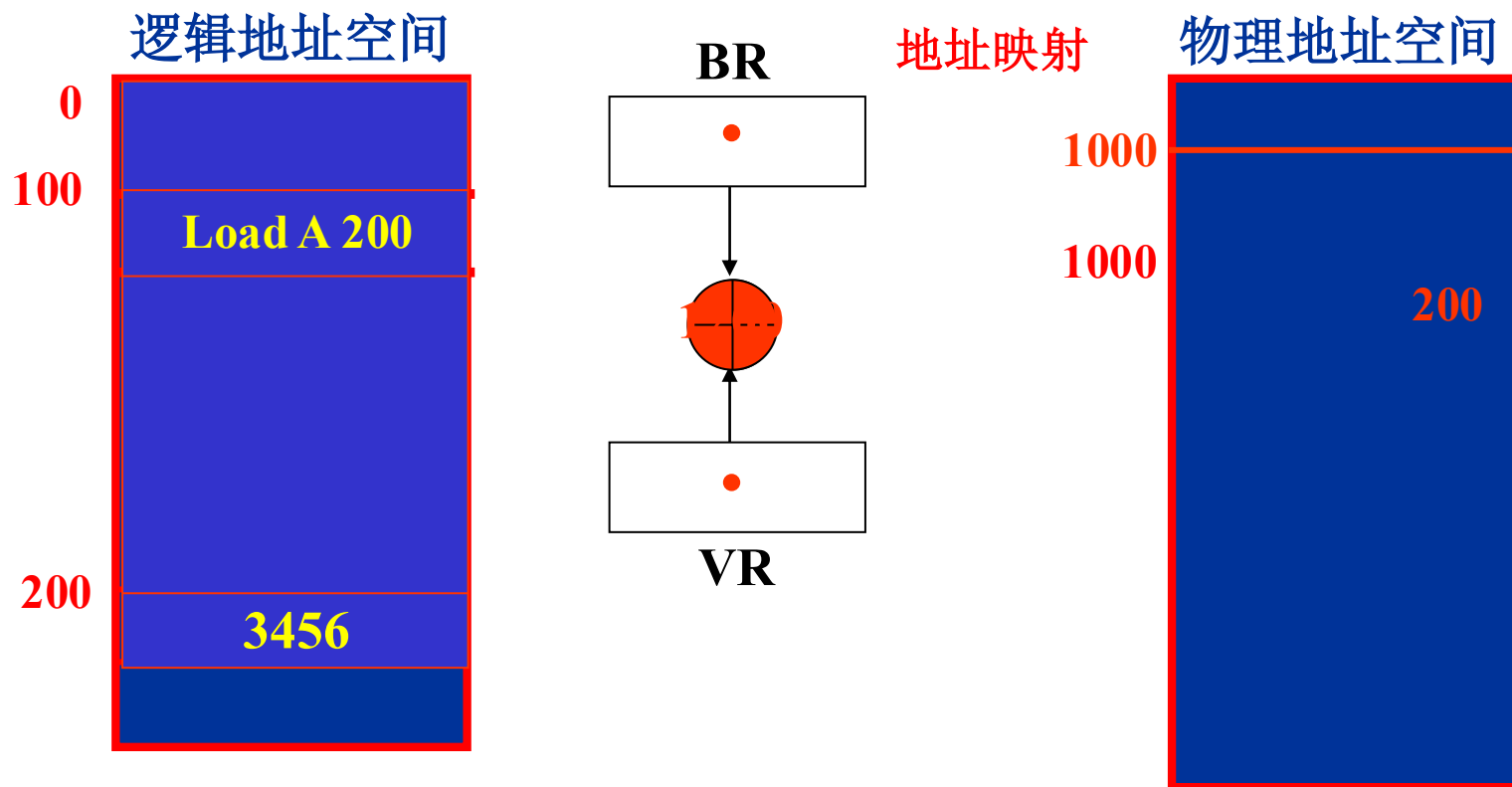
● 缺点：需要硬件支持，OS实现较复杂



8.1 存储管理的功能及目标

8.1.3 地址变换

(3) 动态地址重定位 (dynamic Address Relocation)

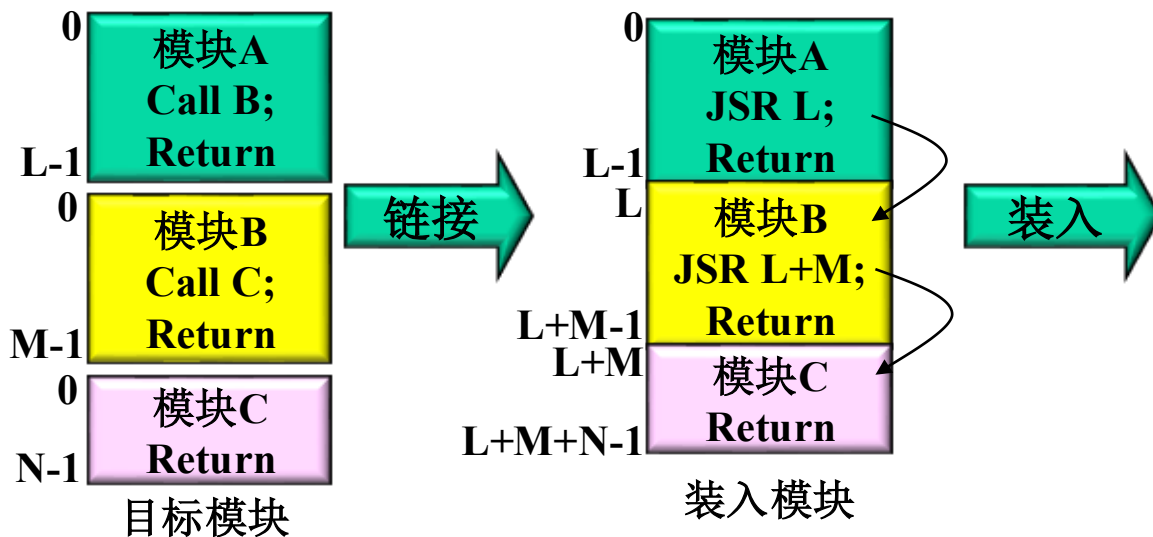
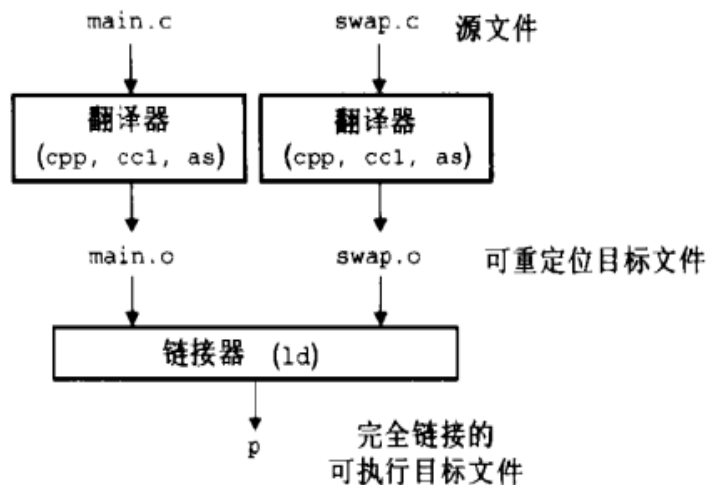




8.1 存储管理的功能及目标

8.1.4 程序链接

(1) **静态链接**：在程序执行之前，必须由链接装配程序把目标文件链接成一个可运行的目标程序，并在程序运行前都装入内存。



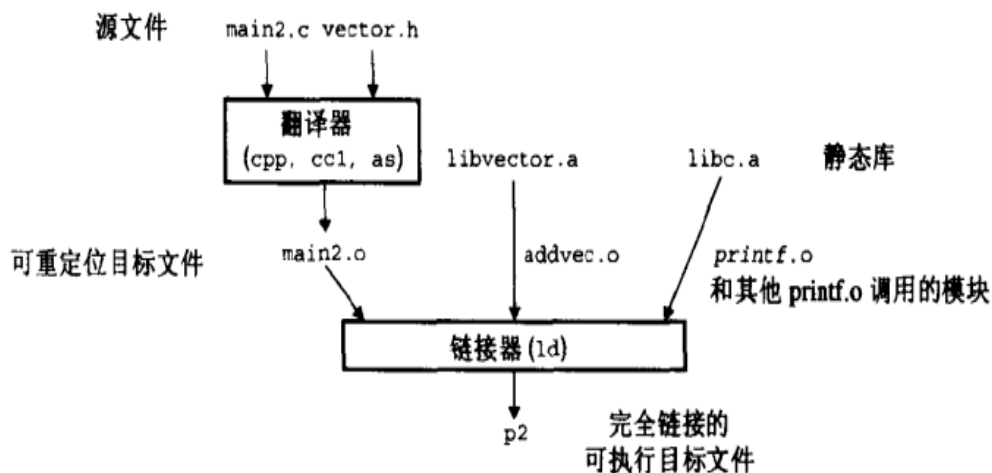
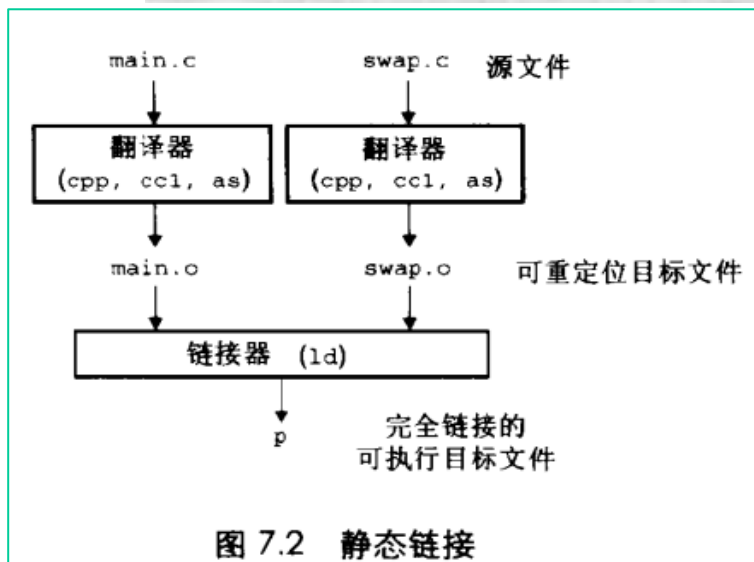
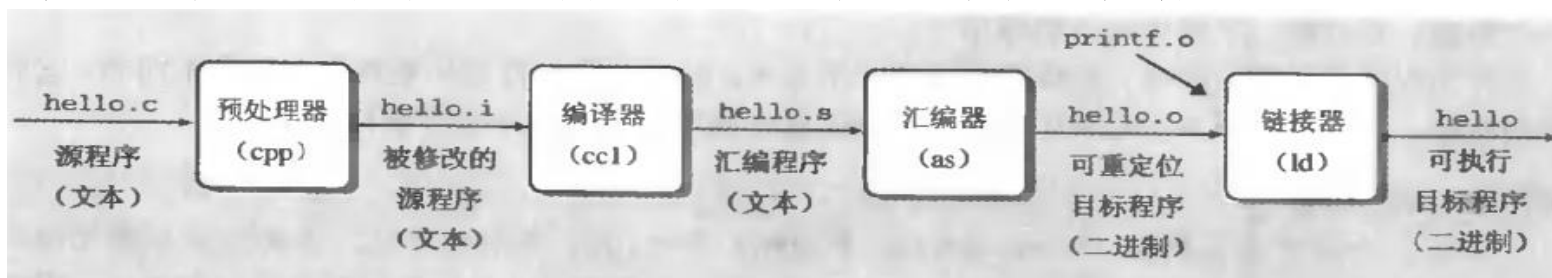


8.1 存储管理的功能及目标



8.1.4 程序链接

(1) **静态链接**: 在程序执行之前, 必须由链接装配程序把目标文件链接成一个可运行的目标程序, 并在程序运行前都装入内存。



- 源自: 深入理解计算机系统, <https://www.cnblogs.com/longjiang-uestc/p/10853201.html>
- 程序员的自我修养—链接、装载与库, 电子工业出版社

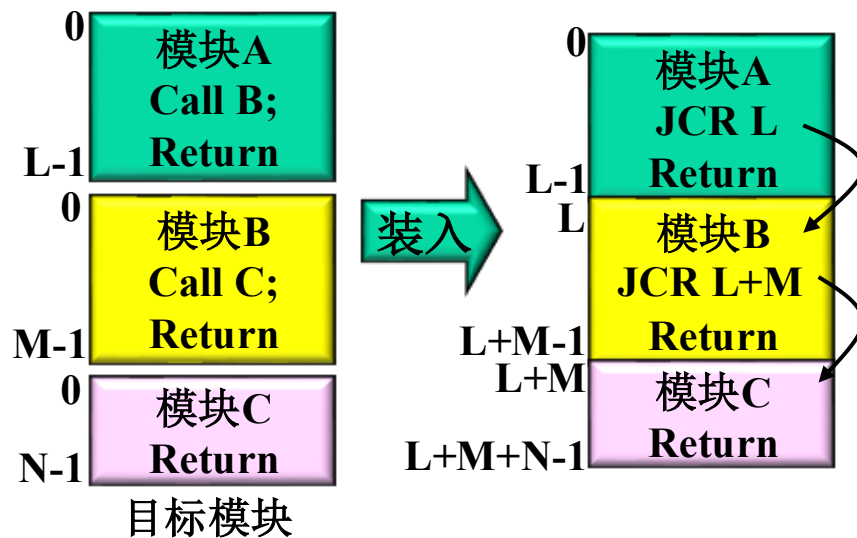


8.1 存储管理的功能及目标



8.1.4 程序链接

(2) **装入时动态链接**：边装入边链接。装入一个模块时，若发生外部模块调用，装入程序将去找出相应的外部模块并装入。



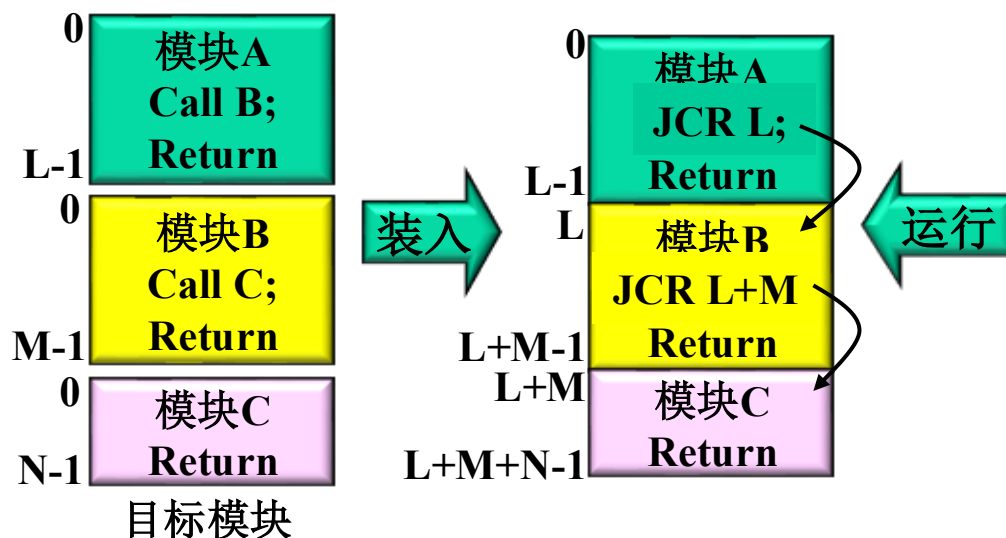


8.1 存储管理的功能及目标



8.1.4 程序链接

(3) 运行时动态链接：边执行边链接。在执行过程中，若发现一个被调用模块尚未装入内存时，由OS去找该模块并将其装入内存，链接到调用者模块上。



- 加快程序装入过程
- 节省内存



8.1 存储管理的功能及目标



8.1.5 内外存数据传输的控制

- **目的：** 将执行的程序和数据段调入内存，把处于等待状态的程序和数据段调出内存。
- **方法：**
 - 用户程序控制：例如，覆盖（Overlay）
 - 操作系统控制
 - 交换（Swapping）
 - 请求调入（on demand）和预调入（on prefetch）



8.1 存储管理的功能及目标



8.1.6 内存的分配与回收

- 将外存中的数据调入内存，在内存中安排合适的位置。
- **目标：**可使多个程序同时驻留在内存中，以提高CPU利用率；进程执行结束时，回收该进程所占内存资源
- **策略和数据结构：**
 - ✓ **分配结构：**登记内存使用情况，提供分配程序使用的数据结构，如内存空闲区表、空闲页面表等
 - ✓ **放置策略：**确定调入内存的程序和数据在内存中的位置，即按照什么原则选择空闲区。
 - ✓ **交换策略：**在需要将某个程序段和数据调入内存时，若内存无足够空间，使用交换策略，确定换出哪些内容
 - ✓ **调入策略：**决定外存中程序段和数据段调入的时间和控制方式
 - ✓ **回收策略：**回收的时机和空闲区的调整



8.1 存储管理的功能及目标



8.1.7 内存信息的共享与保护

● 存储保护的目

- 保护系统程序区不被用户侵犯（有意或无意的）
- 不允许用户程序读写不属于自己地址空间的数据（系统区地址空间，其他用户程序的地址空间）

● 保护方法

- 界限保护
- 访问方式保护（保护键）
- 其它

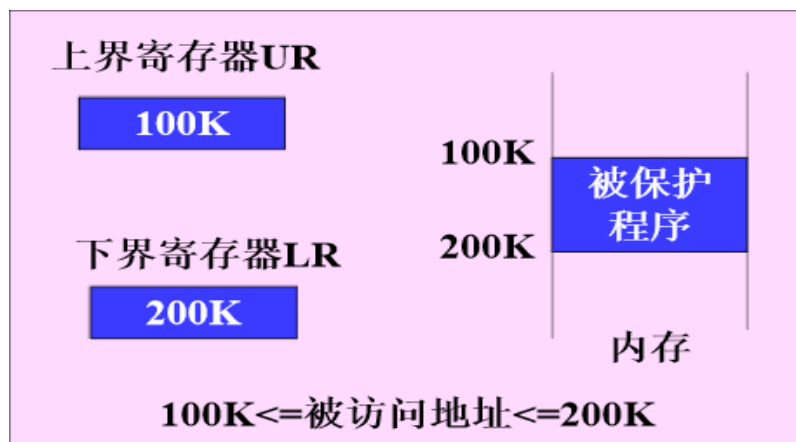


8.1 存储管理的功能及目标

8.1.7 内存信息的共享与保护

(1) 界限保护

- 设置一对寄存器：上界寄存器/下界寄存器，或基址寄存器/限长寄存器
- 所有访问地址必须在上下界之间；每个进程都有自己独立的进程空间，如果一个进程在运行时所产生的地址在其地址空间之外，则发生地址越界。当程序要访问某个内存单元时，由硬件检查是否允许，如果允许则执行，否则产生地址越界中断，由OS进行相应处理。





8.1 存储管理的功能及目标

8.1.7 内存信息的共享与保护

(2) 访问方式保护（保护键）

- 通过**保护键匹配**来判断**存储访问方式**是否合法。
- 对于允许多个进程共享的存储区域，每个进程都有自己的**访问权限**
- 如果一个进程对共享区域的访问违反了**权限规定**，则发生**操作越权** (即读写保护)
- 对每个**内存区域**指定一个**键值**和若干禁止的访问方式，**进程中也指定键值**，如果访问时键值不匹配而且是被禁止的访问方式，则出错





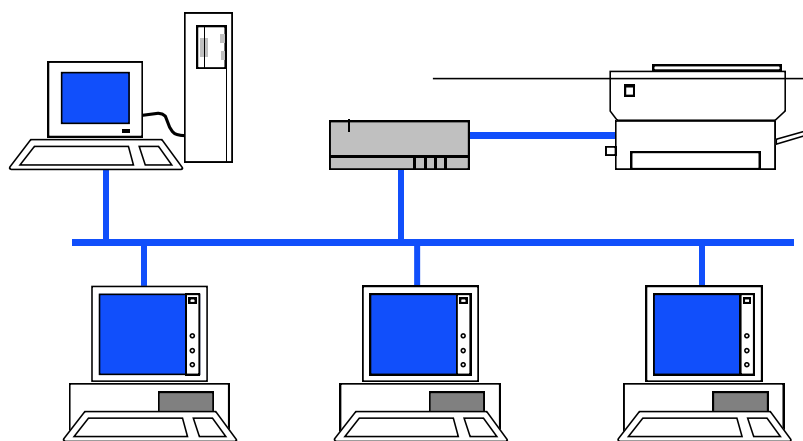
8.1 存储管理的功能及目标



8.1.7 内存信息的共享与保护

(3) 界限寄存器与CPU状态(用户态、核心态)相结合:

- 用户态进程只能访问界限寄存器所规定范围内的内存;
- 核心态进程可以访问整个内存地址空间





Review



- **存储器的分层**：速度、容量、价格
- **存储器的抽象**：一个字节数组
- **虚拟存储器**：部分内存+部分外存、内外可交换、虚拟地址空间可比实际物理空间大；程序的代码和数据构成的空间
- **基本概念**：虚拟、虚地址、虚拟地址空间、物理地址空间、地址重定位、链接
- **存储管理的功能**
 - **内存的分配和回收**：数据结构、放置、换入、交换、回收
 - **地址变换**：静态重定位、动态重定位
 - **内存扩充**：覆盖技术、虚拟存储器技术
 - **内外存数据传输的控制**：交换、请求调入/与调入
 - **内存信息的共享与保护**：地址界限法、保户键法
- **程序链接**：静态链接、装入时链接、运行时链接

1.虚拟存储器的最大容量是由()决定的.

- ☒ A 计算机系统的地址结构和外存空间
- ☐ B 页表长度
- ☐ C 内存空间
- ☐ D 逻辑空间

提交

2.存储管理的目的是（ ）。

- ☐ A 方便用户
- ☐ B 提高内存利用率
- ☒ C 方便用户和提高内存利用率
- ☐ D 增加内存实际容量

提交

3. 逻辑地址转变为内存的物理地址的过程称作（）。

- ☐ A 编译
- ☐ B 连接
- ☐ C 运行
- ☒ D 重定位或地址映射

提交



8.2 分区存储管理

8.2.1 分区管理的基本原理

(1) **原理**: 把内存分为一些大小相等或不等的**分区** (partition), **每个进程占用一个或几个分区**。OS占用其中一个分区。**或者**给内存中的每个进程划分一个适当大小的存储区, 以**连续存放**进程的程序和数据。

✓ **要考虑的问题**: 什么时候划分、怎么划分?

● **类型**: 固定式分区、动态式分区

● **特点**: 适用于多道程序系统和分时系统

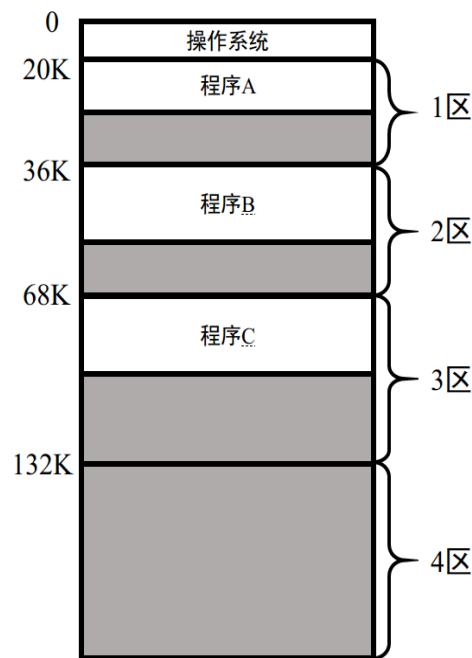
✓ 支持多个程序并发执行

✓ 难以进行内存分区的共享

● **问题**: 可能存在内碎片和外碎片

✓ **内碎片**: 占用分区内未被利用的空间

✓ **外碎片**: 占用分区间难以利用的空闲分区
(通常是空闲分区)





8.2 分区存储管理

8.2.1 分区管理的基本原理

(2) 分区用的数据结构：分区说明表或分区链表

- ✓ 可以只记录空闲分区，也可以同时记录空闲和占用分区
- ✓ 分区表中，表项数目随着内存的分配和释放而动态改变，可以规定最大表项数目。分区链表如何组织？
- ✓ 分区表可以划分为两个表格：空闲分区表，占用分区表。
空闲分区表中按不同分配算法相应对表项排序

空闲分区表

区号	大小	起始地址	标志
1	128k	132k	占用
2			未占用
...			

占用分区表

区号	大小	起始地址	标志
1	16k	20k	占用
2	32k	36k	占用
...	64k	68k	占用



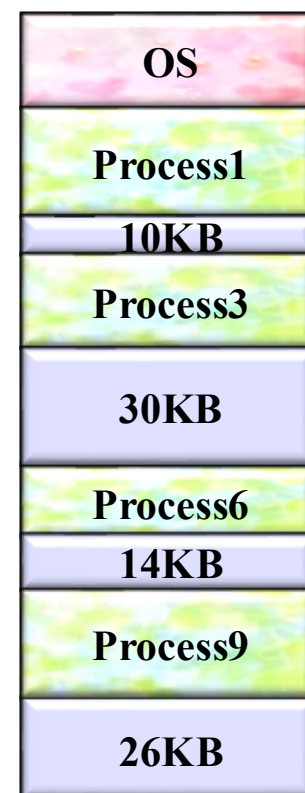
8.2 分区存储管理



8.2.1 分区管理的基本原理

(3) 内存紧缩(compaction)

- 为什么要紧缩?
- 定义: 将各个占用分区向内存一端移动, 使各个空闲分区聚集在另一端, 然后将各个空闲分区合并成为一个空闲分区
- 特点:
 - 对占用分区进行内存数据搬移占用CPU时间
 - 如果对占用分区中的程序进行“浮动”, 则其重定位需要硬件支持
- 紧缩时机(回答):
 - 每个分区释放后, 如果没有邻接空闲区但内存中有其他空闲区时, 则马上进行拼接; 或
 - 内存分配找不到满足条件的空闲分区时, 而所有空闲区总容量却能满足需求量时, 再进行拼接

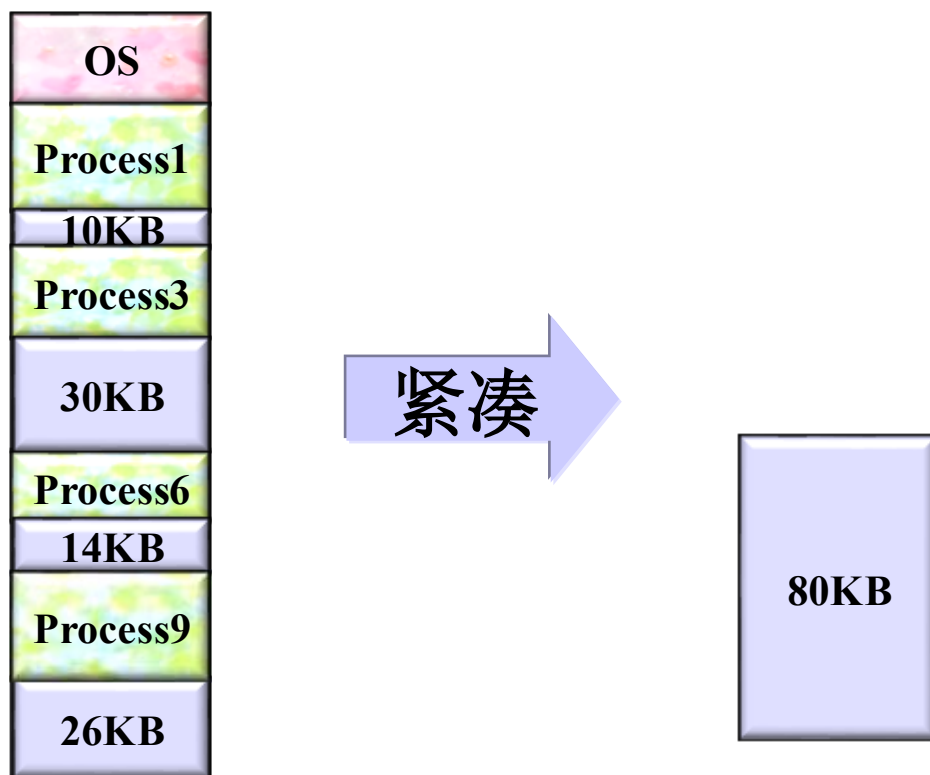




8.2 分区存储管理

8.2.1 分区管理的基本原理

(1) 内存紧缩(compaction)



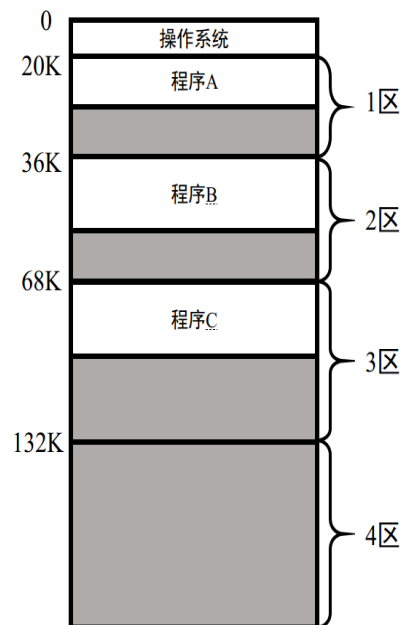


8.2 分区存储管理



8.2.2 固定分区 (Fixed Partitioning)

- **划分策略**: 把内存区**固定地**划分为若干个大小**相等** (或不等) 的连续分区。每个分区装一个且只能装一个作业。分区一旦划分, 整个执行过程中每个分区的长度和内存中总的分区数不变。
- **时机**: 何时确定分区数目和大小?
- **放置策略**: 将**每个进程**指定到适应它的**最小分区**
- **数据结构**: 分区说明表, 记录分区的大小和使用情况



分区号	起始地址	长度	状态	进程名
-----	------	----	----	-----



8.2 分区存储管理

8.2.2 固定分区 (fixed partitioning)



固定分区
(相等)



固定分区
(不等)



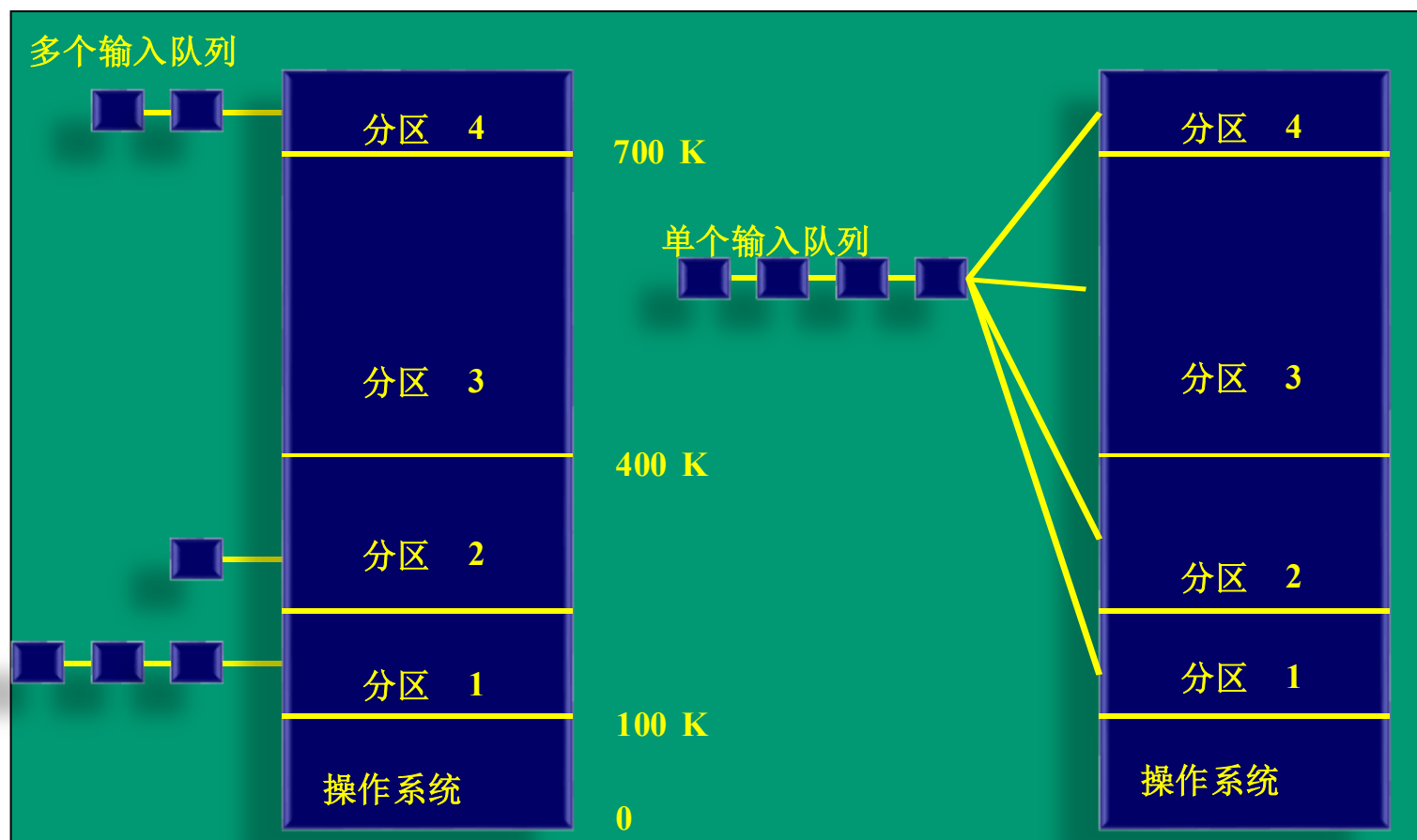
8.2 分区存储管理



8.2.2 固定分区 (fixed partitioning)

进程的放置算法

- 固定分区的进程队列：多队列、或单队列





8.2 分区存储管理



8.2.2 固定分区 (fixed partitioning)

● 固定分区的评价

- 优点：易于实现，开销小
- 缺点
 - 内碎片造成浪费(小作业不能有效地利用分区空间)
 - 分区总数在系统生成时确定（固定），限制了并发执行的程序数目
- 可以与覆盖、交换技术配合使用



8.2 分区存储管理

8.2.3 动态分区(dynamic partitioning)

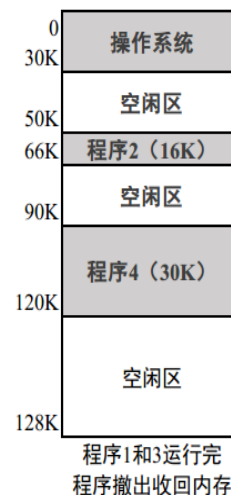
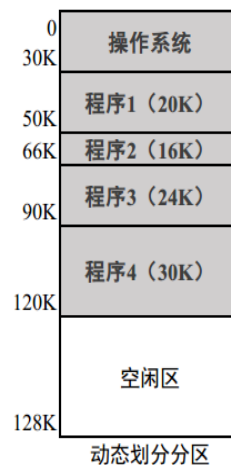
● **基本思想：** 内存分区不是预先划分好的，而是当程序装入时，根据程序的需求和内存空间的使用情况来决定是否分配：

- ✓ 若有足够的空间，则按需要分割一部分内存给该进程，否则令其等待主存空间。
- ✓ 初始时是一个大的连续空闲区

● ? 系统运行一段时间后，内存分区是什么样的

● **评价**

- ✓ 优点：没有内碎片
- ✓ 缺点：有外碎片



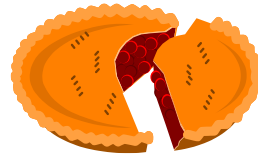
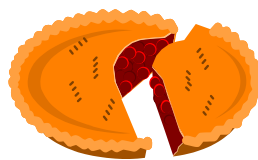
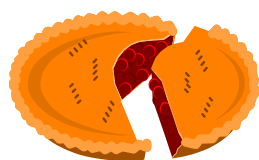
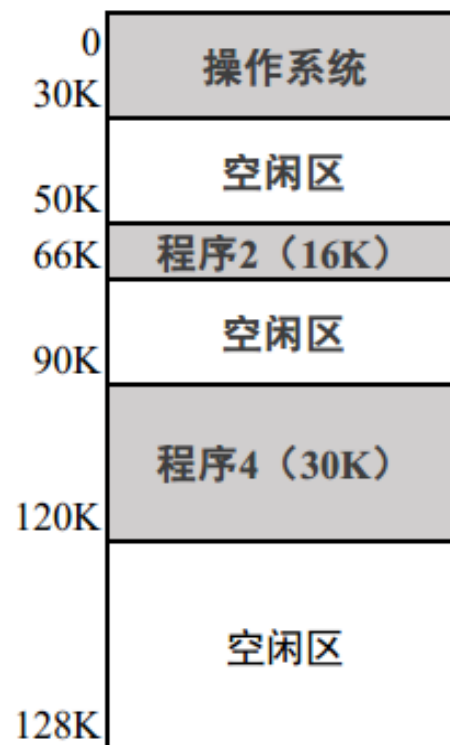


8.2 分区存储管理



8.2.3 动态分区(dynamic partitioning)

- 内存管理：设置**内存空闲块表**，记录了空闲区起始地址和长度
- 内存分配：动态按需分配
- 内存回收：当某一块归还后，前后空间**合并**，修改内存空闲区表，例如程序2回收时，会将其与其上、其下的空闲区合并，形成一个大的空闲区





8.2 分区存储管理

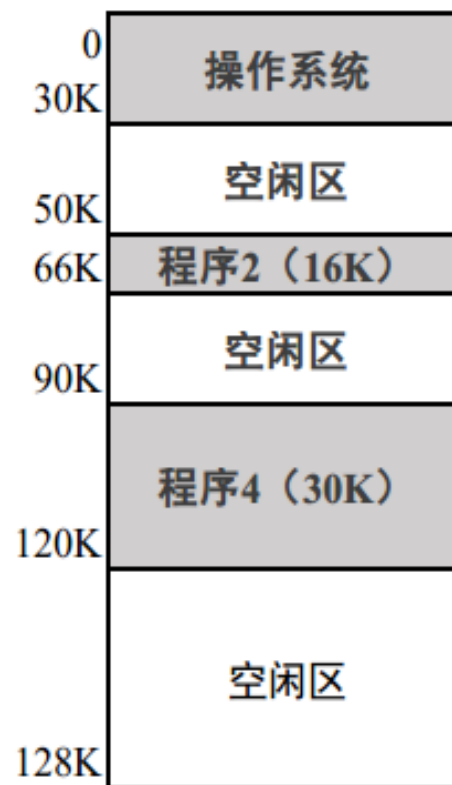


8.2.4 动态分区分配算法

- **分区分配算法**：寻找某个空闲分区，其大小需大于或等于程序的要求。若是大于要求，则将该分区分割成两个分区，其中一个分区为要求的大小并标记为“占用”，而另一个分区为余下部分并标记为“空闲”。分区的先后次序通常是从内存低端到高端。

✓ **思考**：如果有多个分区满足条件，如何选择？

- **分区释放算法**：需要将相邻的空闲分区合并成一个空闲分区。





8.2 分区存储管理

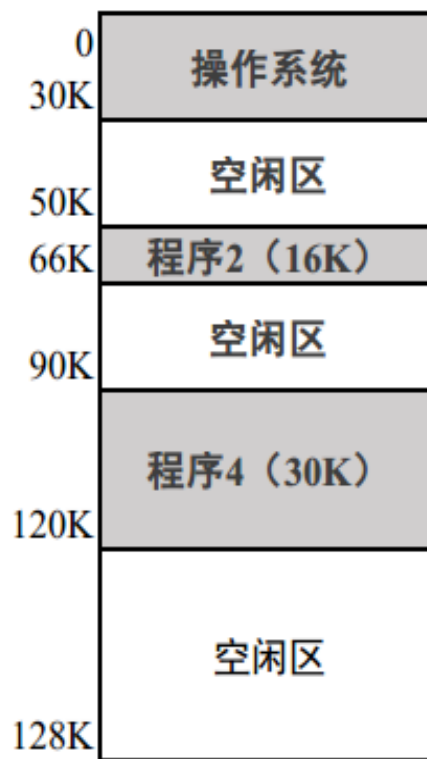
8.2.4 动态分区分配算法

- **最先匹配法(first-fit)**: 按分区的先后次序(分区按起始地址递增的顺序排列), 从头查找, 找到符合要求的第一个分区
 - 例: 一个20k的进程, 如何放置?
- 为快速分配, 空闲分区表如何组织?
- 特点
 - **简单、快速分配**。该算法的分配和释放的时间性能较好, 较大的空闲分区可以被保留在内存高端
 - 但随着低端分区不断划分而产生较多小分区, 每次分配时查找时间**开销会增大**

思考:

1. 这个表多长合适?
2. 有没有更省空间的管理方法?

空闲分区表			
区号	起始地址	大小	标志
1	30k	20k	占用
2	66k	24k	占用
3	120k	128k	占用



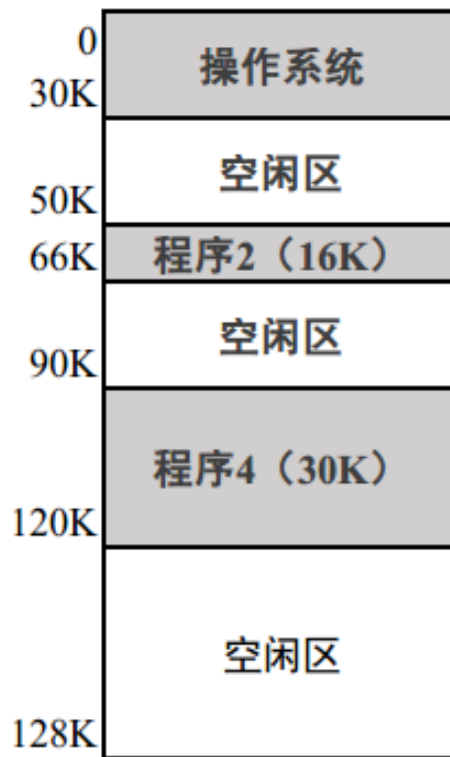


8.2 分区存储管理

8.2.4 动态分区分配算法

- 下次匹配法(next-fit): 按分区的先后次序, 从上次分配的分区起查找 (到最后分区时再回到开头), 找到符合要求的第一个分区
- 例: 一个20k的进程, 如何放置?
- 特点: 该算法的分配和释放的时间性能较好, 使空闲分区分布得更均匀, 但较大的空闲分区不易保留

空闲分区表			
区号	起始地址	大小	标志
1	30k	20k	占用
2	66k	24k	占用
3	120k	128k	占用



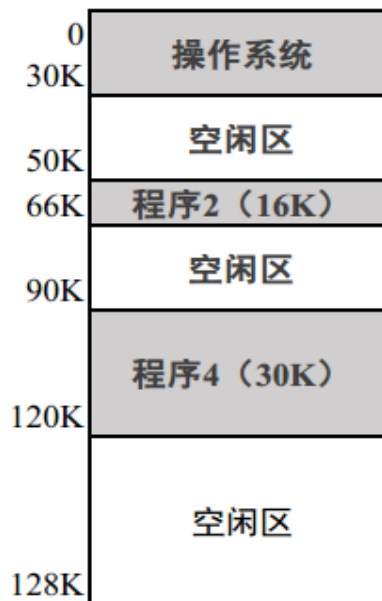


8.2 分区存储管理



8.2.4 动态分区分配算法

- **最佳匹配法(best-fit):** 找到一个空闲分区, 其大小与要求的大小相差最小, 按空闲区大小从小到大的顺序排列
 - 从个别来看, 外碎片较小, 但从整体来看, 会形成较多外碎片。较大的空闲分区可以被保留。(用最小空间满足要求)
- **最坏匹配法(worst-fit):** 找到最大的空闲分区
 - **为快速找到合适分区, 空闲分区说明表表项如何组织?**
 - 按空闲区大小从大到小的顺序排列
 - 基本不留下小空闲分区(当分割后空闲块仍为较大空块), 但较大的空闲分区不被保留



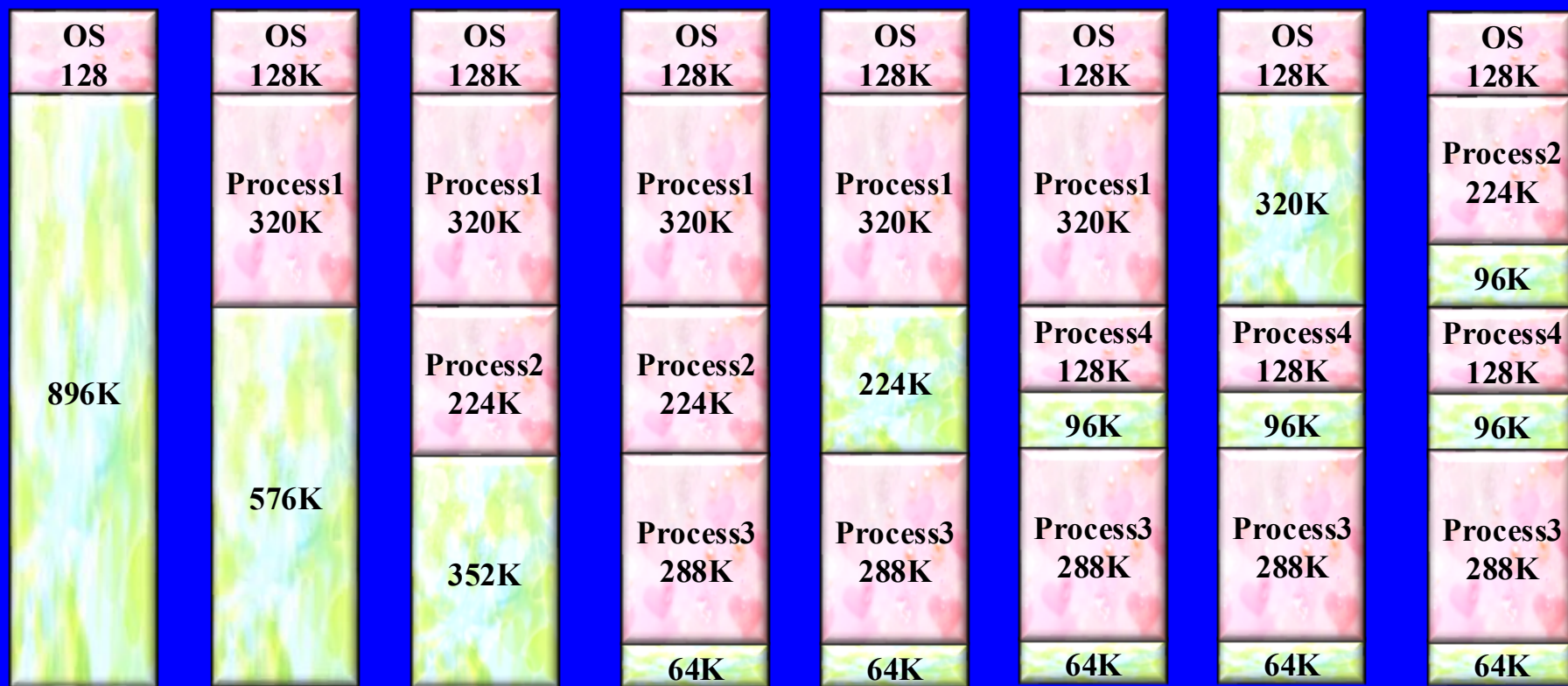


8.2 分区存储管理

8.2.4 动态分区分配算法

注意数据结构（分区表）的变化

采用最先适应法的内存分配变化过程



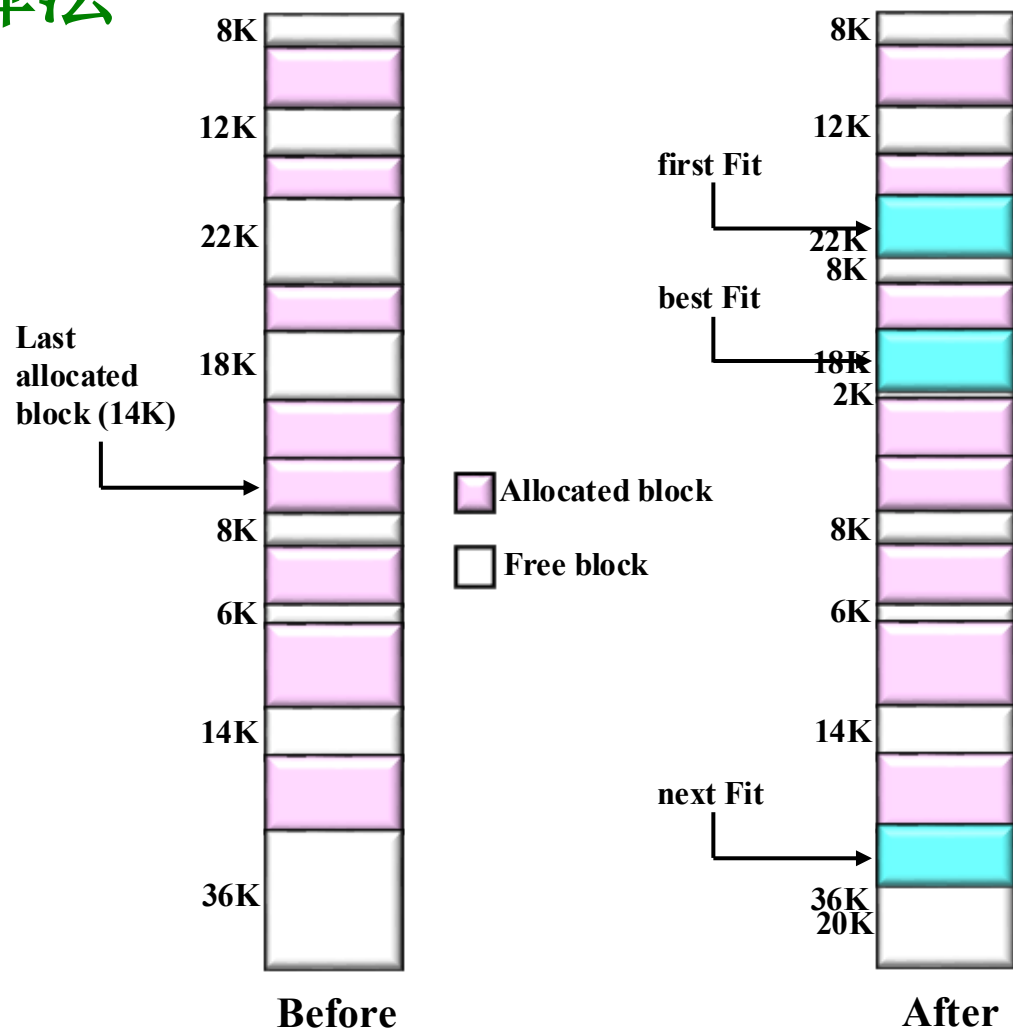


8.2 分区存储管理

8.2.4 动态分区分配算法



各种适应法的内存
分配变化过程
(分配16K进程)



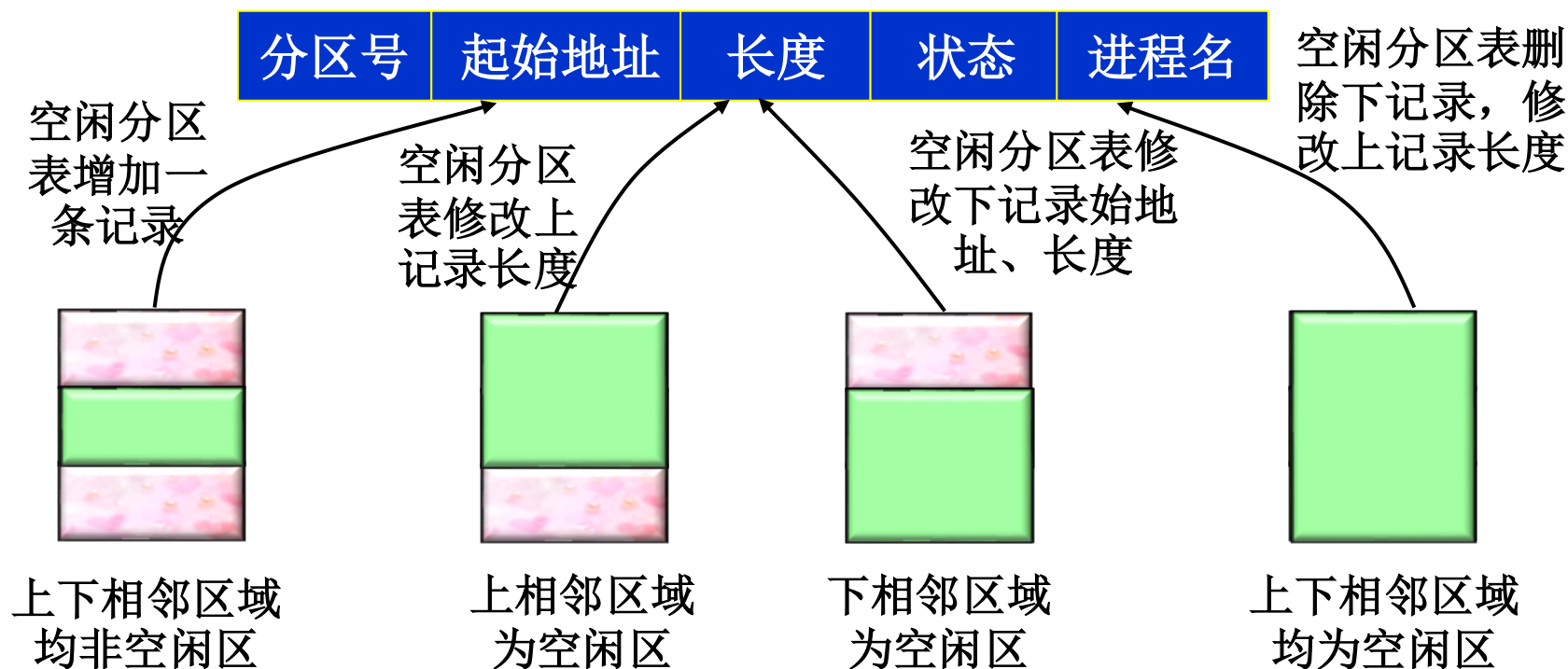


8.2 分区存储管理

8.2.4 动态分区分配算法

● 动态分区时的回收与拼接

占用 空闲



注意空闲分区表中记录的变化



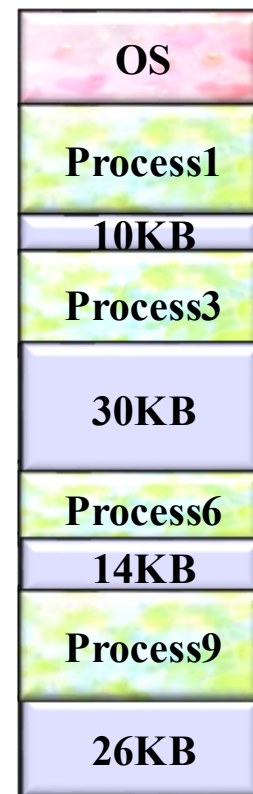
8.2 分区存储管理



8.2.4 动态分区分配算法

● 碎片问题

- 经过一段时间的分配回收后，内存中存在很多很小的空闲块。它们每一个都很小，不足以满足分配要求；但其总和满足分配要求。这些空闲块被称为**碎片**
- **紧凑技术**：通过在内存**移动**程序，将所有小的空闲区域合并为大的空闲区域





8.2 分区存储管理



8.2.5 分区管理的主要优缺点

● 优点：简单

● 缺点

- 碎片问题（外碎片）
- 内存利用率不高
- 受分区大小或实际内存容量限制
- 不利于程序段和数据段的共享



8.2 分区存储管理



8.2.6 MS DOS中的分区存储管理

- DOS提供**动态分区**管理
- 通过DOS功能调用int 21h，支持分区的**创建**(48h)、**释放**(49h)和**改变分区大小**(4Ah)
- 设置或查询分区的**分配策略**(58h)
 - 最先匹配法
 - 最佳匹配法
 - 最后匹配法（last-fit，从内存高端向低端查找）



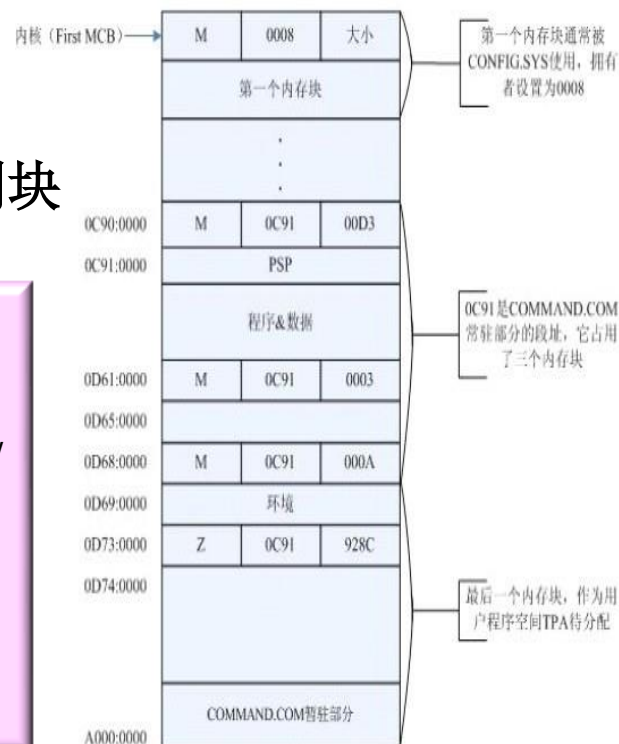
8.2 分区存储管理



8.2.6 MS DOS中的分区存储管理

- 当程序被加载的时候，需要向物理内存申请空间并加载程序。MS-DOS采用了内存控制块（MCB）来标识物理内存块。DOS的内存块以节为单位，一节等于16个字节。
- 每个分区以一个MCB(Memory Control Block)结构开始，MCB占16个字节，按段边界对齐（起始地址可被16整除）
- MCB块链：MCB块地址+内存块大小+1就能知道下一个MCB块的地址。
- PSP(Program Segment Prefix)：起进程控制块的作用，其起始地址可作为进程ID

```
typedef struct {  
    BYTE type; /* 'M' = in chain; 'Z' = at end */  
    WORD owner; /* PSP of the owner process, 0 = free */  
    WORD size; /* in 16-byte paragraphs */  
    BYTE unused[3];  
    BYTE dos4[8]; /* program name(DOS 4.x) */  
} MCB;
```



1.分区管理方式中，当内存碎片容量大于某一作业所申请的内存容量时，（ ）。

- ☐ A 可以为这一作业分配内存
- ☐ B 不可以为这一作业分配内存
- ☒ C 拼接（或者紧凑）后，可以为这一作业分配内存
- ☐ D 一定能够为这一作业分配内存

提交

2. 在 ()，要求空闲分区按空闲区地址递增顺序链接成空闲分区链；在 () 中是按空闲区大小递增顺序形成空闲分区链；在 () 中，是按空闲区大小递减的顺序形成空闲分区链。

A

首次适应算法

B

最坏适应算法

C

最佳适应算法

D

循环首次适应算法

提交



8.3 覆盖和交换技术



- 在多道环境下扩充内存的方法，用以解决在较小的存储空间中运行较大程序时遇到的矛盾
 - 覆盖技术主要用在早期的操作系统中
 - 交换技术被广泛用于小型分时系统中，交换技术的发展导致了虚存技术的出现
- 覆盖(overlay)
- 交换(swapping)



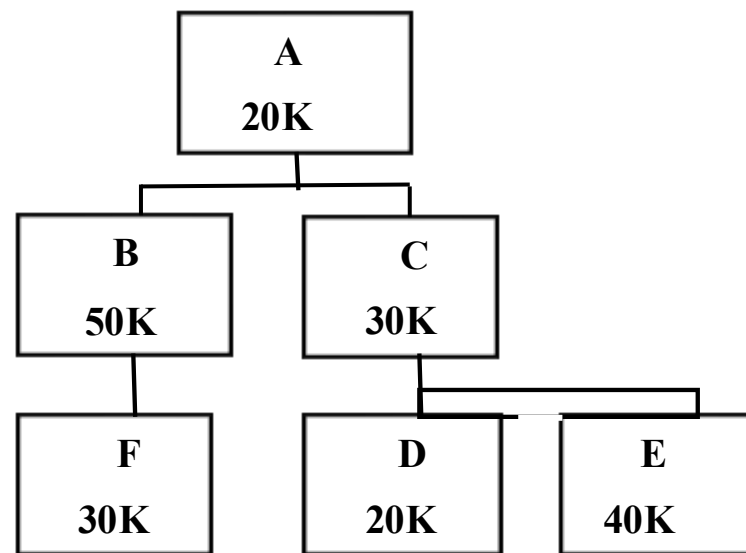
8.3 覆盖和交换技术



8.3.1 覆盖(overlay)

- **引入**：其目标是在较小的可用内存中**运行较大的程序**。常用于多道程序系统，与分区存储管理配合使用
- **原理**：把程序划分为若干个**功能上相对独立的程序段**，按照其自身的逻辑结构将那些不会同时执行的程序段共享同一块内存区域。程序段先保存在磁盘上，当有关程序段的前一部分执行结束，程序员把后续程序段调入内存，覆盖前面的程序段(客观上内存“扩大”了)

- ✓ 将程序的**必要部分**（常用功能）的代码和数据常驻内存
- ✓ **可选部分**（不常用功能）在其他程序模块中实现，平时存放在外存中（覆盖文件），在需要用到时才装入到内存
- ✓ 不存在调用关系的模块不必同时装入到内存，从而可以相互覆盖。(即**不同时用的模块可共用一个分区**)

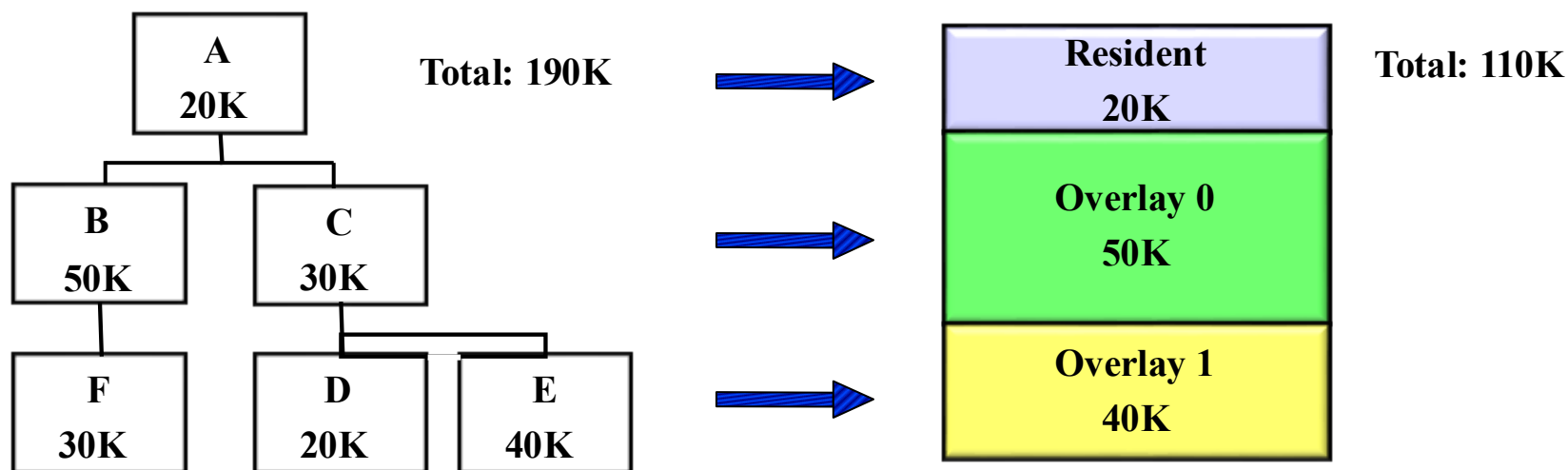




8.3 覆盖和交换技术



8.3.1 覆盖(overlay)



注：另一种覆盖方法：(100K)

- A(20K)占一个分区：20K；
- B(50K)、D(20K)和E(40K)共用一个分区：50K；
- F(30K)和C(30K)共用一个分区：30K；



8.3 覆盖和交换技术



8.3.1 覆盖(overlay)

● 缺点

- 对用户不透明，增加了用户负担。编程时必须划分程序模块和确定程序模块之间的覆盖关系，增加编程复杂度
- 从外存装入覆盖文件，以时间延长来换取空间节省
- 覆盖不需要任何来自OS的特殊支持，所以覆盖通常限于用在微机和其他内存容量有限的或缺乏对更先进技术的硬件支持的系统中，例如MS-DOS



8.3 覆盖和交换技术



8.3.2 交换(swapping)

- 引入：多个程序并发执行，可以将暂时不能执行的程序送到外存中，从而获得空闲内存空间来装入新程序，或读入保存在外存中而满足条件的进程。交换单位为整个进程的地址空间
- 常用于多道程序系统或小型分时系统中，与分区存储管理配合使用，又称作“对换”或“滚进/滚出(roll-in/roll-out)”
 - 程序暂时不能执行的可能原因：处于阻塞状态，低优先级（确保高优先级程序执行）

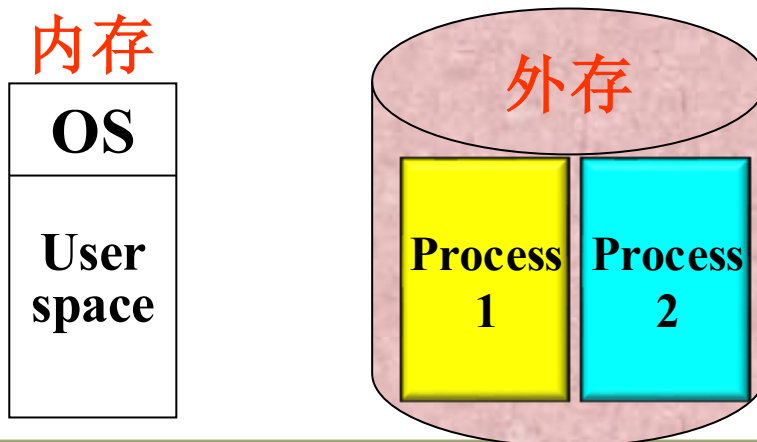


8.3 覆盖和交换技术



8.3.2 交换(swapping)

- 原理：暂停执行内存中的进程，将整个进程的地址空间保存到外存的交换区中（换出swap out），而将外存中由阻塞变为就绪的进程的地址空间读入到内存中，并将该进程送到就绪队列（换入swap in）

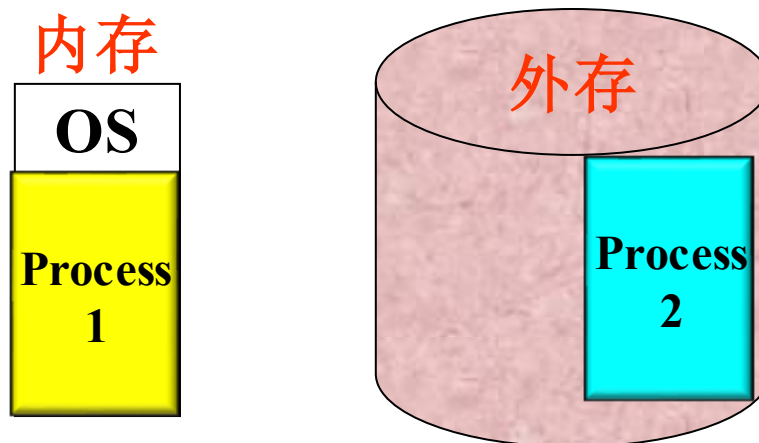




8.3 覆盖和交换技术

8.3.2 交换(swapping)

- 原理：暂停执行内存中的进程，将整个进程的地址空间保存到外存的交换区中（换出swap out），而将外存中由阻塞变为就绪的进程的地址空间读入到内存中，并将该进程送到就绪队列（换入swap in）





8.3 覆盖和交换技术



8.3.2 交换(swapping)

● 优点

- 增加并发运行的程序数目，并且给用户适当的响应时间
- 编写程序时不影响程序结构

● 缺点

- 对换入和换出的控制增加处理机开销；程序整个地址空间都进行传送
- 没有考虑执行过程中地址访问的统计特性



8.4 页式和段式存储管理

- 引入：
 - 分区式存储管理不能根本上解决内存扩充问题
 - 分区式存储管理需要作业装入连续的内存空间
 - 内存利用率不高：存在内碎片或外碎片的问题
- 页式和段式存储管理是通过引入进程的逻辑地址，把进程地址空间与实际存储位置分离，从而增加存储管理的灵活性，提高内存的利用率
 - 简单页式(simple paging)
 - 简单段式(simple segmentation)
 - 页式管理和段式管理的比较



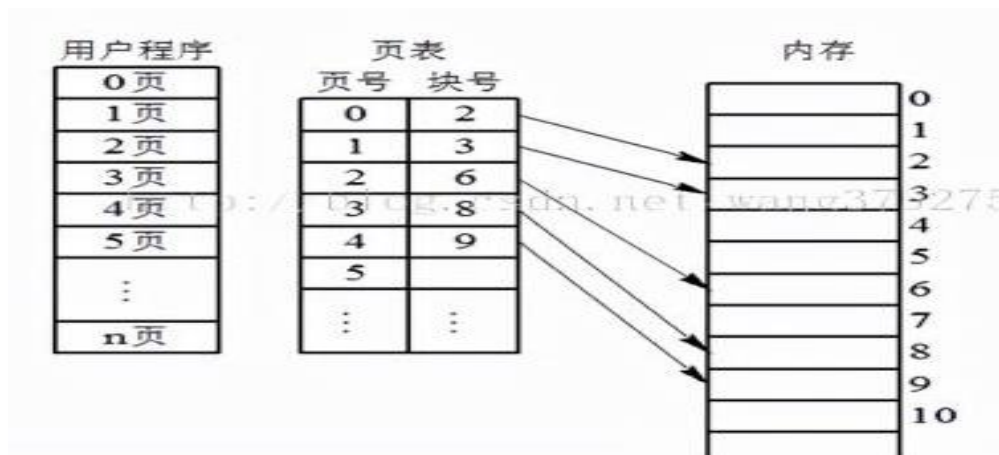
8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

(1) 简单页式管理的基本原理

- ✓ 将程序的逻辑地址空间和物理内存划分为**固定大小**的**页或页面**(page or page frame)
- ✓ 程序加载时，分配其所需的所有页，这些页**不必连续**
- ✓ **建立页表**：记录逻辑页与物理页面的对应关系
- ✓ 需要**CPU**的硬件支持实现地址映射





8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

● 用户程序划分

- 把用户程序按逻辑页划分成大小相等的部分，称为页。从0开始编制页号，页内地址是相对于0编址

逻辑地址



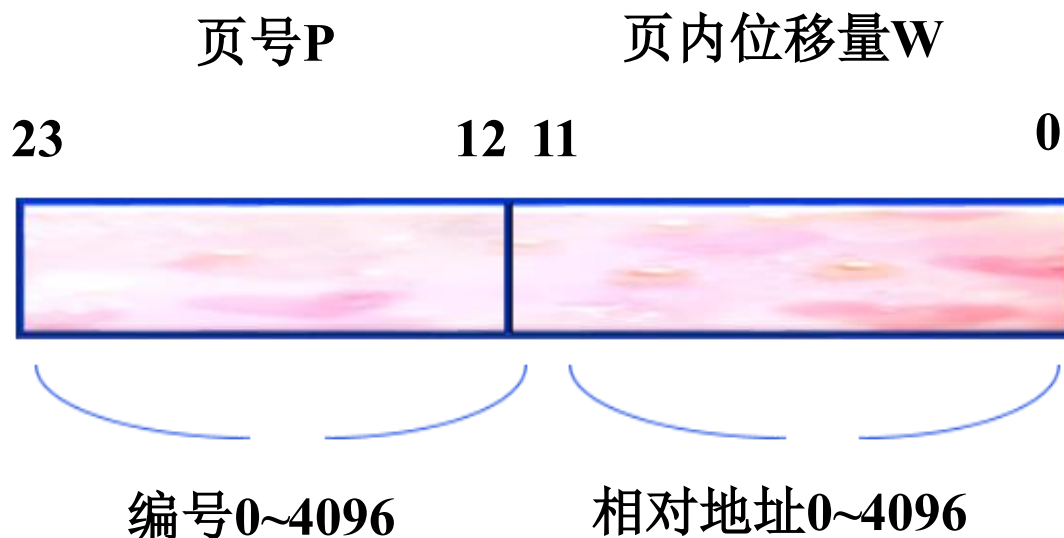
- 用户程序的划分是由系统自动完成的，对用户是透明的。一般，一页的大小为2的整数次幂，例如4k，逻辑地址的高位部分为页号，低位部分为页内地址
 - 例如：0xb8004000，



8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

● 用户程序划分





8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

● 内存空间

- 按页的大小划分为大小相等的块，称为内存块（又叫物理页面，简称页面）



● 内存分配

- 以页为单位进行分配，并按进程的页的数量来分配。
逻辑上相邻的页，物理上不一定相邻



8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

● 优点

- 每个碎片不超过页大小
- 一个程序不必连续存放
- 便于改变程序占用空间的大小。即随着程序运行而动态生成的数据增多，地址空间可相应增长

● 缺点

- 程序需全部装入内存



8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(2) 简单页式管理的数据结构

➤ 进程页表：每个进程有一个页表，描述该进程占用的物理页面及逻辑排列顺序

● 逻辑页号（本进程的地址空间）→物理页面号（实际内存空间）

➤ 物理页面表：整个系统有一个物理页面表，描述物理内存空间的分配使用状况

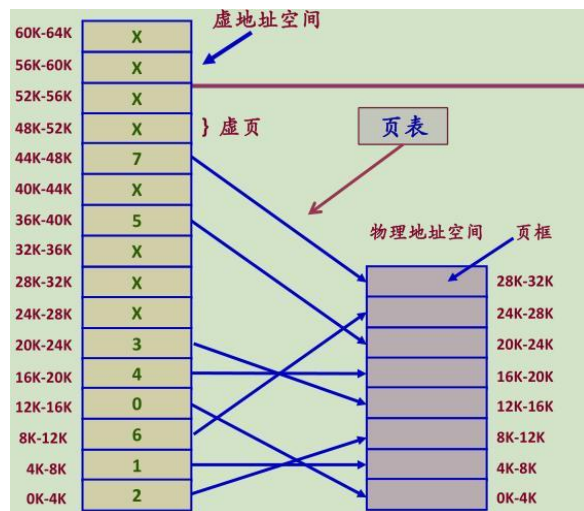
● 位示图

● 空闲页面链表

● 伙伴算法(Buddy)

页表

页号	页面号





8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(2) 简单页式管理的数据结构

- **请求表**：整个系统有一个请求表，描述系统内各个**进程页表的位置和大小**，用于地址转换，另外，还包括各进程所要求的页面数。也可以结合到各进程的PCB里

页表

页号	页面号

请求表

进程号	请求页面数	页表始址	页表长度	状态

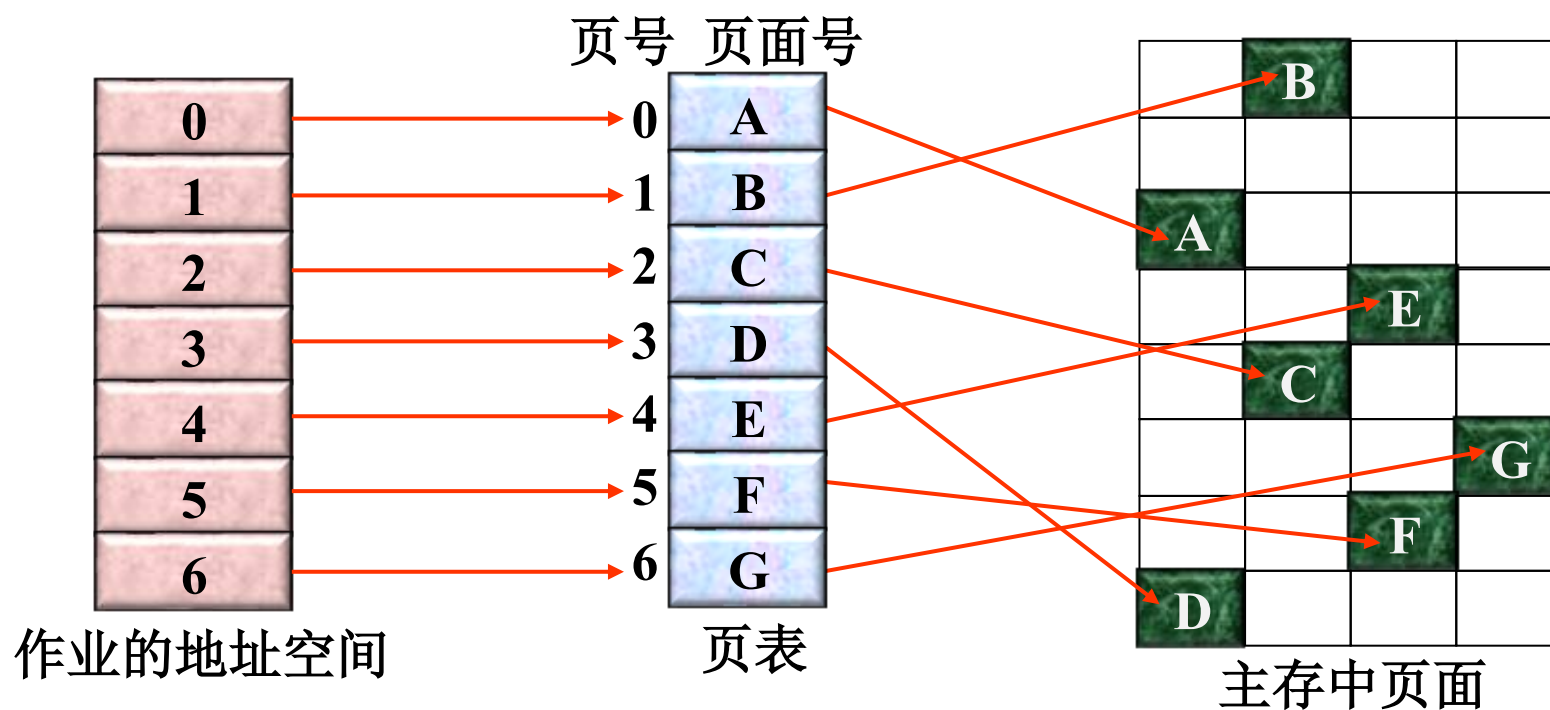


8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

(2) 简单页式管理的数据结构





8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

(2) 简单页式管理的数据结构

Frame No 15个可用页面

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

加载进程A

0	A.0
1	A.1
2	A.2
3	A.3
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

0	0
1	1
2	2
3	3

Process A

加载进程B

0	A.0
1	A.1
2	A.2
3	A.3
4	B.0
5	B.1
6	B.2
7	
8	
9	
10	
11	
12	
13	
14	

0	4
1	5
2	6

Process B

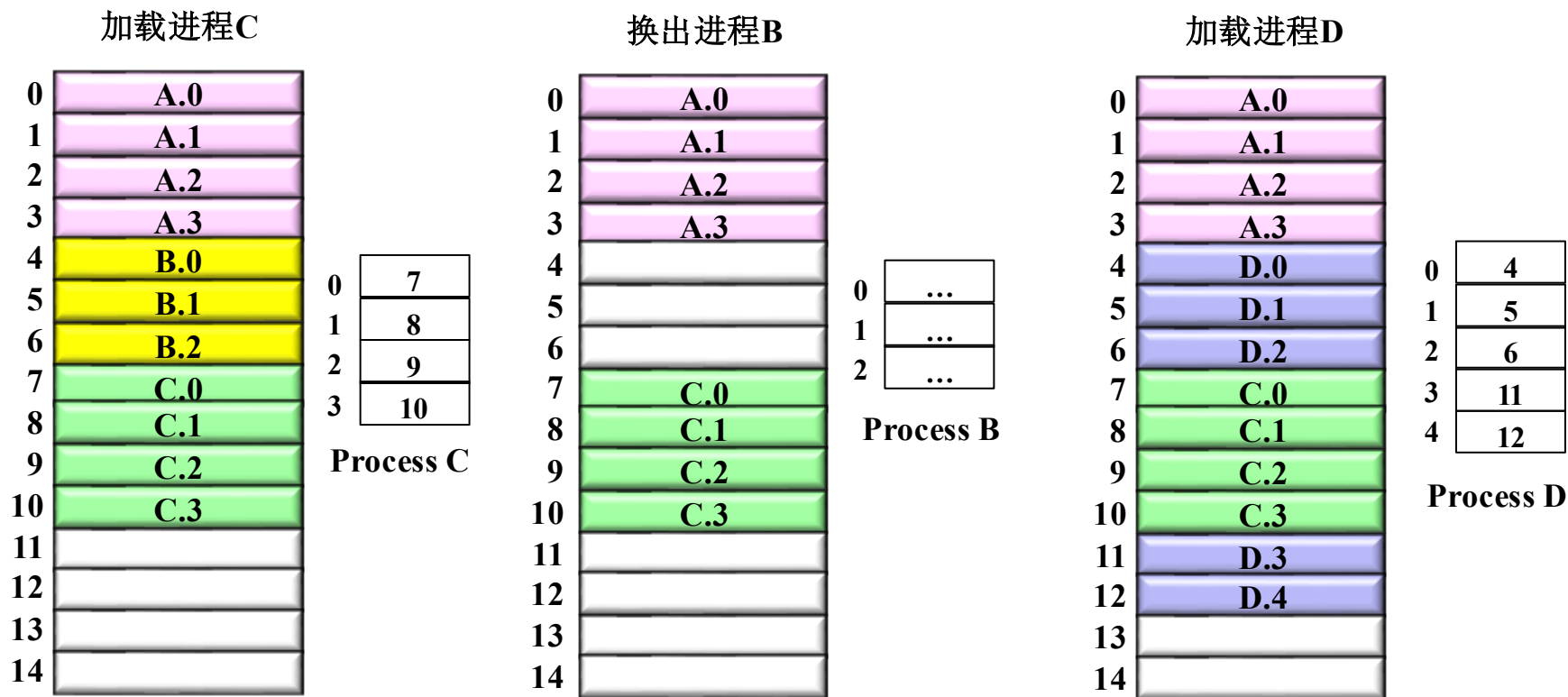


8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

(2) 简单页式管理的数据结构



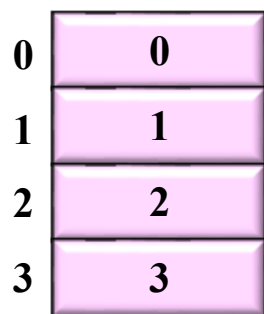


8.4 页式和段式存储管理

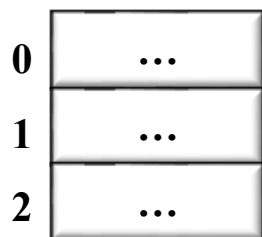


8.4.1 简单页式(simple paging)

(2) 简单页式管理的数据结构



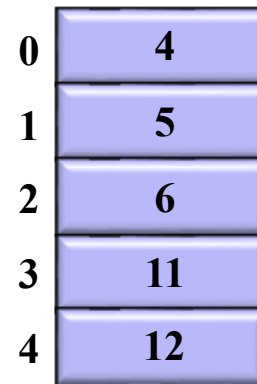
Process A



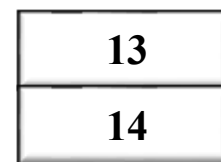
Process B



Process C



Process D



Free Frame List



8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(3) 页面分配算法

- 给出一个进程所需要的总页面数（请求表） N
- 查物理页面表，看是否还有 N 个空闲页面
 - 如果有相应空闲页面（块），则页表长度为 N ，（申请页表区，把页表始址填入请求表）
 - 分配 N 个空闲页面，将页面号和页号填入页表
 - 修改物理页面表
 - 否则，进程等待
- **思考：**这个算法的开销在哪里？如何优化？



8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(2) 基于位示图(bitmap)的页面管理的算法

- 开辟一块空间，用来管理空闲页面块，需要多少空间呢？
 - 例如：4G物理地址空间，4kB页长，位示图多少字节？
- 位示图的每一位对应相应物理块占用情况：0--空闲，1--已占用

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	0	0	0	1	1	1	0	0	1	0	1	1	1	0
1	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1	1
2	1	1	1	0	0	0	1	1	1	1	1	1	0	0	0	0
3																
⋮																
15																

位示图

➤ 算法:

- 块号n在哪个字节（或字）哪个位？
- 第i字节的第j位，对应的块号？
- i, j, n都是从0开始的

已知块号n求i行和j列？

$$i = n / (\text{位宽})$$

$$j = n \bmod (\text{位宽})$$

已知第i行的j列，块号n多少？

$$n = i * \text{位宽} + j$$

注：这里的位宽可以是8、16、32；相应的j的取值上限分别为7、15、31

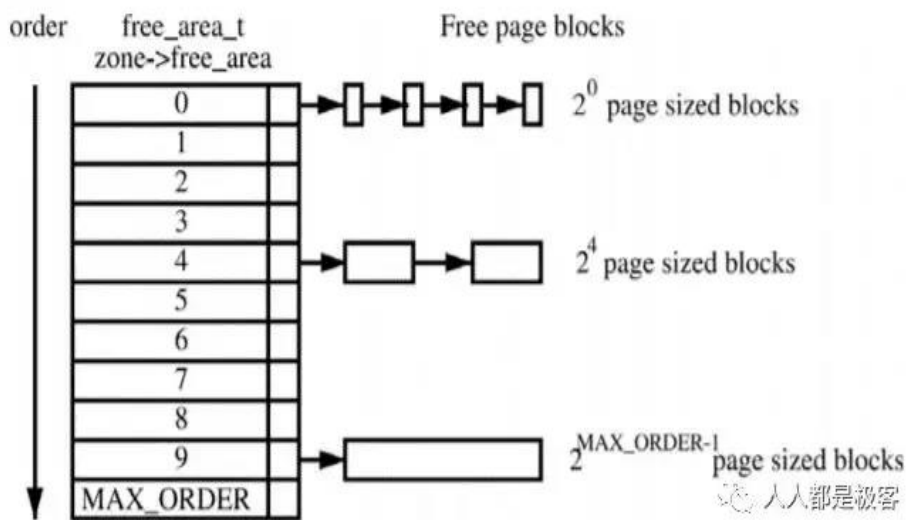
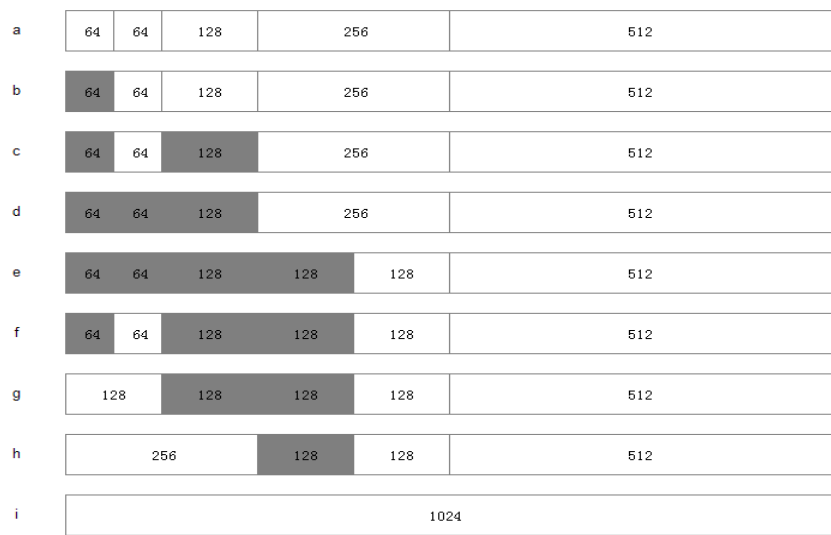


8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(2) 基于Buddy算法的页面管理

- **思想：**把所有空闲页框，按照空闲页块中页框数目，分为11组（MAX_ORDER），形成11个空闲**块链表**，每个块链表分别包含大小为1, 2, 4, 8, ..., 256, 512, 1024个连续的页框。
- **伙伴：**一个被分配块单元平均拆分成两个大小是原来一半的块单元，这两个块单元互为伙伴。来自同一个大块的两个小块才是伙伴。
- **算法：**分配时，找最接近的链表查找，没有就找上一阶链表，直至找到，分配需要的页数，剩余的插入相应的阶链表中；回收时若找到了空闲的伙伴，需要合并。





Review



- 分区式存储管理

- 原理：固定式分区、动态分区，一次性装入连续的内存
- 数据结构：空闲分区说明表、分区占用说明表
- 放置算法：FF、BF、NF、WF，空闲分区表的表项顺序
- 紧凑技术（紧缩技术）

- 覆盖与交换技术

- 覆盖：根据程序结构，让不存在调用关系的模块共用一块内存
- 交换技术：实现内外存的交换
- 作为分区式管理方式的辅助技术
- 目标：小内存运行大作业

- 简单页式存储管理

- 原理：进程地址空间、内存物理地址空间划分程等长的块，称为页、页面（Page Frame）、一次性装入、内存中可以不连续
- 逻辑地址的逻辑解释：页号+页内偏移
- 数据结构：页表、进程请求表、空闲页面表
- 分配算法：

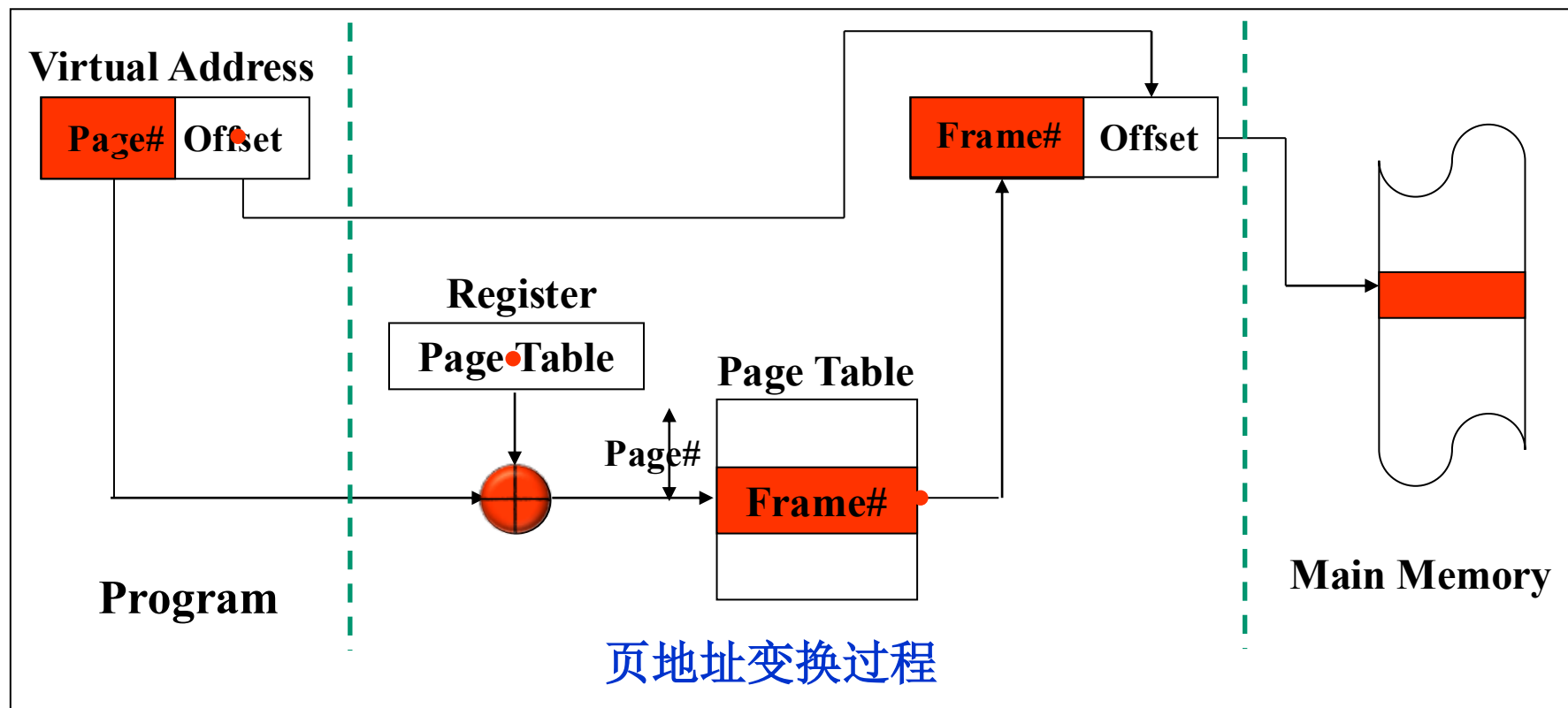


8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(4) 简单页式管理的地址变换

● 问题：怎样由页表将虚拟地址变换到物理地址？



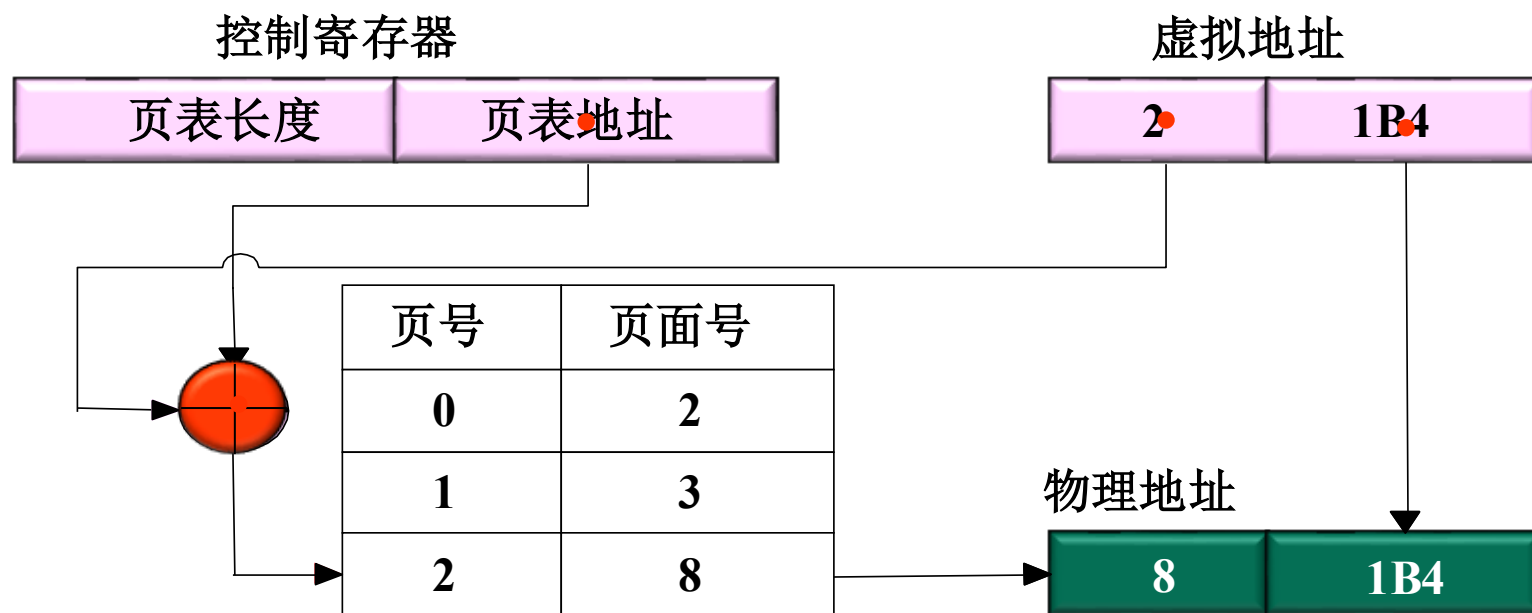


8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(4) 地址变换举例

- 假设页长为4KB，虚拟地址为8628，十六进制是多少？
- 物理地址是多少？





8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

(4) 简单页式管理的地址变换

● 转换过程:

- ✓ 逻辑地址分为两部分: 逻辑页号+页内偏移, 怎么分?
- ✓ 根据页号和页表始址, 查进程页表, 得物理页面号
- ✓ 计算物理地址: 页面号*页长+页内偏移
- ✓ 保护: 页号>进程的最大页号

● 存在的问题及解决方法

● 问题: 取一个数据至少要访问内存2次以上。哪两次?



● 解决方法: 利用关联存储器(高速联想存储器), 构成一张快表(TLB, Translation Lookaside Buffer)。

- ✓ 高速联想存储器中存放那些当前执行进程中最常用的页号及其所对应的页面号

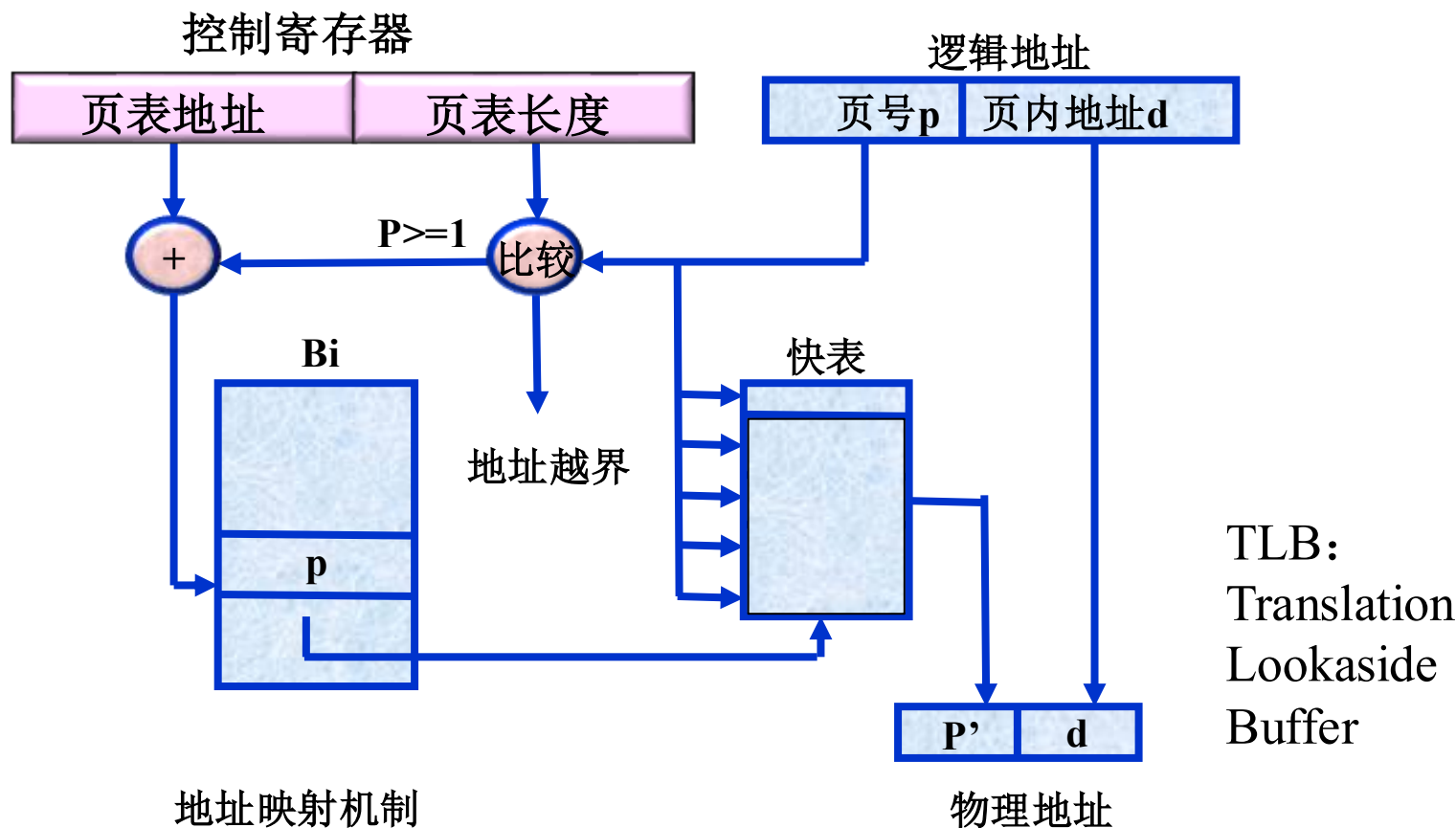


8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(4) 简单页式管理的地址变换

➤ 带快表 (TLB) 的地址变换过程





8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(4) 简单页式管理的地址变换

➤ 进程装入主存（内存分配）

请求表

进程号	请求页面数	页表始址	页表长度	状态
P1	4	...	4	满足
P2	2	...	2	满足
P3	3	80	3	满足

进程P3页表

页号	页面号
A	1
B	7
C	10

进程P3空间

A
B
C

主存空间

	4	8
1	5	
		10
	7	

物理页面表

页面	状态	进程
0	占	P1
1	占	P3
2	占	P2
3	占	P1
4	空	
5	空	
6	占	P2
7	占	P3
8	空	
9	占	P1
10	占	P3
11	占	P1

80
↑
起始地址



8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(4) 简单页式管理的地址变换

● 进程执行（地址变换）

请求表

进程号	请求页面数	页表始址	页表长度	状态
P1	4	...	4	满足
P2	2	...	2	满足
P3	3	80	3	满足

查进程页表，找页号

查请求表，找页表始地址
(可以装入起始地址寄存器)

进程P3页表

页号	页面号
A	2
B	8
C	11

80



起始地址

地址: XXXXX|XXXXXXXX

B

页内地址



地址变换

8

页内地址

物理页面表

页面	状态	进程
1	占	P1
2	占	P3
3	占	P2
4	占	P1
5	空	
6	空	
7	占	P2
8	占	P3
9	空	
10	占	P1
11	占	P3
12	占	P1



8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

(4) 简单页式管理的地址变换

● 地址变换的硬件支持

- 页表始址寄存器
- 页表长度寄存器（判断是否越界）

● 关联寄存器——快表

- 页号
- 页在内存的块号
- 标识位
- 淘汰位



8.4 页式和段式存储管理



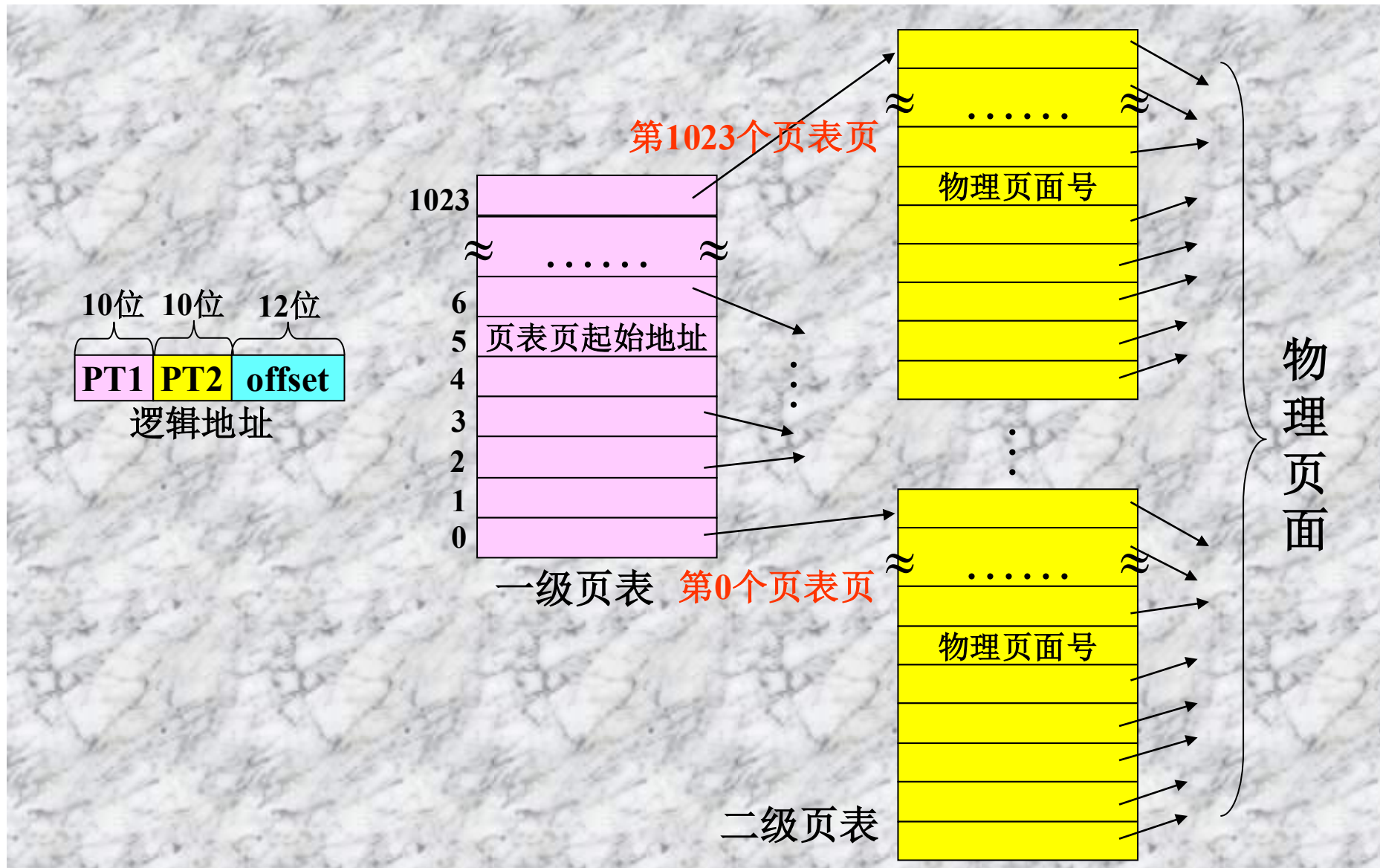
8.4.1 简单页式(simple paging)

(5) 多级页表

- 虚拟地址空间很大而每页比较小，则进程页表太长。
 - 例如，32位系统中，一个进程的虚拟地址最大4G，假设页长为4k，每个表项用4个字节描述，则页表需要多少空间？
 - 庞大的页表始终在主存，不完全在内存可以吗？
 - 需占用连续的内存空间，不连续可以吗？
- 解决方案
 - 采用离散分配方式解决难以找到一块连续大内存空间问题
 - 采用多级页表，只将当前需要部分调入内存，其余页表仍驻留磁盘，需要时调入



8.4 页式和段式存储管理





8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

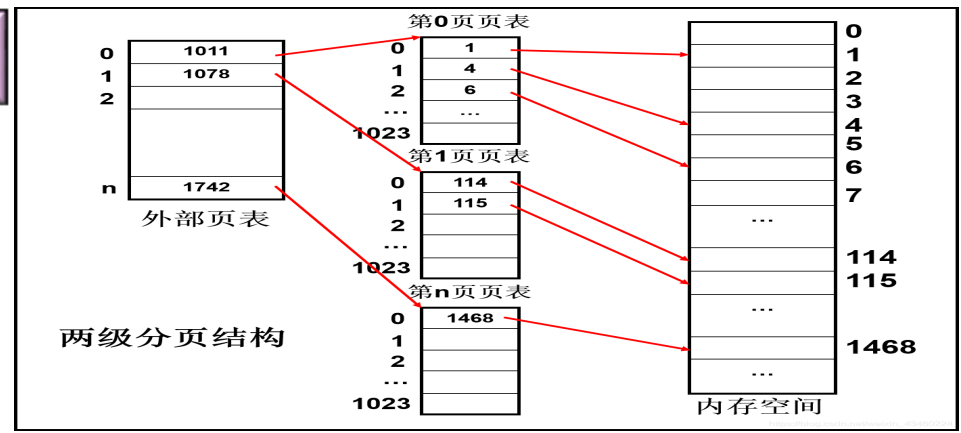
(5) 多级页表

➤ 将页表本身进行分页，形成多级页表

- **页表页**：将整个页表分割成许多小页表，称为**页表页**。每个页表页含有若干**页表表项**，给出逻辑页和物理页面的对应关系。页表页从0开始顺序编号，允许被分散存放在**不连续**页面中。
- **页目录表**：为了找到页表页，应建立地址索引，称为**页目录表**，其表项指出页表页的起始地址。每个进程有一个**页目录表**。
- **逻辑地址**：页目录位移(页表页编号)、页表页位移、页内位移。

页目录位移	页表页位移	页内位移
-------	-------	------

➤ 问题？页目录表地址放在哪里？



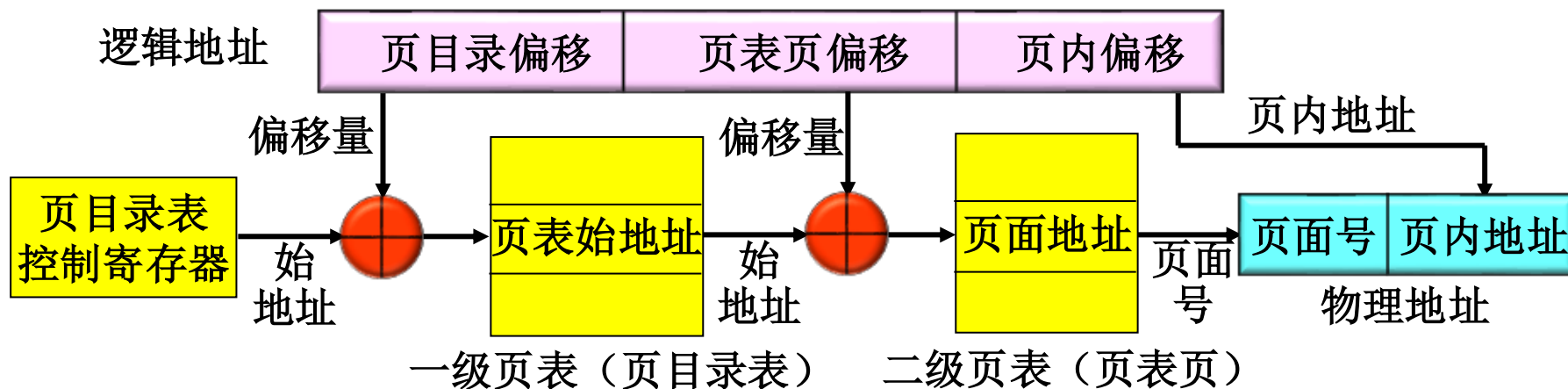
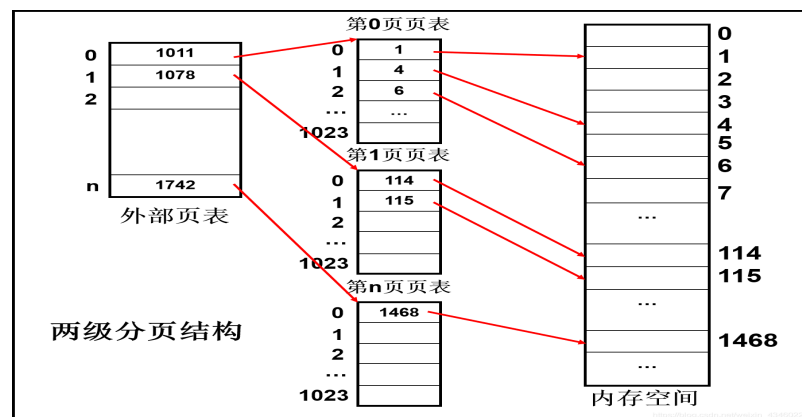


8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(5) 多级页表

● 二级页表的地址映射



- 基于二级页表需访问3次主存，一次访问页目录、一次访问页表页、一次访问指令或数据



8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(5) 多级页表

- 如二级页表仍不够用，可引入三级、四级页表，如SUN SPARC处理器支持三级页表，Motorola 68030支持四级页表。
- 多级页表不连续存放。
- 进程运行涉及页面的页表页应存放在主存，其他页表页使用时可动态调入。为此，需在页目录表中增加标志位，指示对应的页表页是否已调入内存，如未调入，产生“缺页表页”中断，请求调入。
- 为缩短查找时间，多级页表中的每级都可以装入到关联存储器（即页表的高速缓存）中，并按照cache的原理进行更新。

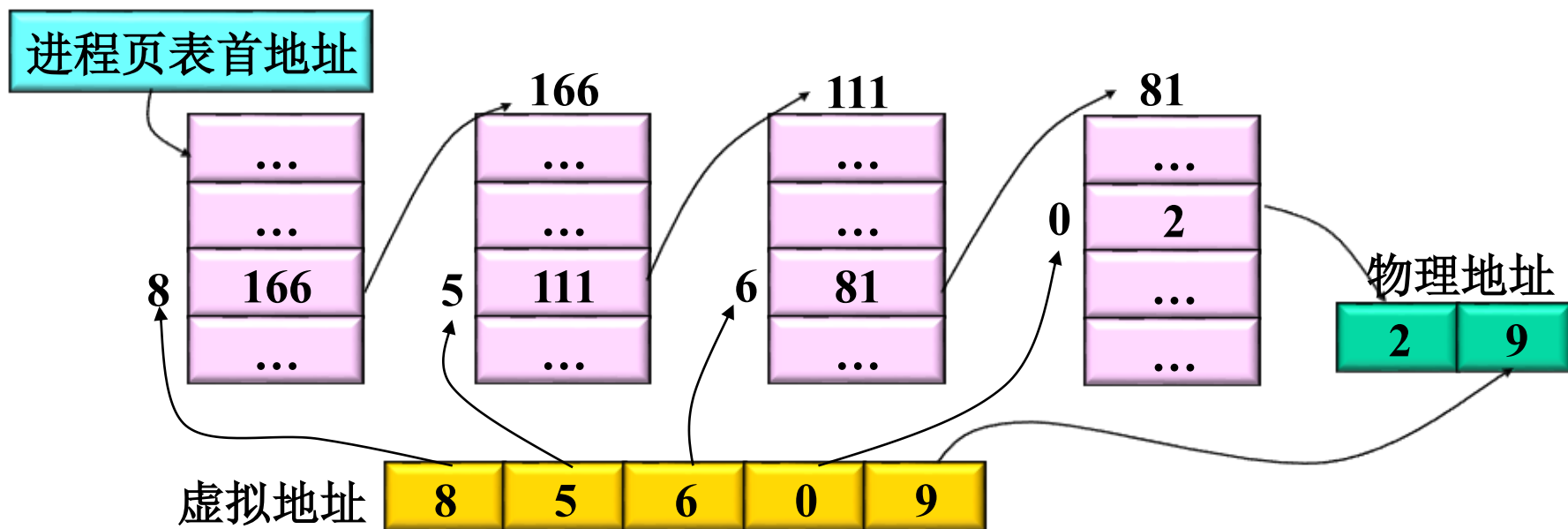


8.4 页式和段式存储管理

8.4.1 简单页式(simple paging)

(5) 多级页表

例：多级页表地址映射





8.4 页式和段式存储管理



8.4.1 简单页式(simple paging)

(6) 页式管理的优缺点

- 优点：解决了碎片问题
- 缺点：不易实现共享
不便于动态连接



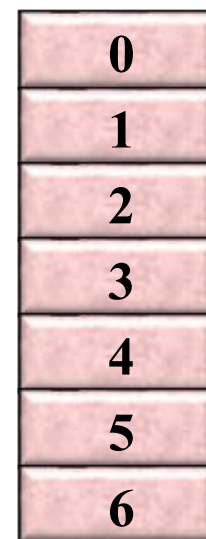
8.4 页式和段式存储管理

8.4.2 简单段式 (simple segmentation)

● 引入的目的（简单页式存在的问题）

✓ 为用户提供一个方便灵活的程序设计环境而提出

- 用户一般采用指定**程序名和数据块**的方法，如果系统不能将用户给定的程序名和数据块名与这些被共享程序和数据在某个**进程中的虚页**对应起来，就**不能实现共享**
- 页式管理中，地址空间是一维的，主程序，子程序都顺序排列，**共享公用子程序很困难**
- **一个功能块被切分在不同的页中**：页式管理中，一个页面中可能装有2个不同的子程序段的指令代码，不能通过页面共享来达到共享一个逻辑上完整的子程序或数据块



作业的
地址空间



8.4 页式和段式存储管理



8.4.2 简单段式 (simple segmentation)

页式管理是把进程的地址空间视为**一维**线性空间；而段式管理是把它视为**二维**空间

(1) 简单段式管理的基本原理

- 将程序的地址空间按**内容或函数关系**划分为若干个段(segment) (按段的逻辑关系进行划分)，段的大小不需要相等
- 程序加载时，**以段为单位分配空间**，这些段**不必连续**
- 当一个进程被调入时，它的所有段都被装入到存储器的可用区域中，并建立一个**段表**
- 物理内存的管理采用**动态分区**
- 地址变换需要**CPU的硬件支持**，然后通过地址映射机构将段式虚地址转换为实际的内存物理地址

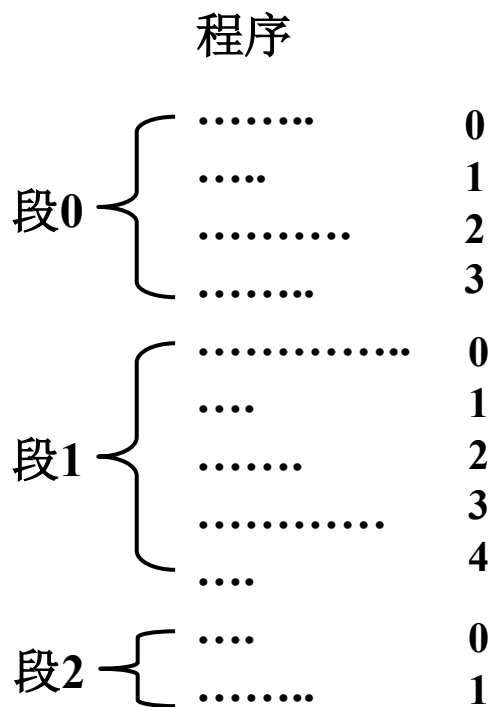


8.4 页式和段式存储管理



8.4.2 简单段式 (simple segmentation)

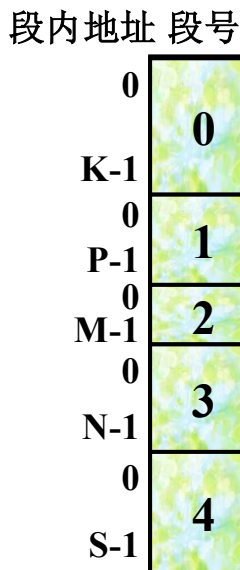
程序空间编址



逻辑地址格式

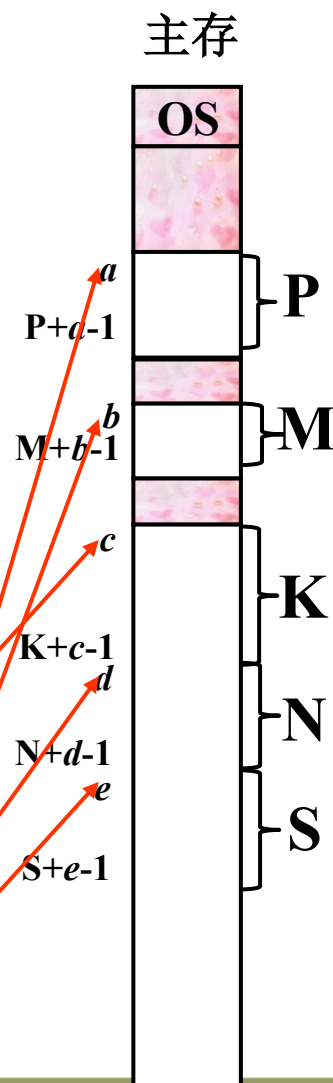


程序空间



进程段表

逻辑段号	段长	内存始址
0	K	c
1	P	a
2	M	b
3	N	d
4	S	e





8.4 页式和段式存储管理



8.4.2 简单段式 (simple segmentation)

(1) 简单段式管理的特点:

● 优点

- 可以按照进程内的逻辑关系划分成段，便于共享
- 便于改变进程占用空间的大小

● 缺点

- 进程全部装入内存

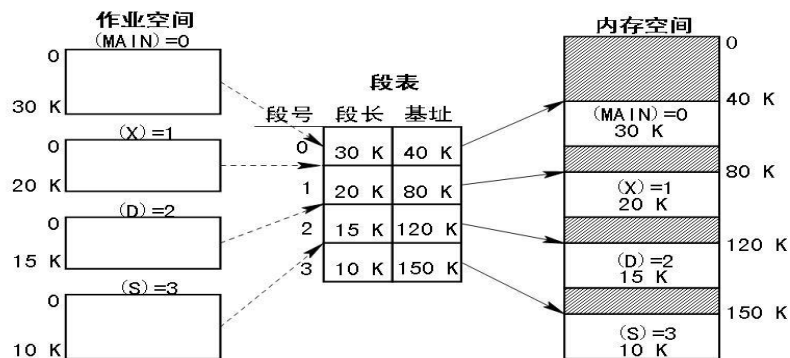


8.4 页式和段式存储管理

8.4.2 简单段式 (simple segmentation)

(1) 与简单页式的初步比较

- 段与段之间**无顺序**关系；而页式管理中，页号是按顺序递增的，即页是**有逻辑顺序**的，属一维空间
- 段的**长度不固定**，页的**大小是一定的**
- 每个段定义一组**逻辑上完整**的程序或数据，如一个进程可以被划分为主程序段，子程序段，数据段，堆栈段等，而页中的内容是按顺序装入的
- 每个段是一个首地址为0的，连续的一维线性空间，段长可以动态增长
- 段式的逻辑地址是：**段名和段内地址**；简单页式的逻辑地址式一维的





8.4 页式和段式存储管理



8.4.2 简单段式 (simple segmentation)

(2) 简单段式管理的数据结构

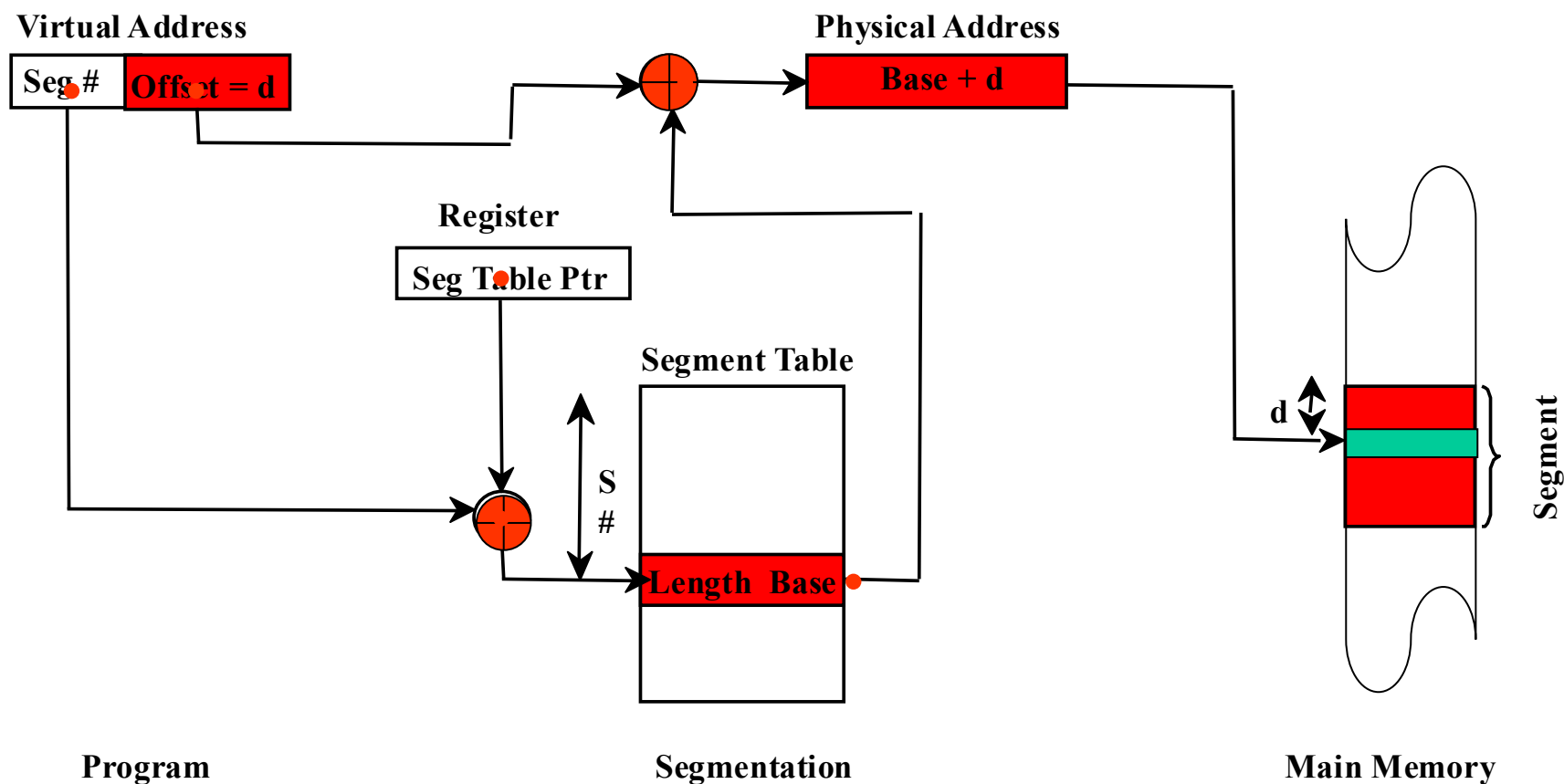
- ① 进程段表：每段有段基址(base address)和段长度
- ② 系统段表：系统内所有占用段
- ③ 空闲分区表：内存中所有空闲分区，记录所有的空闲分区



8.4 页式和段式存储管理

8.4.2 简单段式 (simple segmentation)

(3) 段式地址变换

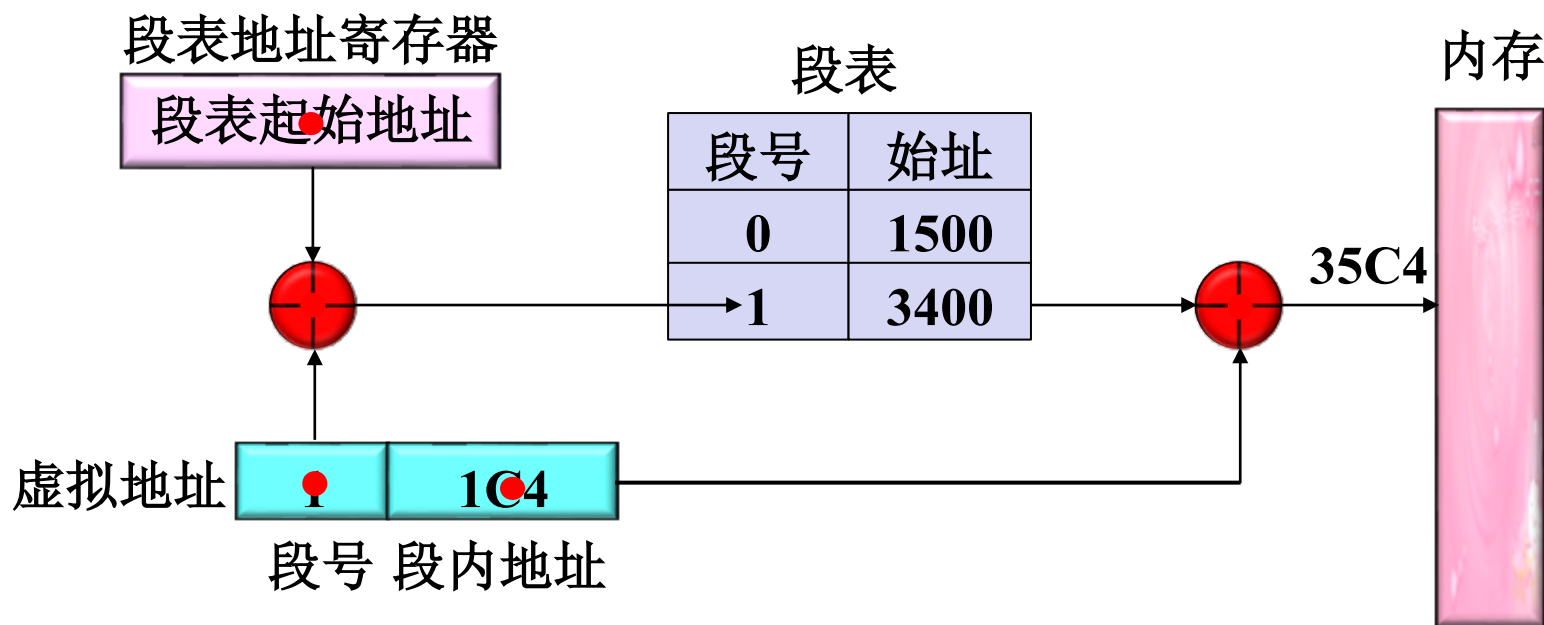




8.4 页式和段式存储管理

8.4.2 简单段式 (simple segmentation)

(3) 段式地址变换--举例





8.4 页式和段式存储管理

8.4.2 简单段式 (simple segmentation)

● 页式管理和段式管理的再比较

- 分页是出于系统管理的需要，分段是出于用户应用的需要
- 页大小是系统固定的，而段大小则通常不固定
- 逻辑地址表示
 - 分页是一维的，各个模块在链接时必须组织成同一个地址空间
 - 分段是二维的，各个模块在链接时可以每个段组织成一个地址空间
- 通常段比页大，因而段表比页表短，可以缩短查找时间，提高访问速度

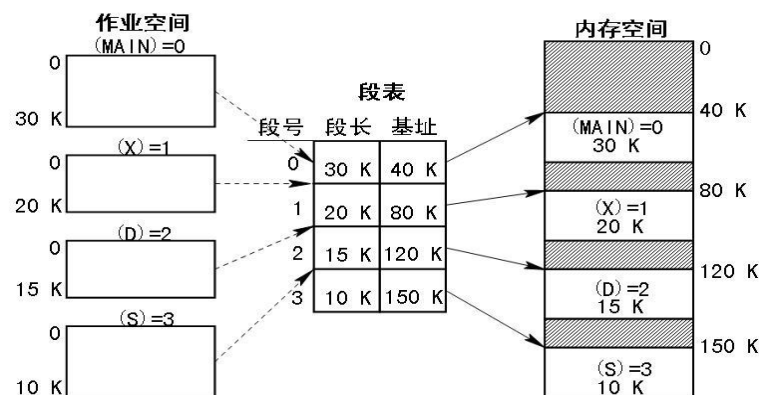


8.4 页式和段式存储管理

8.4.2 简单段式 (simple segmentation)

(4) 小节

- 简单段式管理的基本原理
 - 逻辑地址：段号+段内偏移
- 数据结构
 - 段表
 - 空闲分区表
 - 请求表
- 地址变换
- 特点
- 与简单页式管理的比较





8.5 虚拟存储器

8.5.1 虚拟存储器的原理

(1) 虚拟存储的基本原理

- **部分装入**：在程序装入时，不必将其全部读入到内存，而只需将当前需要执行的部分页或段读入到内存，就可让程序开始执行
- **按需调入**：在程序执行过程中，如果需执行的指令或访问的数据尚未在内存（称为**缺页或缺段**），则由处理器通知OS将相应的页或段**调入**到内存，然后继续执行程序
- **部分换出**：另一方面，OS将内存中**暂时不使用的页或段调出保存在外存**上，从而腾出空间存放将要装入的程序以及将要调入的页或段。只需程序所需的一部分在内存就可执行



8.5 虚拟存储器



8.5.1 虚拟存储器的原理

(2) 引入虚拟存储技术的好处

- **大程序**：可在较小的可用内存中执行较大的用户程序
- **大的用户空间**：提供给用户可用的虚拟内存空间通常大于物理内存(real memory)
- **并发**：可在内存中容纳更多程序并发执行
- **易于开发**：与覆盖技术比较，不必影响编程时的程序结构



8.5 虚拟存储器

8.5.1 虚拟存储器的原理

(3) 虚拟存储技术的特征

- **不连续性**: 物理内存分配的不连续
- **部分交换**: 与交换技术相比较, 虚拟存储的调入和调出是对部分虚拟地址空间进行的
- **大空间**: 通过物理内存和快速外存相结合, 提供大范围的虚拟地址空间
 - 总容量不超过物理内存和外存交换区容量之和



8.5 虚拟存储器

8.5.2 虚拟页式(virtual paging)

在简单页式存储管理的基础上，增加请求调页和页面置换功能

(1) 基本工作原理

- 在进程开始运行之前，不是装入全部页面，而是装入部分页面；
- 运行过程中，根据进程运行的需要，动态装入其它页面；
- 当内存空间已满，而又需要装入新的页面时，则根据某种算法淘汰某个页面，以便装入新的页面

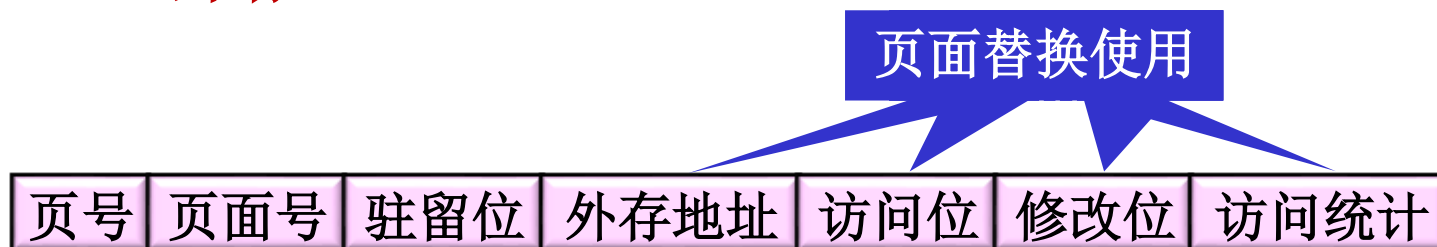


8.5 虚拟存储器

8.5.2 虚拟页式(virtual paging)

(2) 进程页表的修改

- 需要在进程页表中添加若干项
 - **标志位**：存在位（present bit，或驻留位，内存页和外存页），修改位(modified bit)
 - **访问统计**：在近期内被访问的次数，或最近一次访问到现在的时间间隔
 - **外存地址**



又称“存在位”，表示该页是否在主存



8.5 虚拟存储器



8.5.2 虚拟页式(virtual paging)

(3) 缺页中断

- 在地址映射过程中，在页表中发现所要访问的页不在内存，则产生缺页中断
- OS接到此中断信号后，就调用缺页中断处理程序，根据页表中给出的外存地址，将该页调入内存，使作业继续运行下去
- 如果内存中有空闲块，则分配一页，将新调入页装入内存，并修改页表中相应页表项目的驻留位及相应的内存块号
- 若此时内存中没有空闲块，则要淘汰某页，若该页在内存期间被修改过，则要将其写回外存



8.5 虚拟存储器

8.5.2 虚拟页式(virtual paging)

(4) 请求页式管理和预调入管理

- **请求页式：**当需要执行某条指令而又发现它不在内存时，或当执行某条指令需要访问其他的数据或指令时，这些指令和数据不在内存中，从而发生缺页中断（主要讨论）将需要的页装入内存。
- **预调入：**系统对那些在外存中的页进行调入顺序计算，估计出这些页中指令和数据的执行和被访问的顺序，并按此顺序将它们顺次调入和调出内存



8.5 虚拟存储器



8.5.2 虚拟页式(virtual paging)

● 虚拟页式管理的优缺点

● 优点

- 由于它不要求作业或进程的程序段和数据在内存中连续存放，消除了碎片问题
- 实现了虚存

● 缺点

- 要求有硬件支持
- 增加了系统开销，如缺页中断处理
- 请求调页的算法如选择不当，由可能产生抖动现象
- 每个作业和进程的最后一个页内总有一部分空间得不到利用，如果页面较大，则这一部分的损失仍然较大

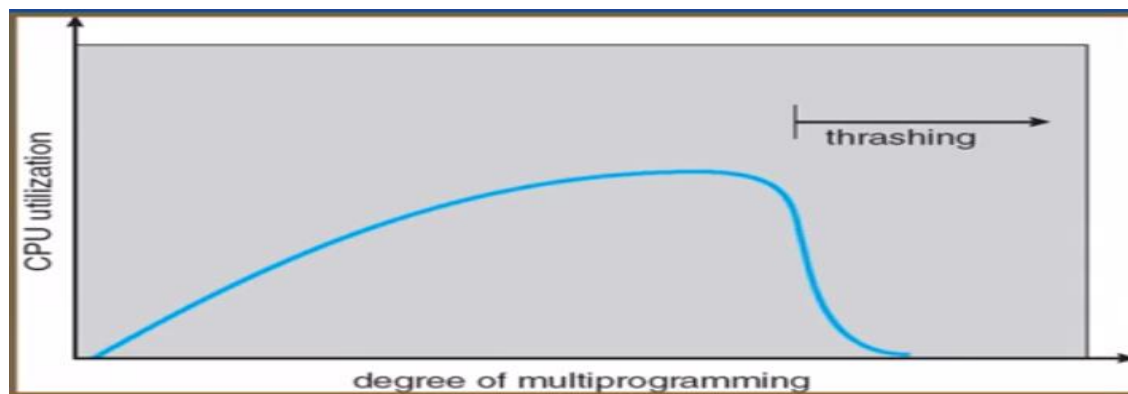


8.5 虚拟存储器

8.5.2 虚拟页式(virtual paging)

● 抖动 (Thrashing)

- ✓ **定义：**在请求页式系统中，从内存中刚刚移出某个页面后，根据请求马上又调入该页，这种反复的页调入调出现象称为“**抖动**”。
- ✓ **害处：**大部分机器时间都用于页面调度，只有小部分时间用于进程的实际运行。





8.5 虚拟存储器

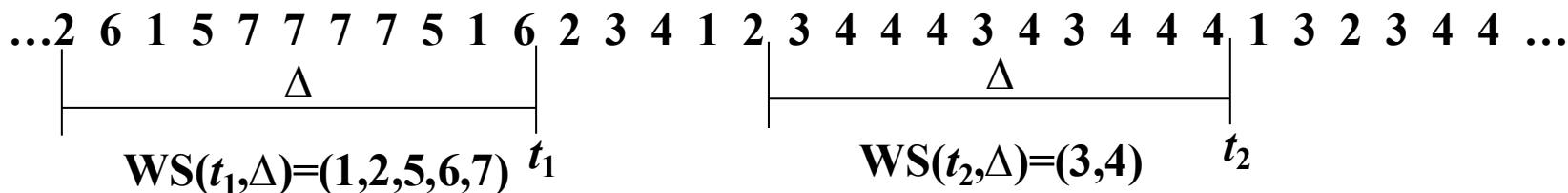
8.5.2 虚拟页式(virtual paging)

工作集

- **工作集 (Work Set)** : 一个进程在某一小段时间 Δ 内访问的页面集合。用 $WS(t_i, \Delta)$ 表示 $t_i - \Delta$ 到 t_i 之间所访问的不同页面, 则 $WS(t_i, \Delta)$ 即为进程在时间 t_i 的工作集
- 随 Δ 单调递增: $WS(t_i, \Delta) \subseteq WS(t_i, \Delta + a)$, 其中 $a > 0$
- 设: 每个进程的工作集大小为 WSS_i , 令:

$$D = \sum WSS_i$$

如果 D 大于可用内存块的总量 m ($D > m$), 将出现抖动, 将需要换入、换出





8.5 虚拟存储器



8.5.2 虚拟页式(virtual paging)

● 产生抖动的原因:

- 进程的物理页面过少，不能包含工作集，造成大量缺页，频繁置换

● 利用工作集模型防止抖动

- OS监督每个进程的工作集，并给它分配工作集所需的内存块。
- 若有足够多的额外块，即可装入另外的进程；
- 若工作集增大，超出可用块的总数，OS要选择一个进程挂起，将其页面写出，将其占用的内存分配给其它进程。被挂起的进程在以后适当时机被激活



Review



- 简单页式变换:

- 进程请求表、页表始址、虚拟地址R、物理地址R
- 快表TLB: 解决地址变换慢 (2次访存)
- 多级页表: 解决页表过大

- 简单段式管理

- 目的: 解决数据共享的问题、也符合编程的习惯
- 虚拟地址空间是2维的: 短号+段内偏移
- 原理: 一个段一个分区、完全不连续装入、动态分区管理内存
- 数据结构: 段表、系统段表、空闲分区表
- 地址变换
- 与简单页式的比较: 虚地址 (1d vs 2d)、页定长 vs 段不定长

- 虚拟存储器

- 原理: 程序部分装入、需要是调入 (预调入、请求调入)、可换出
- 益处: 运行大程序、大用户空间、并发度增大、利于编程
- 虚拟页式: 修改页表项 (存在位、访问位、修改位等)、缺页中断
- 问题: 抖动、工作集



8.5 虚拟存储器



8.5.3 虚拟段式 (virtual segmentation)

(1) 方法

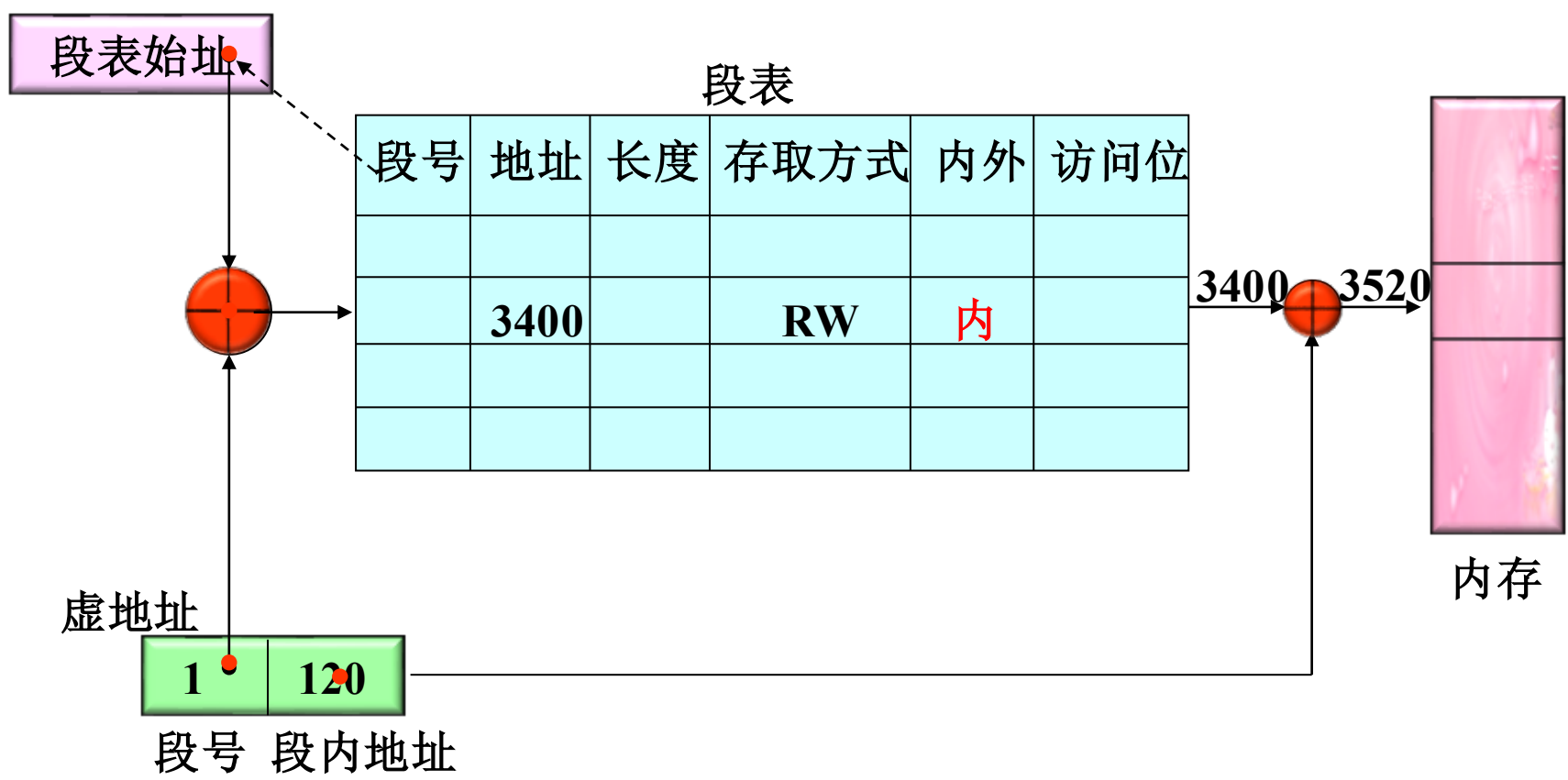
- 在简单段式存储管理的基础上，改为部分装入，增加请求调段和段置换功能。
- 段表：需要在进程段表中添加若干项
 - 标志位：存在位(present bit)，修改位(modified bit/dirty bit)，增长位（该段是否增长过，在虚拟页式中没有该位）
 - 访问统计：如使用位(use bit)
 - 存取权限：如读R，写W，执行X
 - 外存地址
- 动态地址变换和缺段中断



8.5 虚拟存储器

8.5.3 虚拟段式 (virtual segmentation)

(2) 虚拟段式地址变换





8.5 虚拟存储器



8.5.3 虚拟段式 (virtual segmentation)

(3) 段式管理的优缺点

➤ 优点

- 可以实现**虚存**
- 段长可以**动态增加**（可以吸收新数据）；
- 便于对具有完整逻辑功能的信息段进行**共享**
- 便于实现**动态链接**

➤ 缺点

- 每个段的长度受可用内存区大小的**限制**
- 需要硬件支持

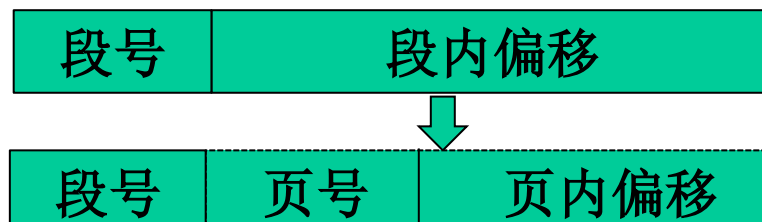


8.5 虚拟存储器

8.5.4 段页式(combined paging and segmentation)

(1) 段页式基本原理

- 是**虚拟页式**和**虚拟段式**存储管理的结合，由于开销比较大，一般只用于大型机系统中。如，Unix System V、Linux内核在x86架构下；
- **每个段再分页**，利用页表记录一个段的各个页的内存占用情况，**按页加载进内存**；
- 内存分块，形成页面集合。内存**按页管理与分配**。
- **逻辑地址**：**段号，段内偏移**（页号，页内偏移地址），程序员可见的仍是段号和段内相对地址
- **地址变换**：先查段表，再查该段的页表

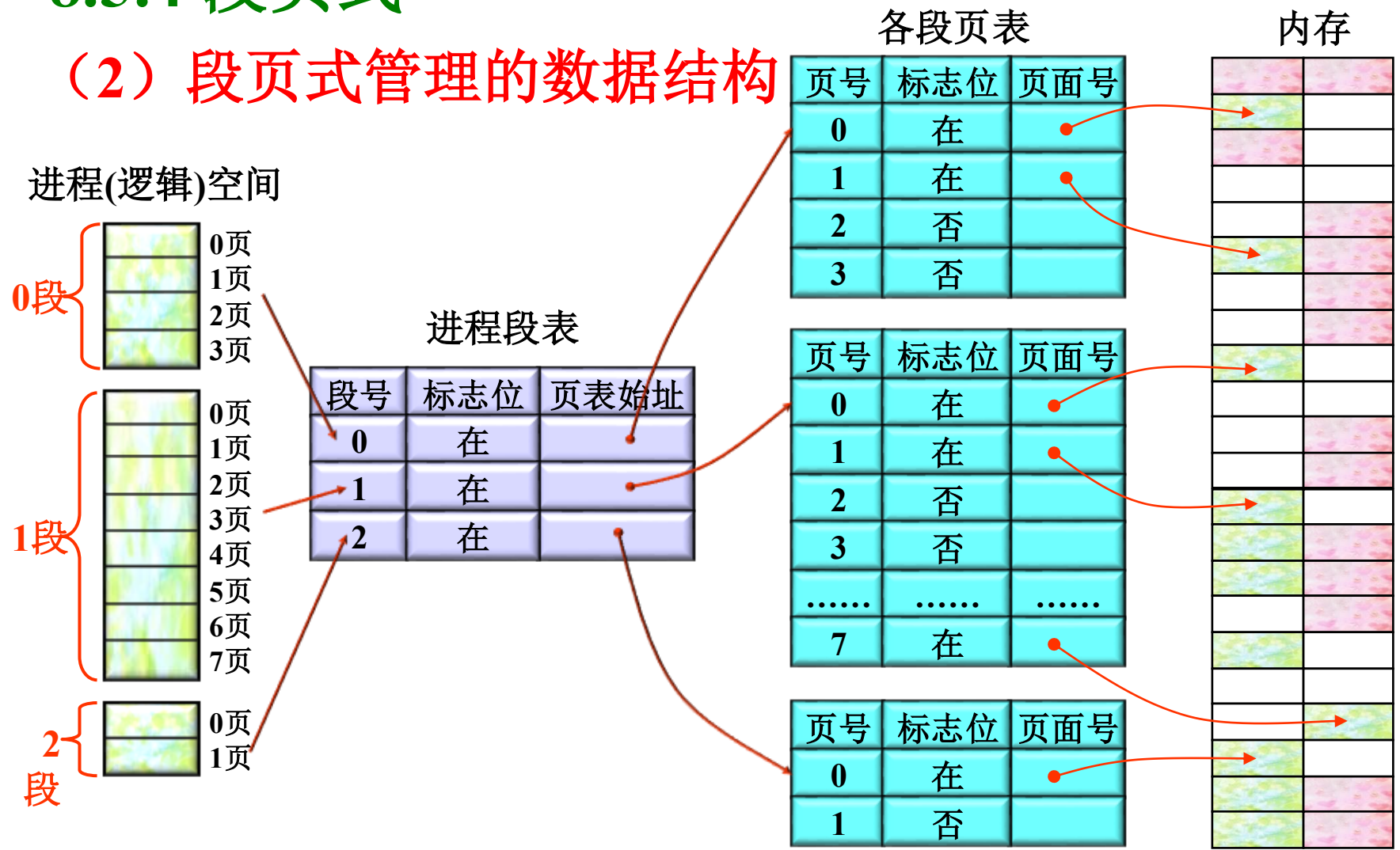




8.5 虚拟存储器

8.5.4 段页式

(2) 段页式管理的数据结构



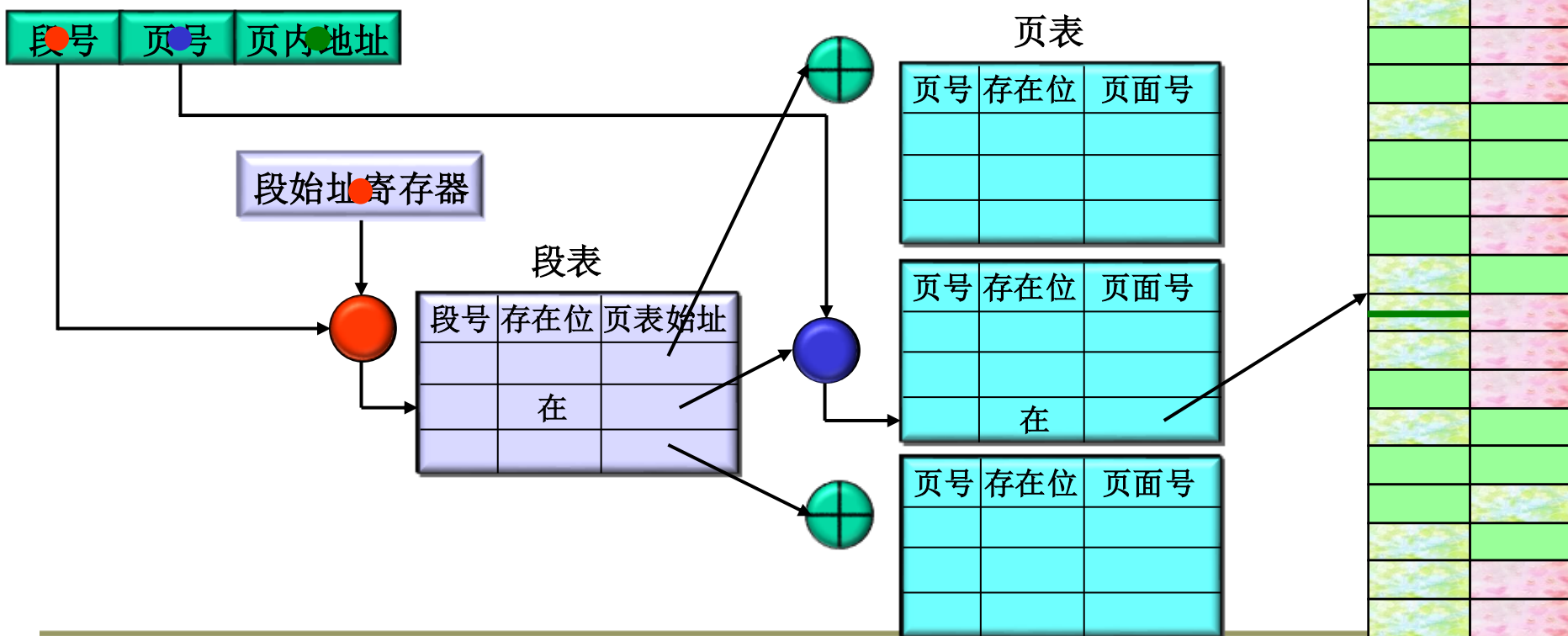


8.5 虚拟存储器

8.5.4 段页式(combined paging and segmentation)

(3) 段页式地址变换

内存





8.5 虚拟存储器



8.5.4 段页式(combined paging and segmentation)

(4) 段页式管理中的TLB

- 段页式管理中，对内存中的指令和数据进行一次存取的话，至少需要访问**三次**以上的内存：**段表、页表、物理单元**。因此，使CPU的执行指令速度大大降低
- 为提高地址转换速度，设置**高速联想寄存器**：存放最常用的段号s、页号p和对应的内存页面与其他控制项
- 当要访问内存空间某一单元时，可在通过**段表、页表**进行内存地址查找的同时，在**高速联想寄存器**中查找其段号和页号



8.5 虚拟存储器

8.5.4 段页式(combined paging and segmentation)

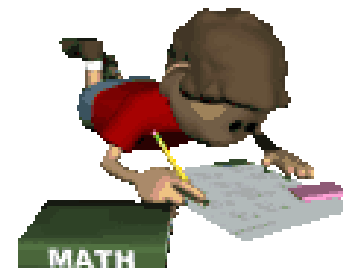
在一个段式存储管理系统中，其段表为：

段号	内存起始地址	段长
0	210	500
1	2350	20
2	100	90
3	1350	590
4	1938	95

例题

求下述逻辑地址对应的物理地址是什么？

段号	段内位移
0	430
1	10
2	500
3	400
4	112





8.5 虚拟存储器

8.5.4 段页式(combined paging and segmentation)

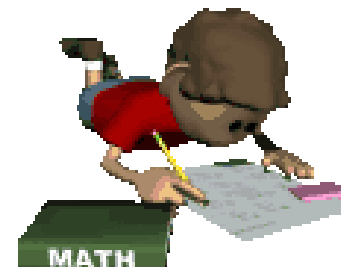
在一个段式存储管理系统中，其段表为：

段号	内存起始地址	段长
0	210	500
1	2350	20
2	100	90
3	1350	590
4	1938	95

例题

求下述逻辑地址对应的物理地址是什么？

段号	段内位移	
0	430	$(210+430=640)$
1	10	$(2350+10=2360)$
2	500	非法
3	400	$1350+400=1750$
4	112	非法

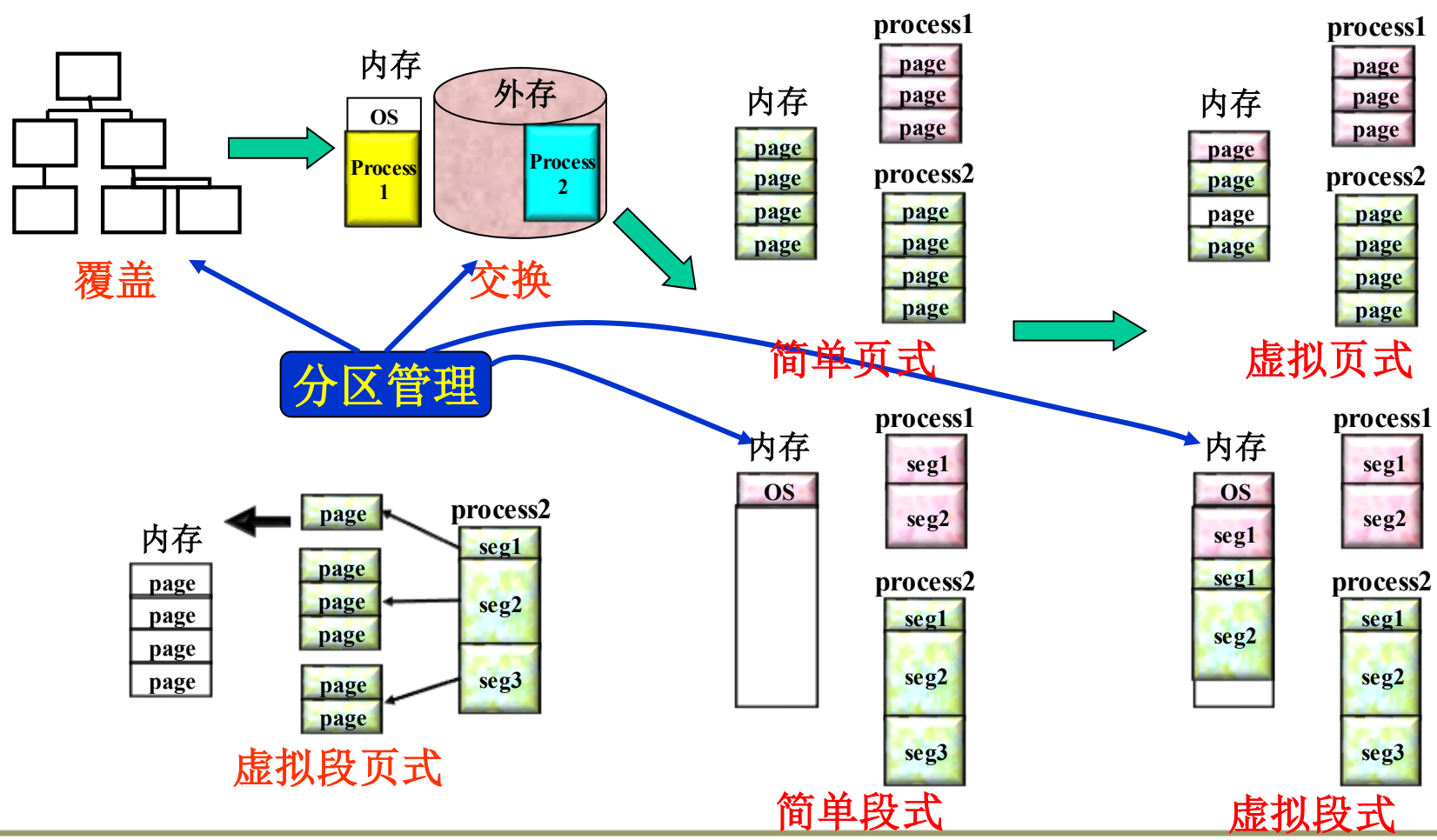




8.5 虚拟存储器

● 总结：存储管理技术的发展

程序处理⇒系统处理 连续存放⇒不连续存放 整体调入⇒部分调入





8.5 虚拟存储器

8.5.5 虚拟存储的调入、分配策略

- 虚拟存储为什么可以工作？
- 程序访问的**局部性原理**：在一定时间内，进程集中在一组子程序或循环中执行，导致所有的存储器访问局限于进程地址空间的一个固定子集
- **时间局部性**：一条指令的一次执行和下次执行以及一个数据的一次访问和下次访问都集中在一个较短的时间内
- **空间局部性**：在一段时间内，程序和数据的访问都集中在一个较小区域内

```
int i= 0;  
int a[100];  
while ( i<100) {  
    a[i] = i;  
    i++;}  
....
```



8.5 虚拟存储器



8.5.5 虚拟存储的调入、分配策略

调入策略 (fetch policy)

- 定义：调入策略确定在外存中的页或段调入时机。
- 在虚拟页式管理中有两种常用策略：
 - 请求调页(demand paging)：只调入发生缺页时所需的页面
 - 预调页(prepaging)：提前调入。在发生缺页需要调入某页时，一次调入该页以及相邻的几个页
- 上述调入策略同样适用于虚拟段式。



8.5 虚拟存储器



8.5.5 虚拟存储的调入、分配策略

● 分配策略 (assignment policy)

- 在**虚拟段式**管理中，采用**动态分区**对物理内存进行分配，可采用最佳适应、最先适应等
- 在**虚拟页式和段页式**管理中，内存**按页**管理，地址变换最后通过页表进行，因此不必考虑分配策略



8.5 虚拟存储器



8.5.5 虚拟存储的调入、分配策略

● **分配策略**: 固定 vs 可变

● **置换策略**: 局部 vs 全局

● **固定分配局部置换**

- 为每个进程分配**一定数目**的内存块，在整个运行期间不再改变
- 若进程在运行中缺页，只能从**该进程**在内存的n个页面中选出一页置换
- 为每个进程分配的内存块数难以确定



8.5 虚拟存储器



8.5.5 虚拟存储的调入、分配策略

●可变分配全局置换

- 先为每个进程分配一定数量的物理块，系统自身保持一个空闲物理块队列
- 某进程缺页时，系统从空闲物理块队列中取出一块分配给该进程
- 仅当空闲物理块队列空时，OS从内存中选择一页调出



8.5 虚拟存储器

8.5.5 虚拟存储的调入、分配策略

● 可变分配局部置换

- 先为每个进程分配一定数量的物理块
- 某进程缺页时，只能从该进程在内存的页面中选出一页置换
- 若某进程运行时频繁发生缺页中断，系统为该进程分配若干附加物理块，直至该进程缺页减少
- 若某进程运行时缺页率极低，适当减少分配给该进程的物理块，但不应引起其缺页率明显增加--Smart

● 审时度势



8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

- **功能：**需要调入页面时，选择内存中哪个物理页面被置换，称为replacement policy
- **目标：**把未来不再使用的或短期内较少使用的页面调出
- **常用置换策略**
 - ① 最佳算法 (OPT, optimal)
 - ② 最近最久未使用算法 (LRU, Least Recently Used)
 - ③ 先进先出算法 (FIFO)
 - ④ 时钟置换算法 (clock)
 - ⑤ 最不常用算法 (LFU, Least Frequently Used)
 - ⑥



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 选择“未来不再使用的”或“在离当前最远位置上出现的”页面被置换
- 这是一种理想情况，是实际执行中无法预知的，因而不能实现
- 可用作性能评价的依据
- 例（见下页）：

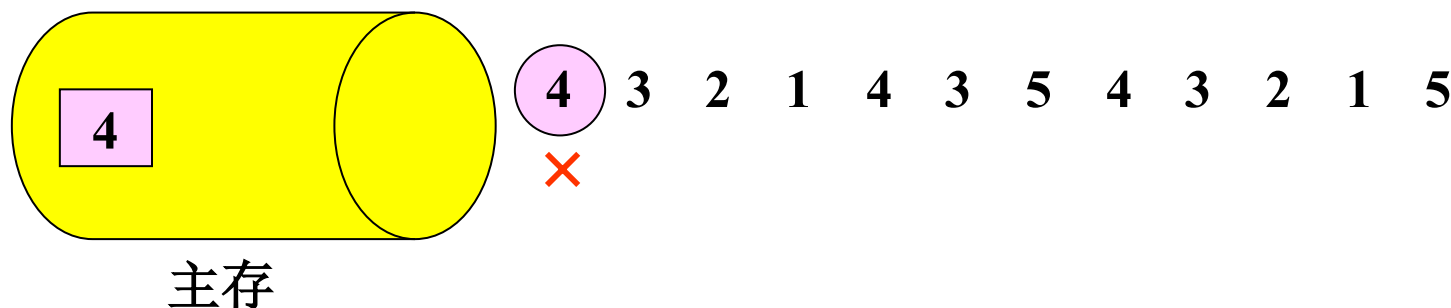


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

一个进程的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该进程的物理块数为3，计算采用最佳算法时的缺页率



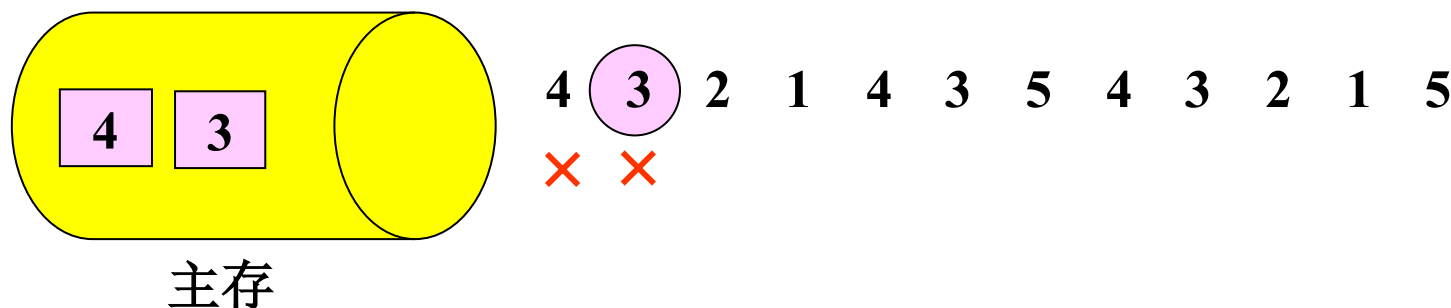


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

一个进程的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该进程的物理块数为3，计算采用最佳算法时的缺页率



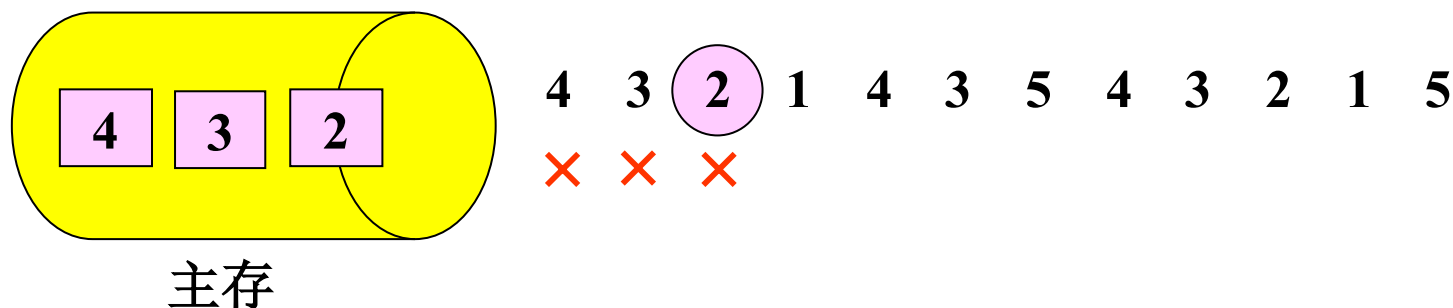


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



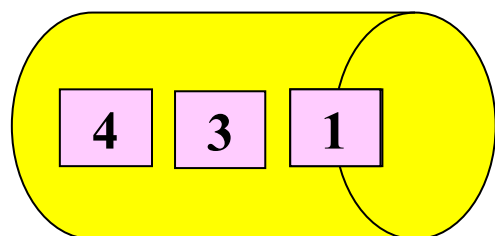


8.5 虚拟存储器

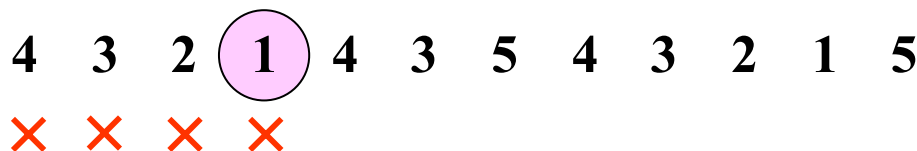
8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



主存



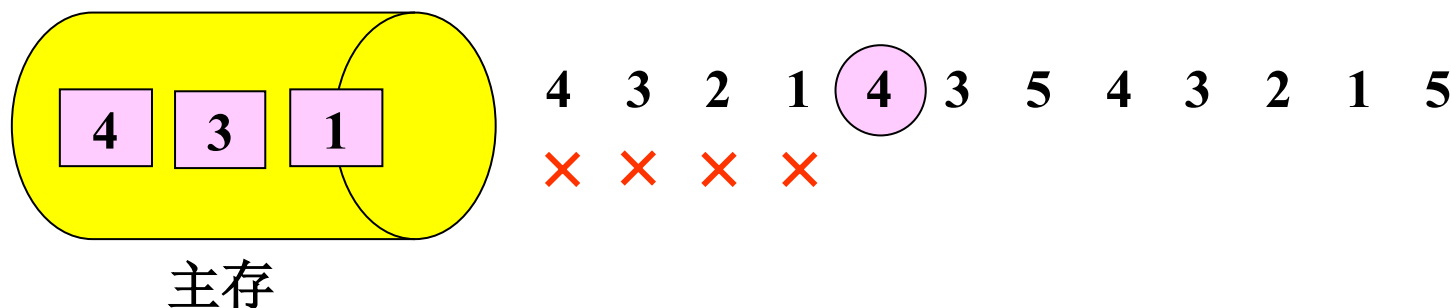


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



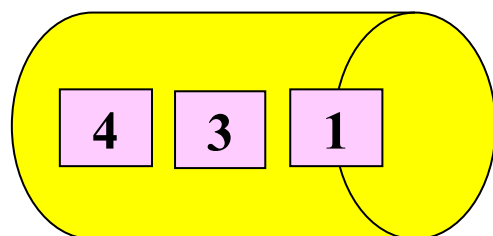


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



主存

4 3 2 1 4 3 5 4 3 2 1 5
× × × ×

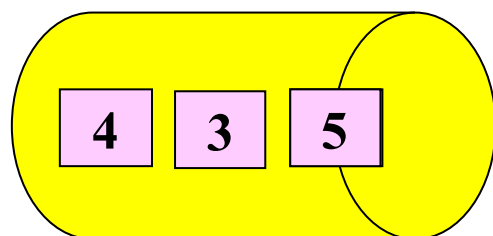


8.5 虚拟存储器

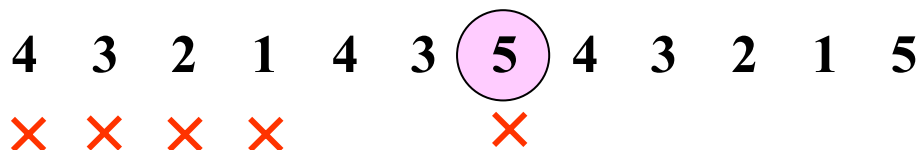
8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



主存



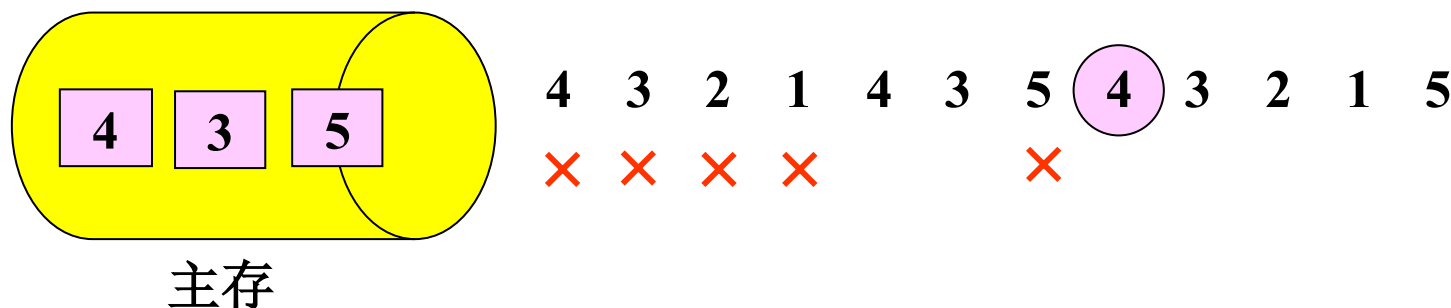


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



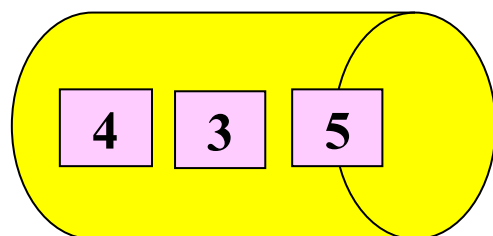


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



主存

4 3 2 1 4 3 5 4 3 2 1 5
× × × × ×

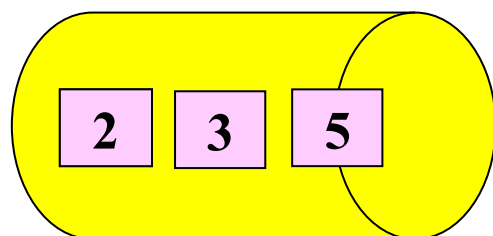


8.5 虚拟存储器

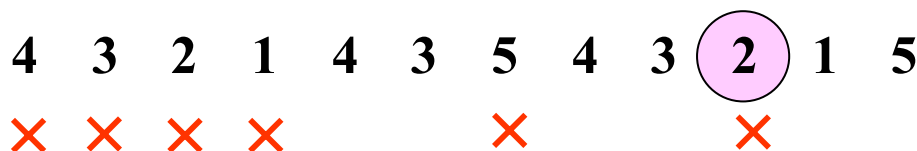
8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



主存



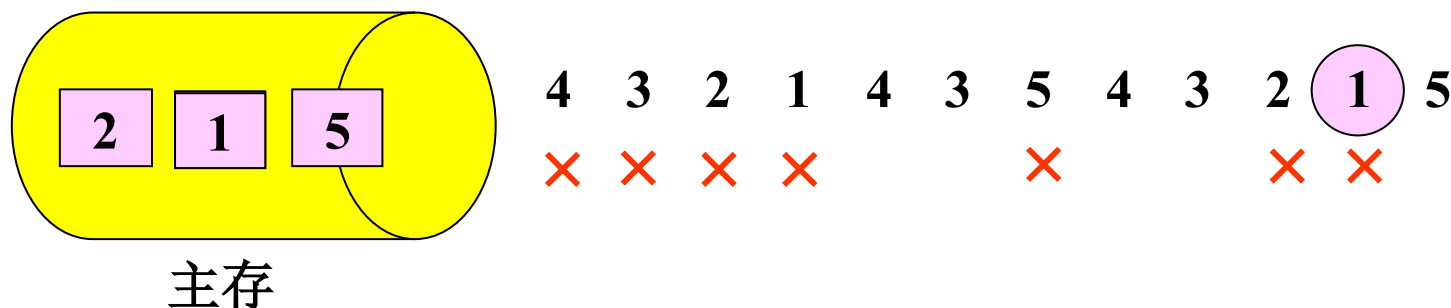


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



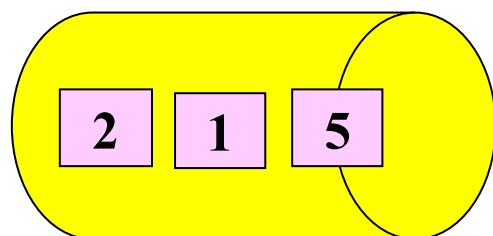


8.5 虚拟存储器

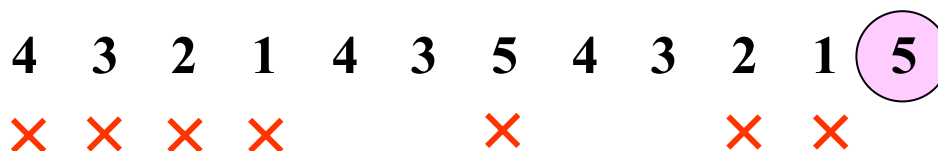
8.5.6 置换算法 (replacement policy)

(1) 最佳算法 (OPT, optimal)

- 一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算采用最佳算法时的缺页率



主存



缺页率=7/12



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法(LRU, Least Recently Used)

- **策略：** 选择内存中最久未使用的页面被置换
- **特点：** 但由于需要记录页面使用时间的先后关系，硬件开销太大
 - ✓ 问题：有什么不需要硬件计时器的替换策略？
- **硬件或结构支持：** 寄存器或栈

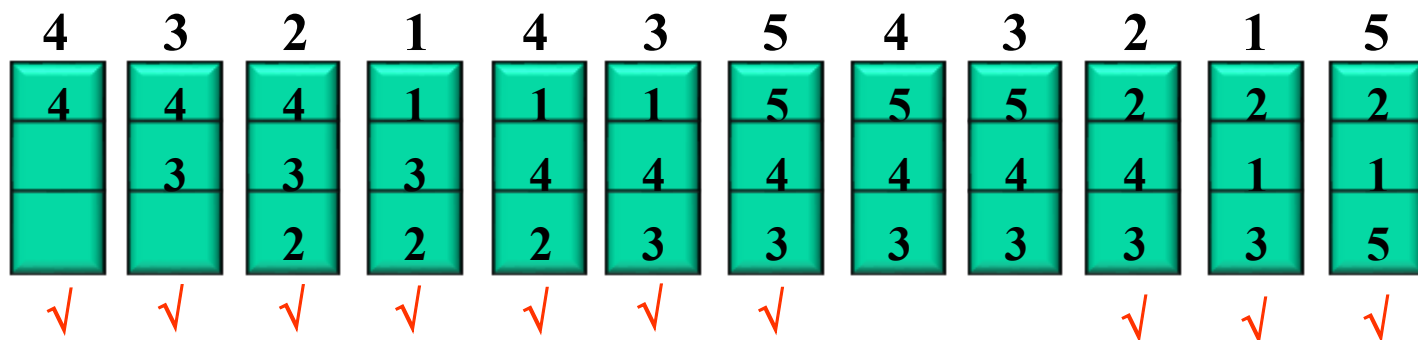


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU-Least Recently Used)

●例：一个作业的页面走向为4、3、2、1、4、3、5、4、3、2、1、5，分配给该作业的物理块数为3，计算LRU算法时的缺页率



缺页10次



8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法LRU

● 硬件支持：寄存器

- 为每个在内存中的页面配置 $R = R_{n-1}R_{n-2} \dots R_1R_0$
- 进程访问某页时，将该页的 R_{n-1} 置1
- 定时信号每隔一定时间右移1位
- 寄存器 R 被视为一个整数，置换时，具有最小数值的寄存器对应的页面被淘汰

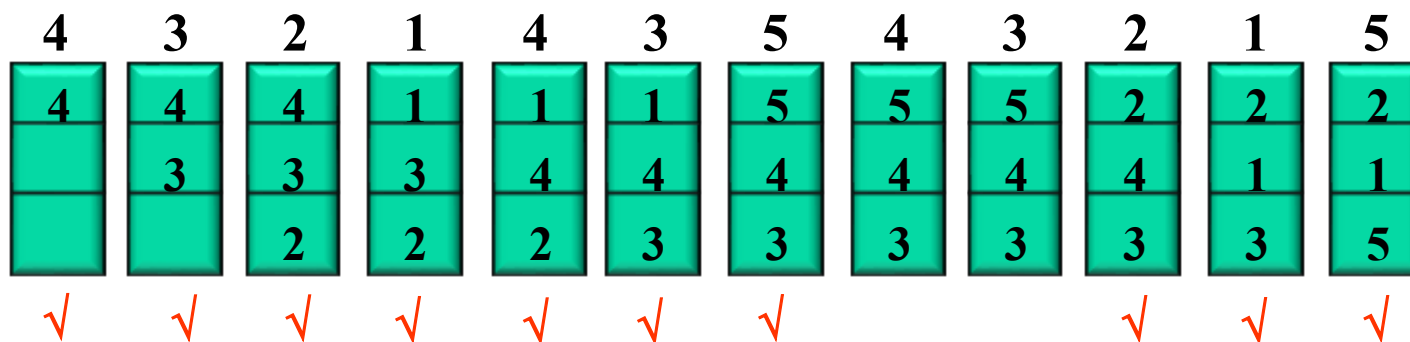


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)

基于寄存器的LRU实现:



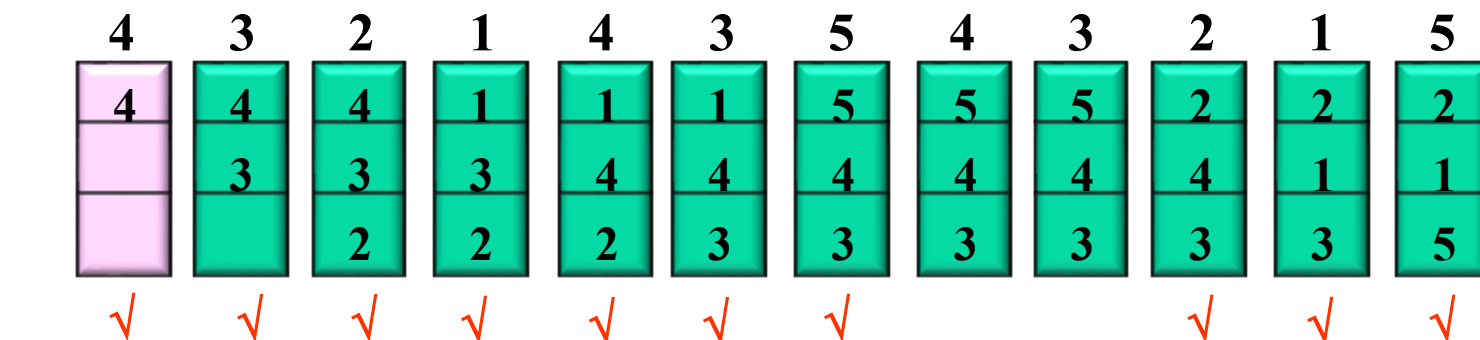


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)

基于寄存器的LRU实现：



R_2 R_1 R_0

1
2
3
4 1
5

移位寄存器状态

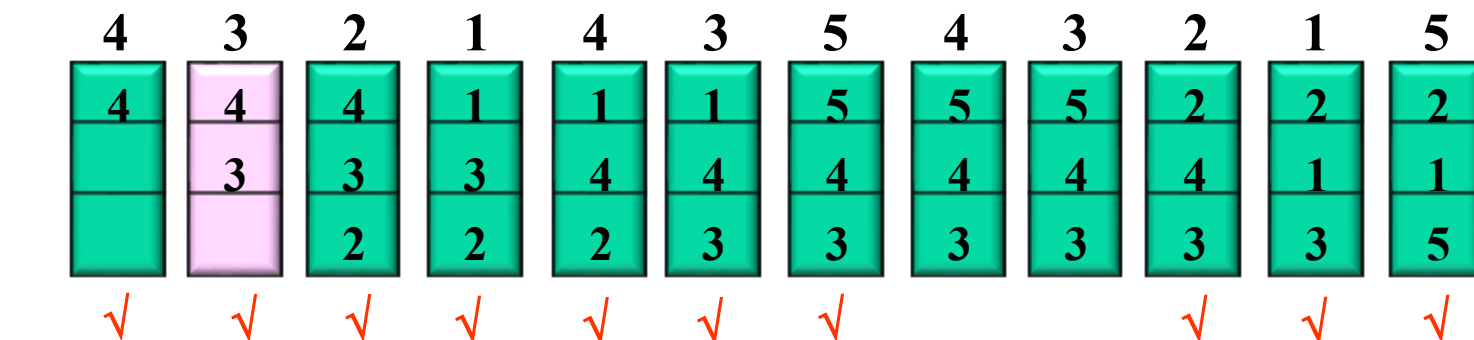


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)

基于寄存器的LRU实现:



R_2 R_1 R_0

1
2
3
4
5

1

1

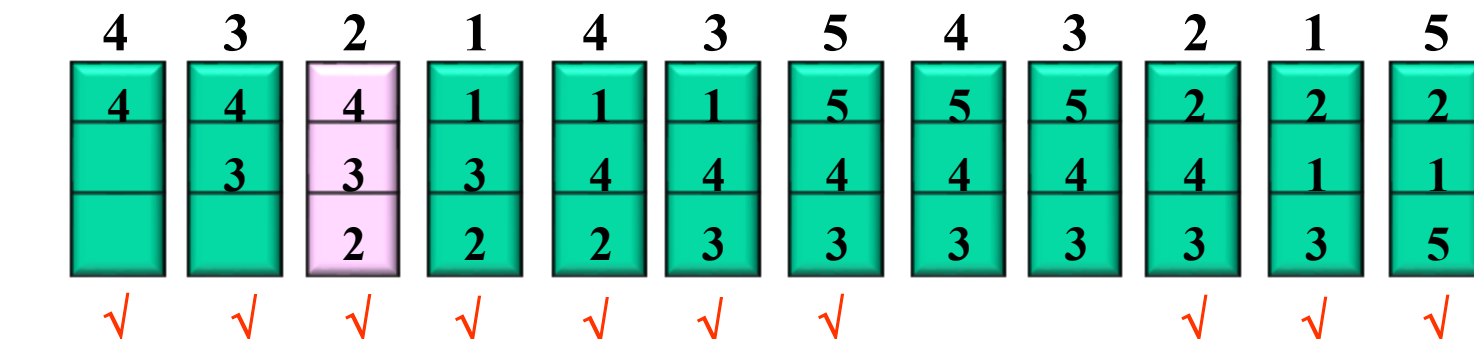
移位寄存器状态



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1			
2	1		
3		1	
4			1
5			

移位寄存器状态

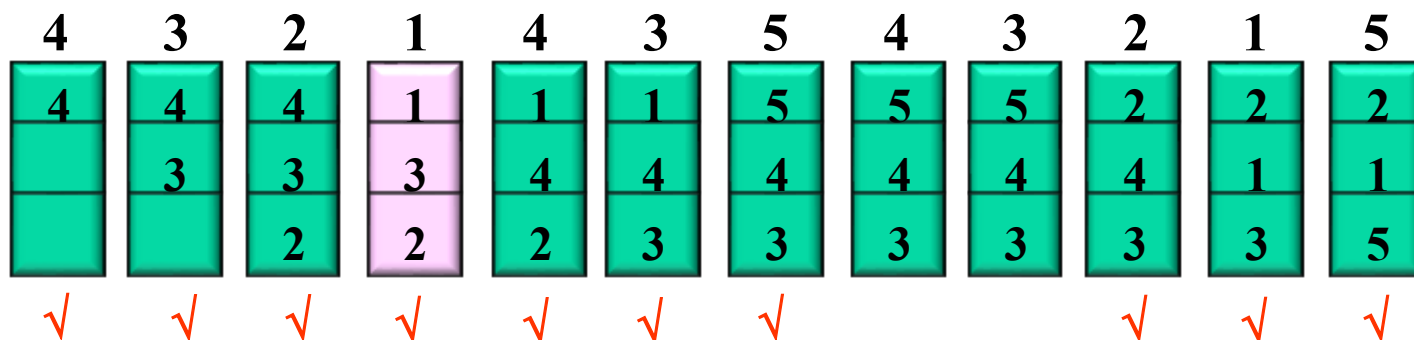


8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

● 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1
2
3
4
5

1

1

1

移位寄存器状态

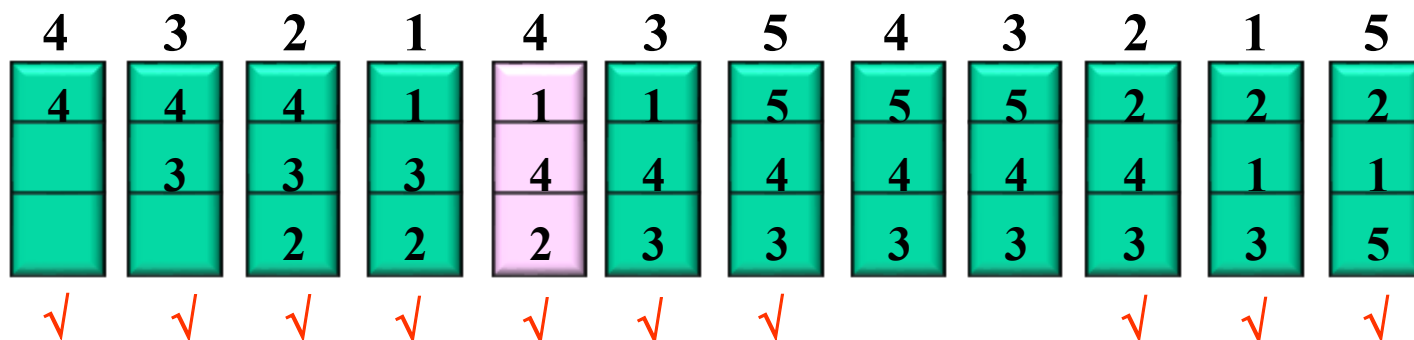


8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1
2
3
4 1
5

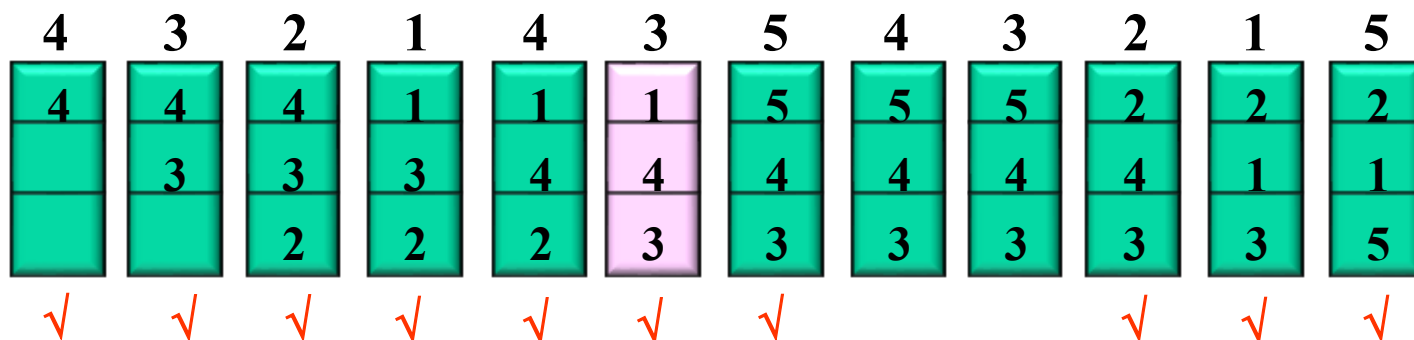
移位寄存器状态



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1
2
3 1
4 1
5

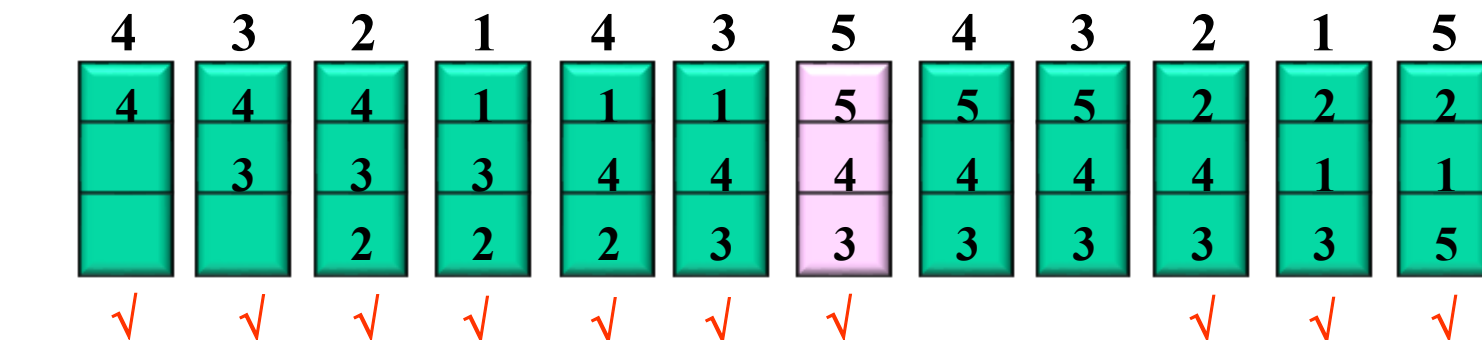
移位寄存器状态



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1			
2			
3	1		
4		1	
5	1		

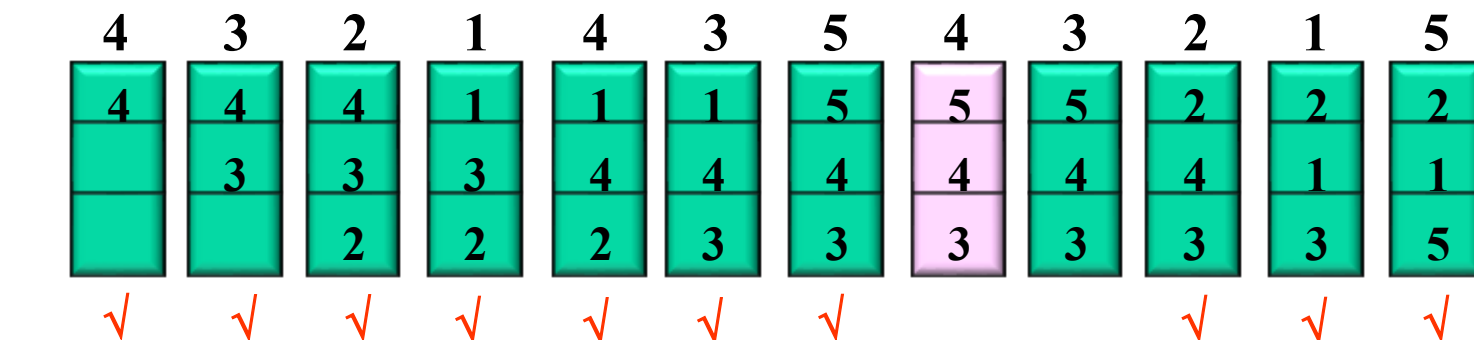
移位寄存器状态



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1			
2			
3		1	
4	1		
5		1	

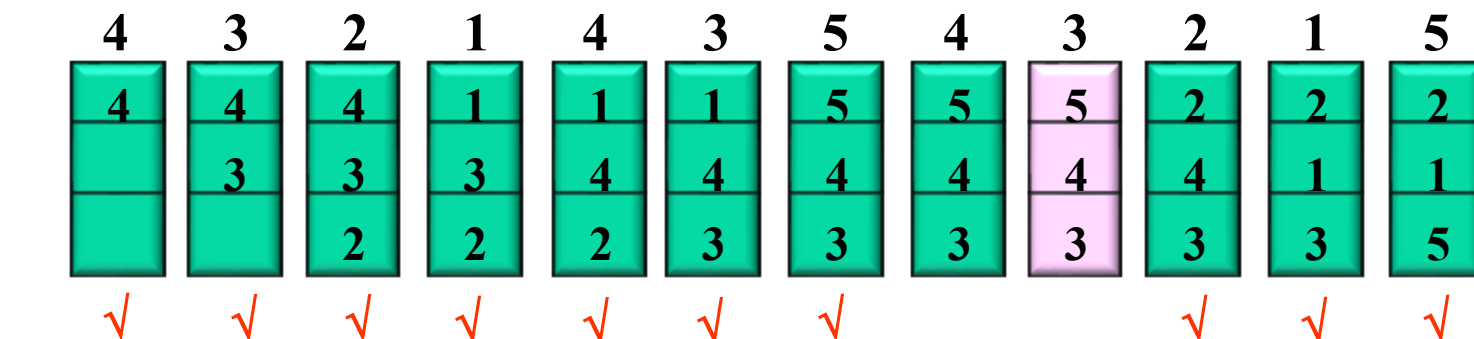
移位寄存器状态



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1			
2			
3	1		
4		1	
5			1

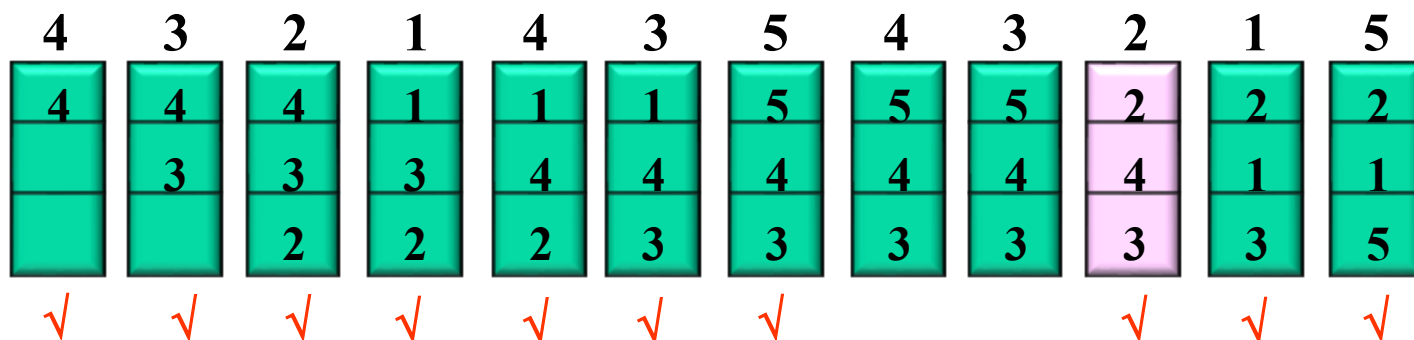
移位寄存器状态



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1			
2	1		
3		1	
4			1
5			

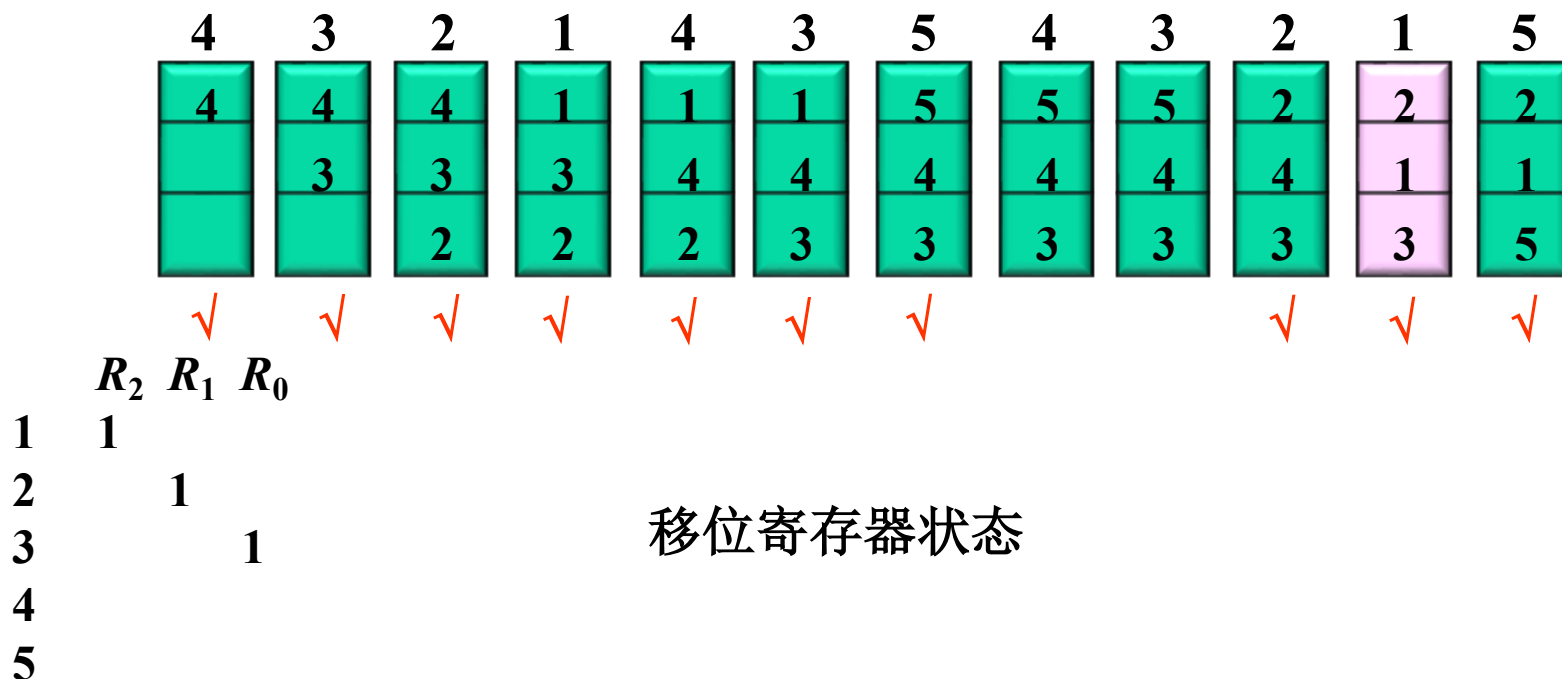
移位寄存器状态



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)

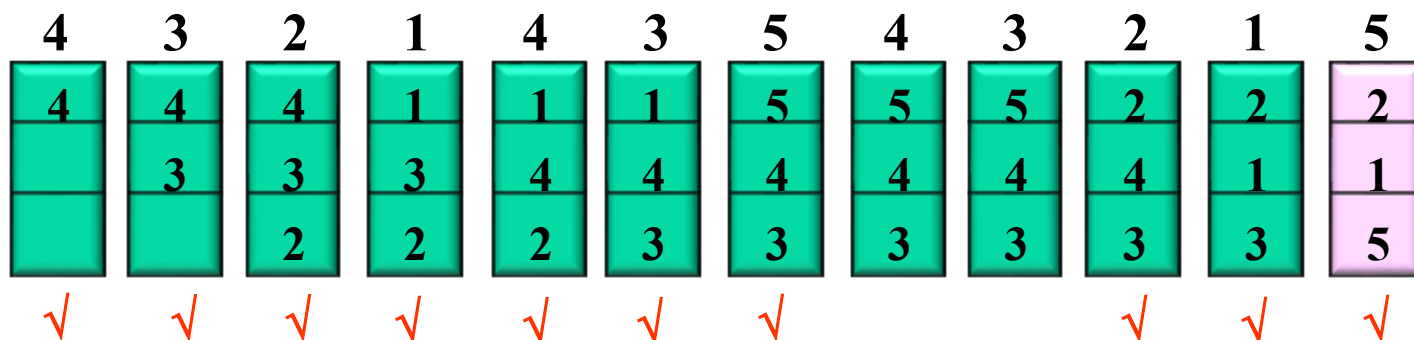




8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)



R_2 R_1 R_0

1	1	
2		1
3		
4		
5	1	

移位寄存器状态



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法

● 支持结构：特殊栈

- 利用一个特殊的栈保存当前使用的各页面的页面号
- 进程访问某页面时，将该页面的页面号从栈中移出，压入栈顶
- 置换时，淘汰栈底的

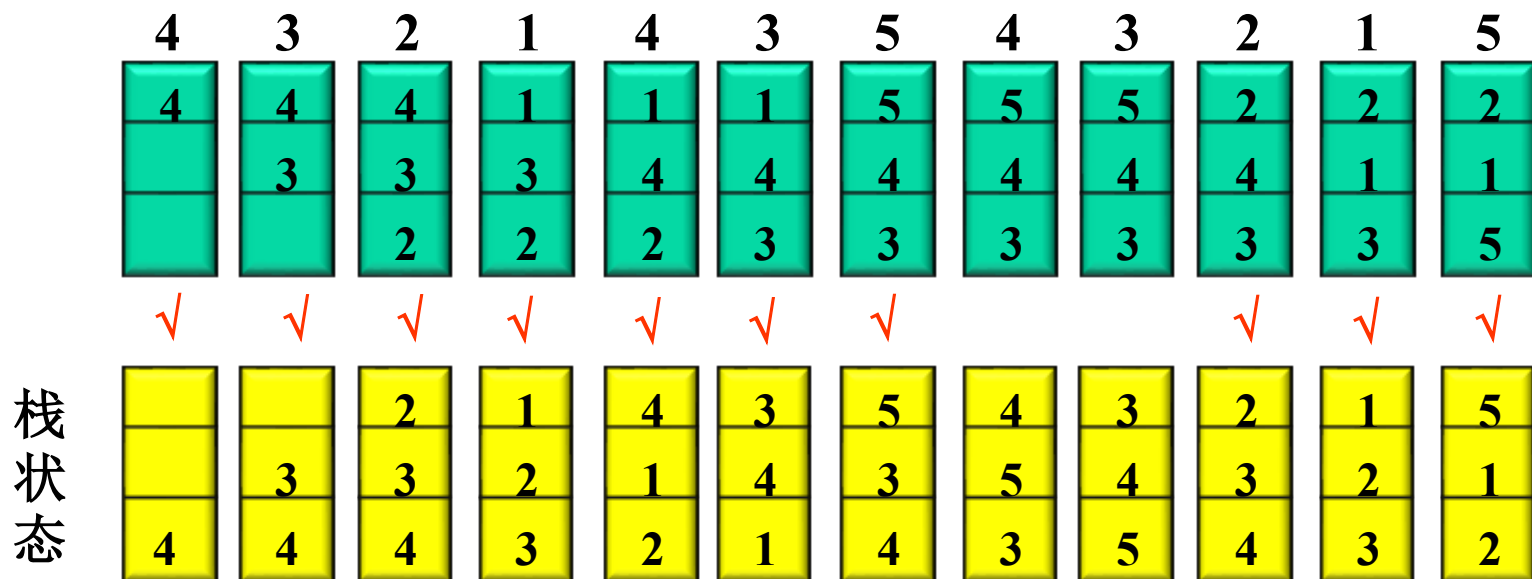


8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(2) 最近最久未使用算法 (LRU)

基于栈的LRU实现:





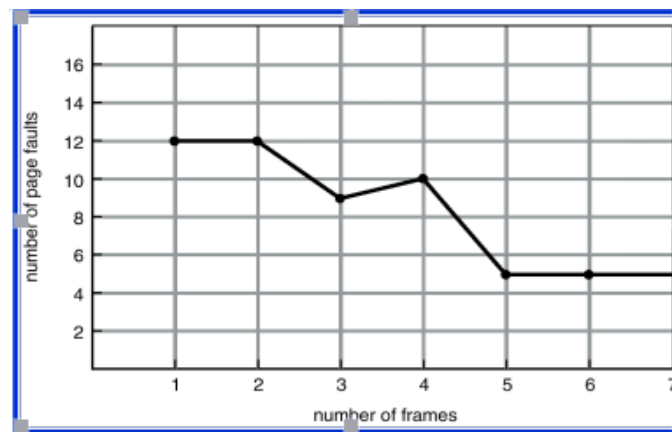
8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

(3) 先进先出算法 (FIFO)

- 选择建立最早、在内存驻留时间最长的页面被置换
- 可以通过链表来表示各页的建立时间先后
- 性能较差。较早调入的页往往是经常被访问的页，这些页在FIFO算法下被反复调入调出
- 并且有Belady现象
 - **Belady现象**：采用FIFO算法时，如果对一个进程未分配它所要求的全部页面，有时就会出现分配的页面数增多，缺页率反而提高的异常现象





8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

(3) 先进先出算法 (FIFO)

● Belady现象的示例

有5个页的进程P, 访页顺序为: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5;
如果在内存中给其分配3个页面, 则缺页情况如下: 12次
访问中有缺页9次;

FIFO	1	2	3	4	1	2	5	1	2	3	4	5
	1	1	1	4	4	4	5	5	5	5	5	5
		2	2	2	1	1	1	1	1	3	3	3
			3	3	3	2	2	2	2	2	4	4
	○	○	○	○	○	○				○	○	



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(3) 先进先出算法 (FIFO)

● Belady现象的例子

如果在内存中分配4个页面，则缺页情况如下：12次访问中有缺页10次；

FIFO	1	2	3	4	1	2	5	1	2	3	4	5
	1	1	1	1	1	1	5	5	5	5	4	4
		2	2	2	2	2	2	1	1	1	1	5
			3	3	3	3	3	3	2	2	2	2
				4	4	4	4	4	4	3	3	3
	○	○	○	○			○	○	○	○	○	○



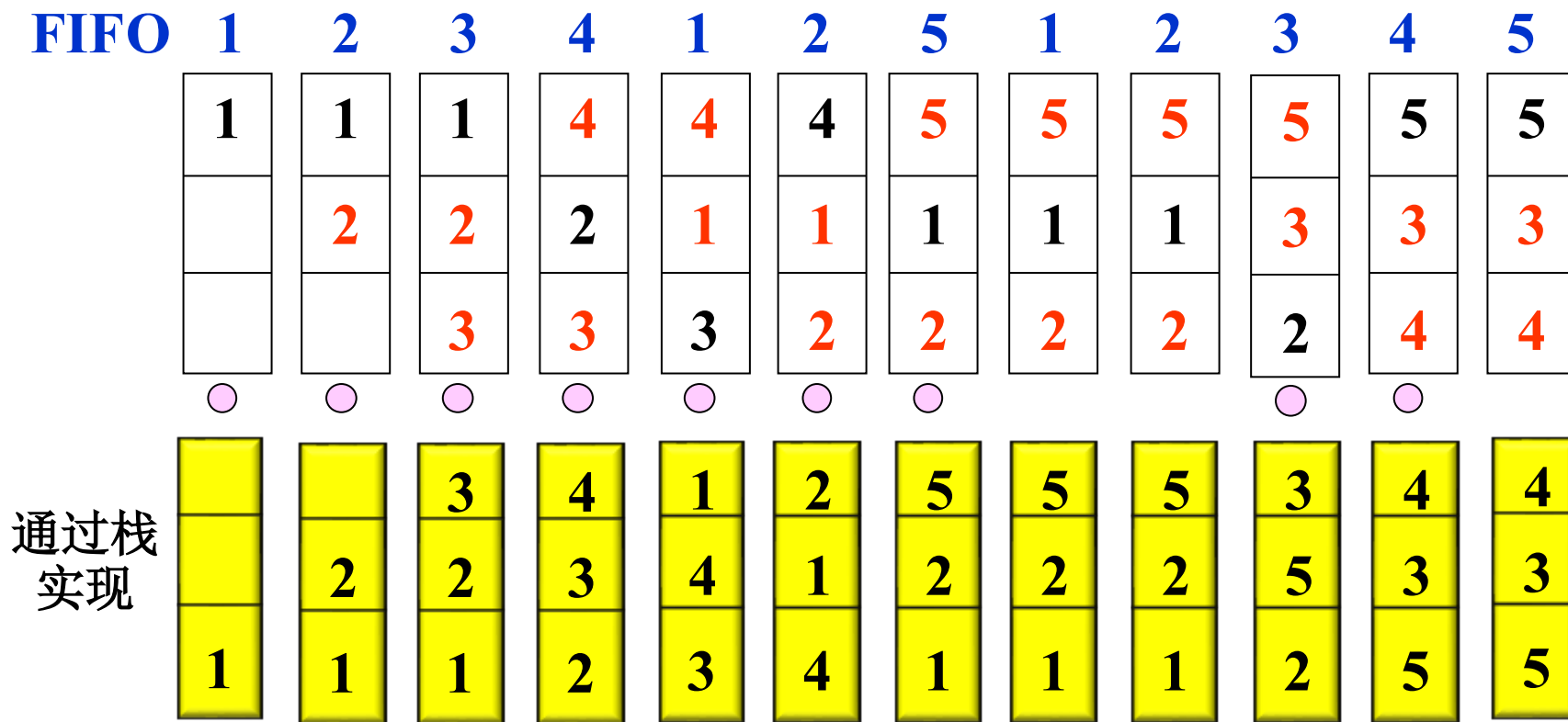
8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

(3) 先进先出算法 (FIFO)

- 基于栈的FIFO实现



LRU保持最新访问的页在栈顶, FIFO保持最先进入主存的页在栈底



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)

- LRU的一种近似算法,
- 为每页面设置一位访问位, 初始时的访问位为0; 某页被访问时, 其访问位被置1;
- 将内存中为该进程分配的所有页面链接成一个循环队列; 设置一个指针, 指向下一次首先访问的页面, 即指向最“老”的页面;

算法: 一个访页请求

step1: 如果要访问的页在内存, 则其访问位置1, 指针不动;

step2: 如果缺页, 需要置换时:

- ① 检查表针指向页的访问位: 为1则将其置0, 且指针下移, 按照FIFO顺序循环检查各页, 直到找到一个页的访问位为0;
- ② 将指针指向的页换出, 装入新页, 其访问位置1, 指针下移。



图3-16 时钟页面置换算法



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)

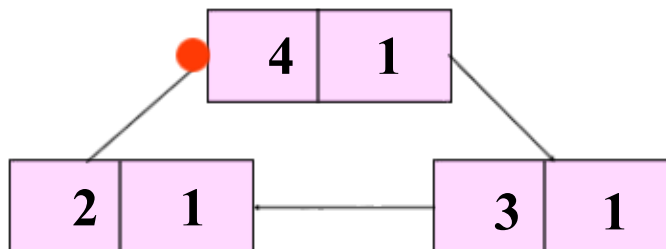
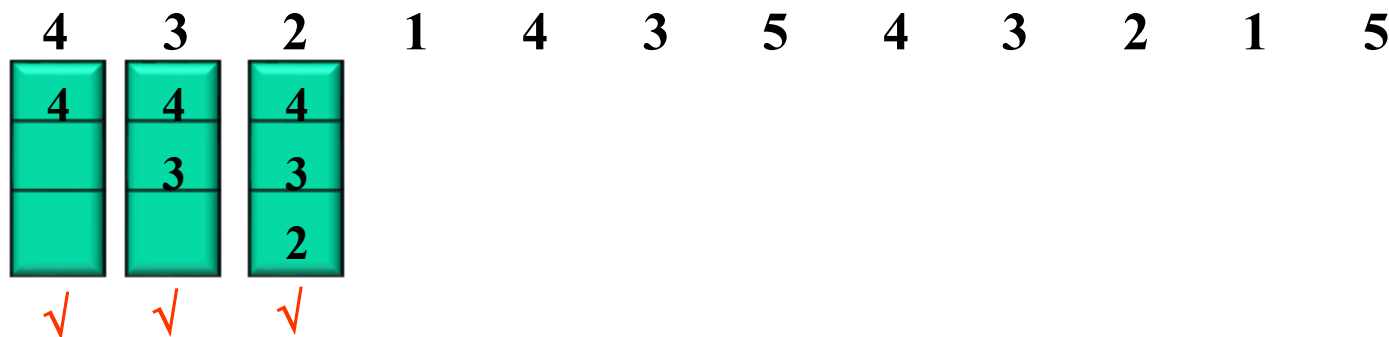
算法：一个访页请求

Step1: 如果要访问的页在内存，则其访问位置1，指针不动；

Step2: 如果缺页，需要置换时：

① 检查表针指向页的访问位：为1则将其置0，且指针下移，按照FIFO顺序循环检查各页，直到找到一个页的访问位为0；

② 将指针指向的页换出，装入新页，其访问位置1；然后指针下移；



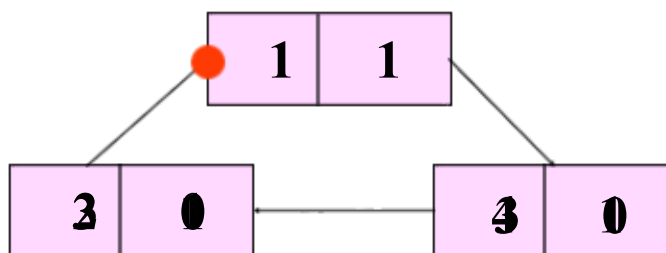
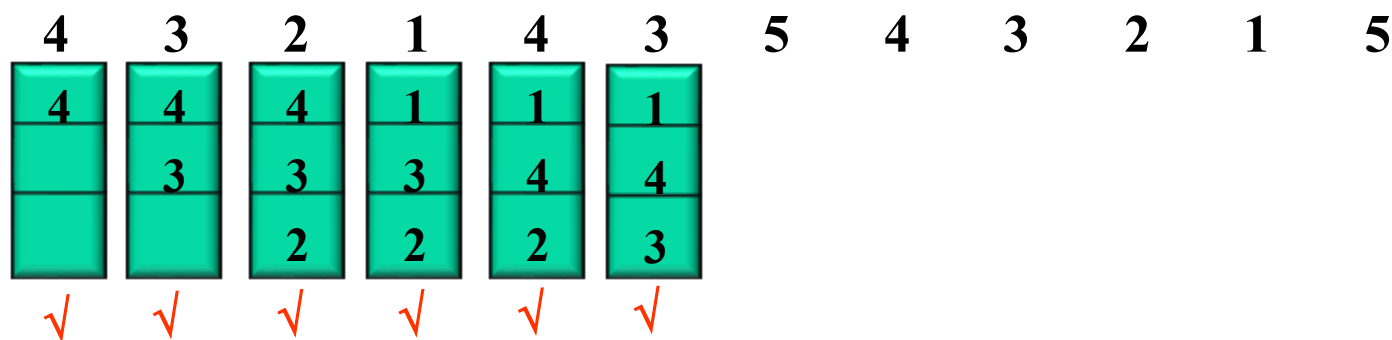




8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)

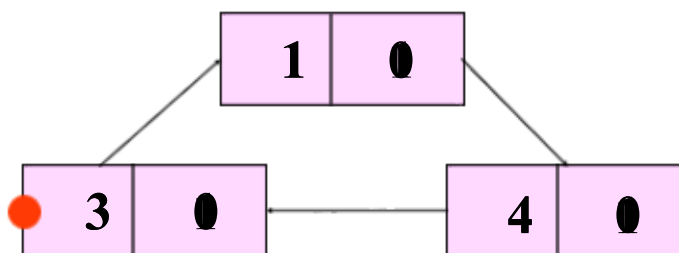
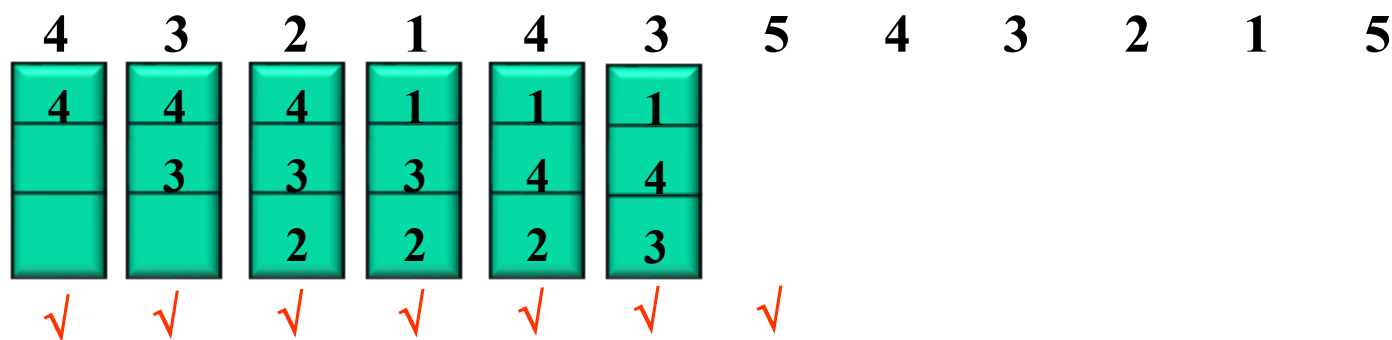




8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)

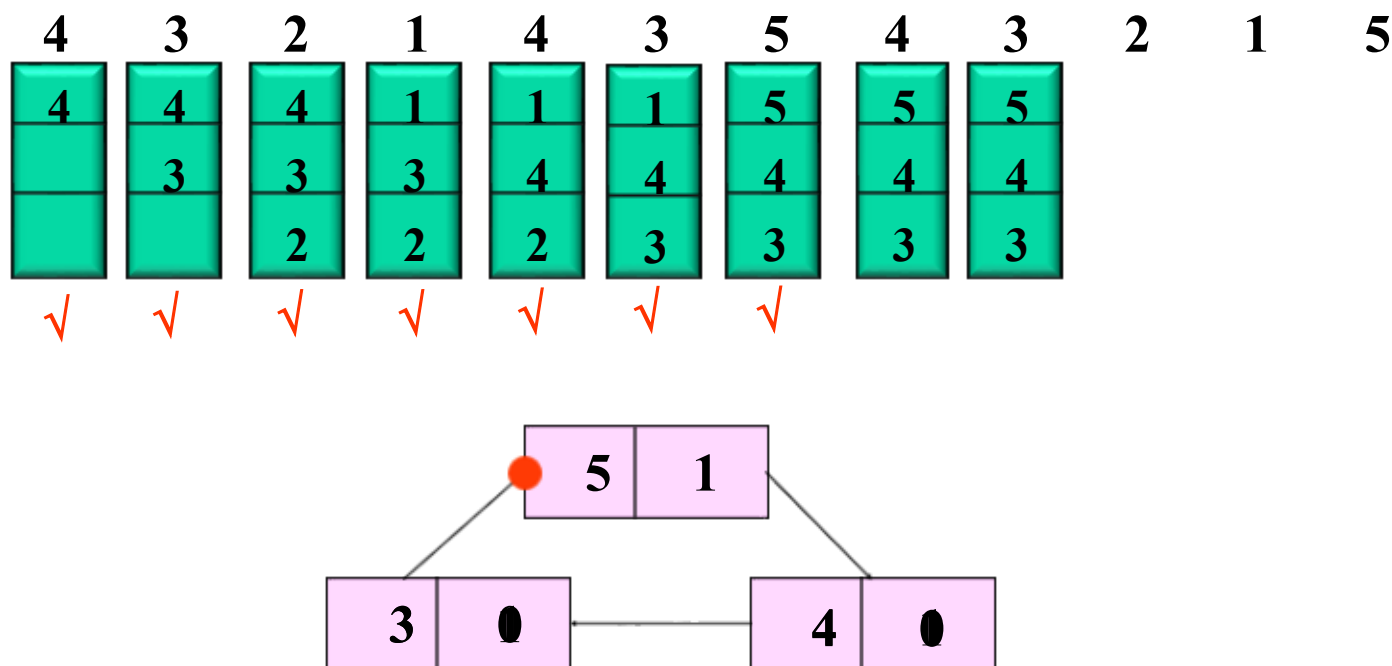




8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)

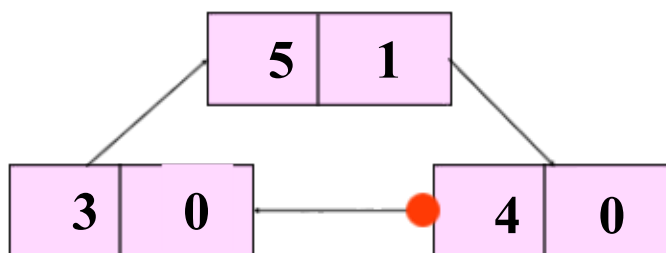
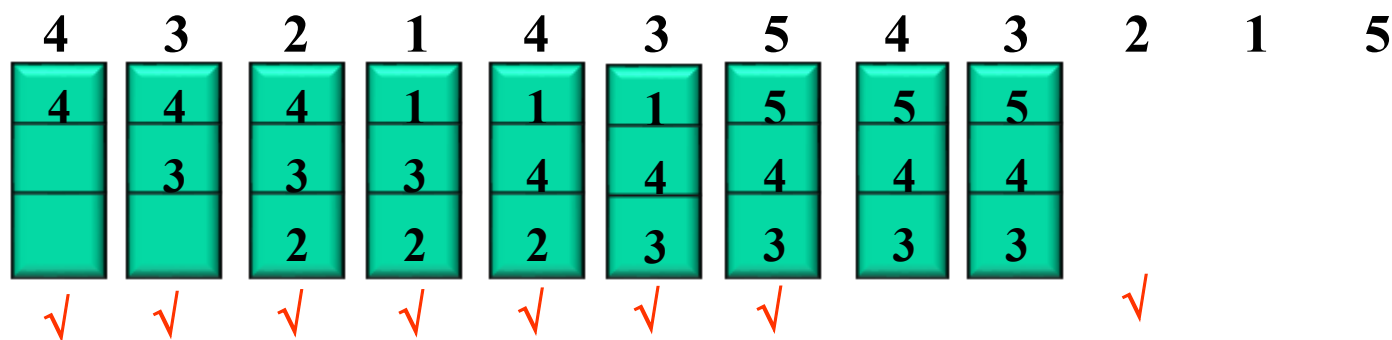




8.5 虚拟存储器

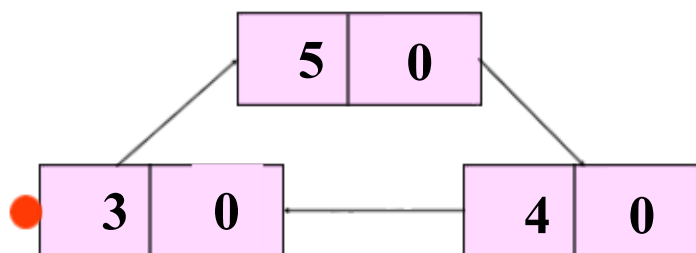
8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)





8.5.6 置換算法 (replacement policy)

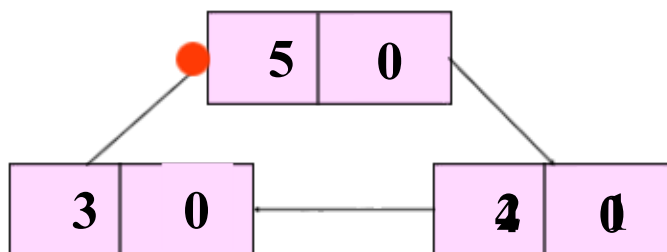
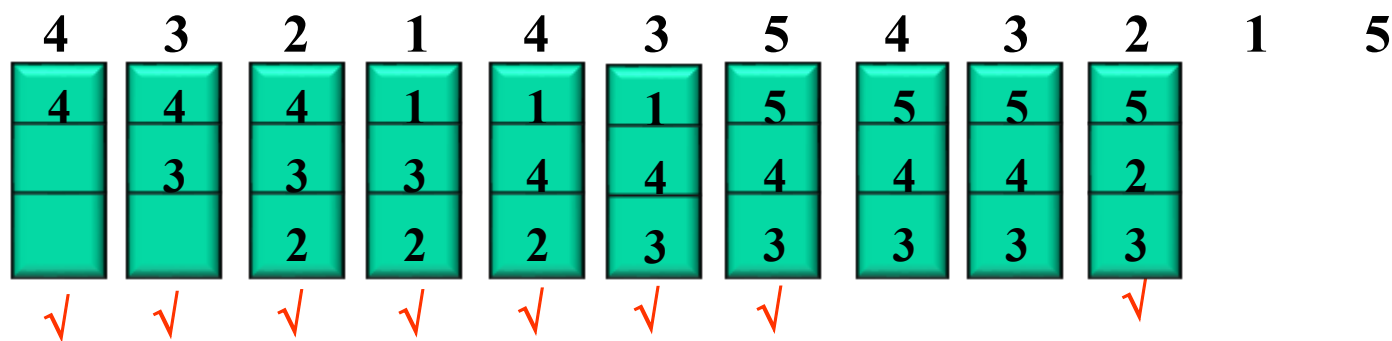




8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)

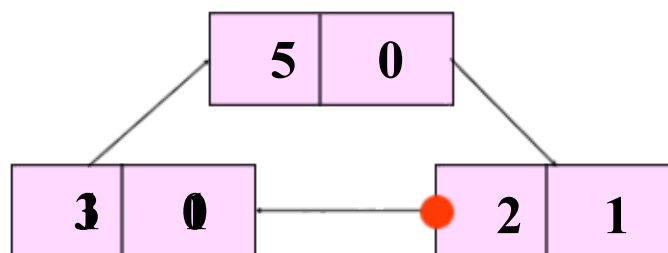
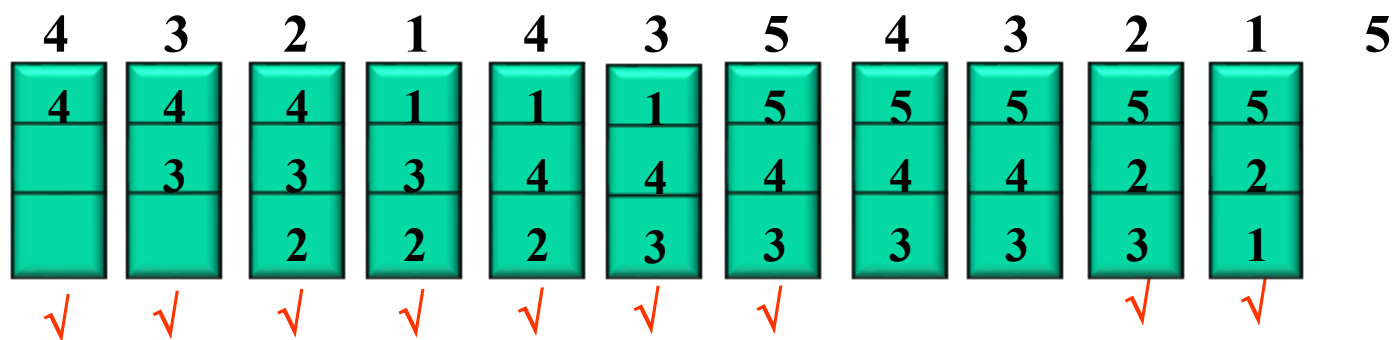




8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)

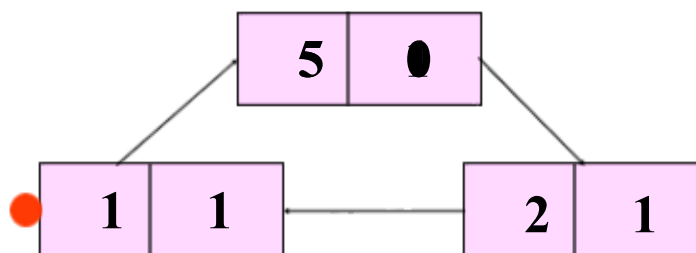
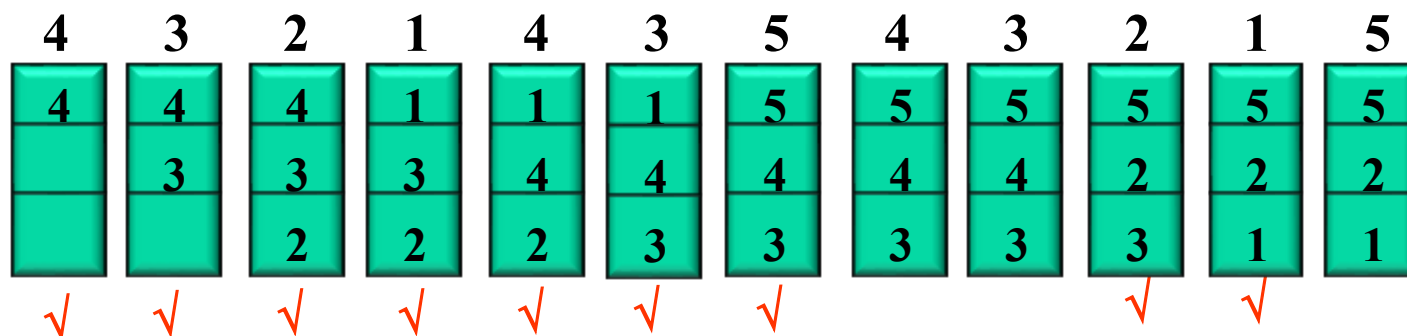




8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(4) 时钟置换算法 (CLOCK)



- 若存在未被访问过的页，该页被换出—LRU
- 若所有的页均被访问过，先进入的被换出—FIFO



8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

(4) 改进型CLOCK置换算法

- 考虑页面使用情况和置换代价
- 设置访问位A和修改位M
 - 1类(A=0, M=0): 未被访问, 未被修改, 最佳淘汰页;
 - 2类(A=0, M=1): 未被访问, 被修改, 淘汰页;
 - 3类(A=1, M=0): 被访问, 未被修改, 可再被访问;
 - 4类(A=1, M=1): 被访问, 被修改, 可再被访问。



8.5 虚拟存储器



8.5.6 置换算法 (replacement policy)

(4) 改进型CLOCK置换算法

● 置换算法:

- Step 1: 找1类, 找到则淘汰; 否则,
- Step 2: 找2类, 找到则淘汰; 查找期间, 将所有扫描过的页面的A置0; 若未找到2类,
- Step 3: 将所有页面的A复位为0, 重复Step 1



8.5 虚拟存储器

8.5.6 置换算法 (replacement policy)

(5) 最不常用算法(LFU, Least Frequently Used)

- 选择到当前时间为止被访问次数最少的页面被置换
- 每页设置访问计数器，每当页面被访问时，该页面的访问计数器加1
- 发生缺页中断时，淘汰计数值最小的页面，并将所有计数清零



- 虚拟存储器
 - 虚拟页式
 - 虚拟段式
 - 虚拟段页式
 - **分配策略**: 固定分配 vs 可变分配
 - **调入策略**: 请求调入 vs 预调入
 - **置换算法**: **OPT、FIFO、LRU、LFU、Clock**



8.5 虚拟存储器

例题1:

● 请求分页管理系统中，假设某进程的页表内容如下表所示：

页面大小为4KB，一次内存的访问时间是100ns，一次快表（TLB）的访问时间是10ns，处理一次缺页的平均时间为 10^8 ns（已含更新TLB和页表的时间），进程的驻留集大小固定为2，采用**最近最少使用**置换算法（LRU）和局部淘汰策略。

假设，①TLB初始为空；②地址转换时先访问TLB，若TLB未命中，再访问页表（忽略访问页表之后的TLB更新时间）；③有效位为0表示页表不在内存，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。

设有虚地址访问序列2362H、1565H、25A5H，请问：

(1) 依次访问上述三个虚地址，各需要多少时间，给出计算过程；

(2) 基于上述访问序列，
虚地址1565H的物理地址
是多少？请说明理由。

页号	页框(page frame)号	有效位(存在位)
0	101H	1
1	-	0
2	254H	1



8.5 虚拟存储器

例1:

- ① 页面大小为4KB，所以2362H、1565H、25A5H对应的页号分别为2、1、2，根据页表内容，2362H、25A5H在内存，1565H不在内存。
- ① 初始快表为空，故2362H需装入快表，25A5H直接访问快表，1565H需装入内存，再装入快表。据此：
 - ① 2362H: $10(\text{访TLB}) + 100(\text{访页表}) + 100(\text{访主存单元}) = 210\text{ns}$
1565H: $10(\text{访TLB}) + 100(\text{访页表}) + 10^8(\text{调页}) + 10(\text{访TLB}) + 100(\text{访主存单元}) = 100000220\text{ns}$
25A5H: $10(\text{访TLB}) + 100(\text{访主存单元}) = 110\text{ns}$
 - ② 1565H的物理地址是多少？
 - ✓ 根据页表，第0、2页在内存中，由于第2页刚刚被访问（访问2362H），根据LRU算法，将第0页替换，因此对应页框号为101H，故1565H的物理地址为101565H。



8.5 虚拟存储器

例题2:

在一采取局部置换策略的请求分页系统中，分配给某个作业的内存块数为4，其中存放的4个页面的情况如表所示：

物理块	虚页号	装入时间	最后一次访问时间	访问位	修改位
0	2	60	157	0	1
1	1	160	161	1	0
2	0	26	158	0	0
3	3	20	163	1	1

上面所有时间均是从进程开始运行时从0开始计时的时钟数，如果系统采用：(1) FIFO算法；(2) LRU算法；(3) 改进的Clock算法，将选择哪一页换出？

- (1) 物理3、虚拟3 (2) 物理0、虚拟2 (3) 物理2、虚拟0



8.5 虚拟存储器

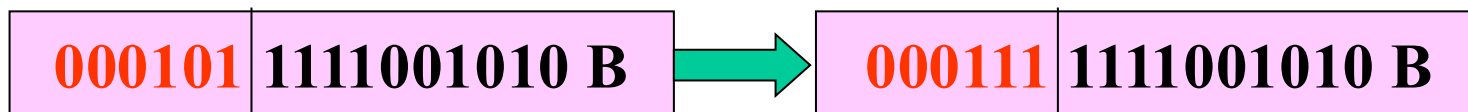
例题3:

➤ 设某计算机的逻辑地址空间和物理地址空间均为64KB，按字节编址。若进程最多需要6页存储空间，页的大小为1KB，OS若采用**固定分配局部置换策略**为此进程分配4个页面。在时刻260前的该进程访问情况如表所示（访问位即使用位）

● 当该进程执行到时刻260时，要访问逻辑地址为17CAH的数据，**请回答：**

- 1、逻辑地址对应的页号是多少
- 2、若采用FIFO置换算法，该逻辑地址对应的物理地址是多少？给出计算过程。

页号	页面号	装入时间	访问位
0	7	130	1
1	4	230	1
2	2	200	1
3	9	160	1





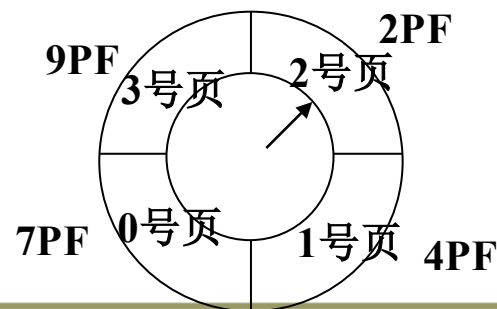
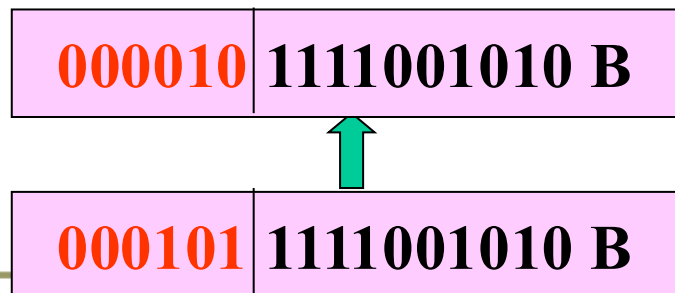
8.5 虚拟存储器

例题3:

设某计算机的逻辑地址空间和物理地址空间均为64KB，按字节编址。若进程最多需要6页存储空间，页的大小为1KB，操作系统若采用固定分配局部置换策略为次进程分配4个页面。在时刻260前的该进程访问情况如表所示（访问位即使用位）

页号	页面号	装入时间	访问位
0	7	130	1
1	4	230	1
2	2	200	1
3	9	160	1

3、若采用**时钟置换**算法，该逻辑地址对应的物理地址是多少？给出计算过程。设顺时针搜索下一页，且当前指向2号页面，示意图如下。



解：
0BCAH



8.5 虚拟存储器



- **例4：**某请求分页系统的局部页面置换策略如下：系统从0时刻开始扫描，每隔5个时间单位扫描一轮驻留集（扫描时间忽略不计），本轮**没有被访问**过的页框将被系统回收，并放入**空闲页框链尾**，其中内容在下一次被分配之前不被清空。当发生缺页时，如果该页曾被使用过且还在空闲页框链表中，则重新放回进程的驻留集中；否则，从空闲页框链表头部取出一个页框。
- 假设不考虑其他进程的影响和系统开销，初始时进程驻留集为空。目前空闲页框链表中页框号依次为32、15、21、41。进程P依次访问的**<虚拟页号，访问时刻>**是：<1,1>、<3,2>、<0,4>、<0,6>、<1,11>、<0,13>、<2,14>。
- 请回答下列问题。
 - 1) 访问<0,4>时，对应的页框号是多少？
 - 2) 访问<1,11>时，对应的页框号是多少？说明理由。
 - 3) 访问<2,14>时，对应的页框号是多少？说明理由。
 - 4) 该策略是否适合于时间局部性能好的程序？说明理由。



8.5 虚拟存储器



● 思考:

空闲页框链表: 32、15、21、41

访问序列: $\langle 1,1 \rangle$ 、 $\langle 3,2 \rangle$ 、 $\langle 0,4 \rangle$ 、 $\langle 0,6 \rangle$ 、 $\langle 1,11 \rangle$ 、 $\langle 0,13 \rangle$ 、 $\langle 2,14 \rangle$

$\langle 1,1 \rangle$: 空闲页框链表: 15、21、41; 驻留集: 32:1

$\langle 3,2 \rangle$: 空闲页框链表: 21、41; 驻留集: 32:1, 15:3

$\langle 0,4 \rangle$: 空闲页框链表: 41; 驻留集: 32:1, 15:3, 21:0

时刻5 (扫描): 空闲页框链表: 41; 驻留集: 32:1, 15:3, 21:0

$\langle 0,6 \rangle$: 空闲页框链表: 41; 驻留集: 32:1, 15:3, 21:0

时刻10 (扫描): 空闲页框链表: 41, 32:1, 15:3; 驻留集: 21:0

$\langle 1,11 \rangle$: 空闲页框链表: 41, 15:3; 驻留集: 21:0, 32:1

$\langle 0,13 \rangle$: 空闲页框链表: 41, 15:3; 驻留集: 21:0, 32:1

$\langle 2,14 \rangle$: 空闲页框链表: 15:3; 驻留集: 21:0, 32:1, 41:2



8.5 虚拟存储器

<1,1> : 空闲页框链表: 15、21、41; 驻留集: 32:1

<3,2> : 空闲页框链表: 21、41; 驻留集: 32:1, 15:3

<0,4> : 空闲页框链表: 41; 驻留集: 32:1, 15:3, 21:0

<0,6> : 空闲页框链表: 41; 驻留集: 32:1, 15:3, 21:0

<1,11> : 空闲页框链表: 41, 15:3 ; 驻留集: 21:0, 32:1

<0,13> : 空闲页框链表: 41, 15:3 ; 驻留集: 21:0, 32:1

<2,14> : 空闲页框链表: 15:3 ; 驻留集: 21:0, 32:1, 41:2

- 1) 访问<0,4>时, 对应的页框号是多少? 2) 访问<1,11>时, 对应的页框号是多少? 说明理由。3) 访问<2,14>时, 对应的页框号是多少? 说明理由。4) 该策略是否适合于时间局部性能好的程序? 说明理由。



8.5 虚拟存储器



● 答案

- 1) 页框号为21（1分）
- 2) 页框号为32（1分）；理由：因 $11 > 10$ 故发生第三轮扫描，页号为1的页框在第二轮已处于空闲链表中，此刻该页又被重新访问，因此应被重新放回驻留集中，其页框号为32（1分）
- 3) 页框号为41（1分）；理由：因第2页从来没被访问过，它不再驻留集中，因此从空闲页框链表中取出链表头的页框41，页框号为41（1分）
- 4) 适合（1分）；理由：如果程序的时间局部性能越好，从空闲页框表中重新取回的机会越大，该策略的优势越明显。



8.5 虚拟存储器

- 某计算机系统按字节编址，采用二级页表的分页存储管理方式，虚拟地址格式如下所示：



- 请回答如下问题

- ① 页和页框的大小各为多少字节？进程的虚拟地址空间大小为多少页？
- ② 假定页目录项和页表均占4个字节，则进程的页目录和页表共占多少页？要求写出计算过程
- ③ 某指令周期内访问的虚地址为01000000H和01112048H，则进行地址转换时共访问多少个二级页表？要求说明理由。



8.5 虚拟存储器

10位	10位	12位
页目录号	页表索引	页内偏移量

- ① 页和页框的大小各为多少字节？进程的虚拟地址空间大小为多少页？

页=页框= $2^{12}=4\text{KB}$ ，进程的虚拟地址空间 $2^{32}/2^{12}=2^{20}$ 页

- ② 假定页目录项和页表均占4个字节，则进程的页目录和页表共占多少页？要求写出计算过程

页目录所占页数： $(2^{10} \times 4) / 2^{12} = 1$ 页

页表所占页数： $(2^{20} \times 4) / 2^{12} = 1024$ 页

共占 $1 + 1024 = 1025$ 页



8.5 虚拟存储器



- ③ 某指令周期内访问的虚地址为01000000H和01112048H，则进行地址转换时共访问多少个二级页表？要求说明理由。

01000000H=(0000 0001 0000 0000 0000 0000 0000)B

 页目录号 页号

 4

01112048H=(0000 0001 0001 0001 0010 0000 0100 1000)B

 页目录号 页号

 4

访问一个二级页表



8.5 虚拟存储器



作业

● P144 习题

- 某系统采用页式存储管理策略，拥有逻辑空间32页，每页2K，拥有物理空间1M。
 - 写出逻辑地址格式
 - 若不考虑访问权限，进程的页表有多少项？每项至少多少位？
 - 如果物理空间减少一半，页表结构应相应作怎样的改变？



8.6 Linux(openEuler)存储管理(自学)



1. **Linux在x86中的分段和分页**
 - 之处分段、支持多级分页
2. **Linux物理内存管理模型及数据结构**
 - **UMA、NUMA、NODE、Zone、Page**
3. **物理内存的分配和回收**
 - 基于**BUDDY**的页框管理、基于**slab**的内存区管理
4. **内核地址空间管理**
 - 划成多中类型的区域进行内存的映射
5. **进程的用户空间管理**
 - 申请内存先分配虚拟地址段、读写时才分配物理空间
6. **页面失效处理**
 - 缺页中断**page_fault()**
7. **交换机制**
 - **kswapd**线程完成、监控几个指标、**LRU**算法



- ❖ 自80386之后，历经80486、Pentium等型号，但是结构无本质的改变，统称为**IA32**（Intel Arch. 32 bit）体系结构
- ❖ **IA64处理器I-tanium**（安腾）是Intel自推出32位微处理器以来，在高性能计算机领域的又一座里程碑。
 - 基于IA64处理器架构具有**64位**的运算能力、寻址空间和数据通路，突破了传统IA32架构的许多限制。
- ❖ 为了保证向上兼容，保留了16位段寄存器
- ❖ 只用**分页机制**就可以实现有效的内存保护机制，但是还是不得不在**分段机制**的基础上实现保护模式，
- ❖ **IA32体系结构的分段基址是向上兼容的结果**



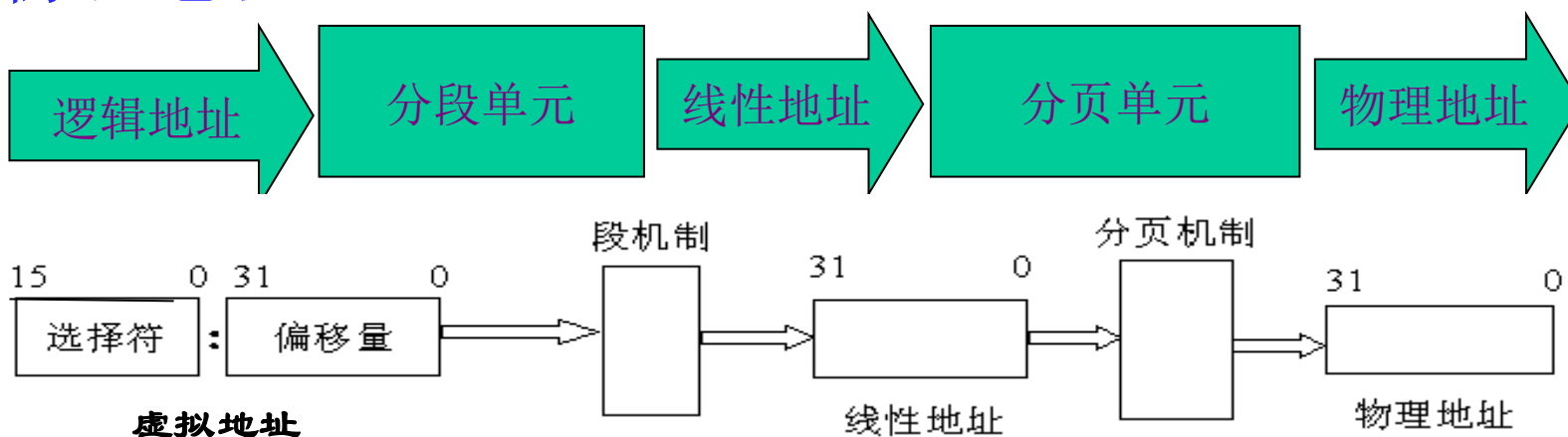
• 逻辑地址

- 机器语言指令中用来指定一个操作数或一条指令的方式或方法
- 实模式：段基址+偏移量
- 保护模式：16位段选择子(如cs)+ 32位段内偏移量

• 线性地址

- 32位的无符号整数

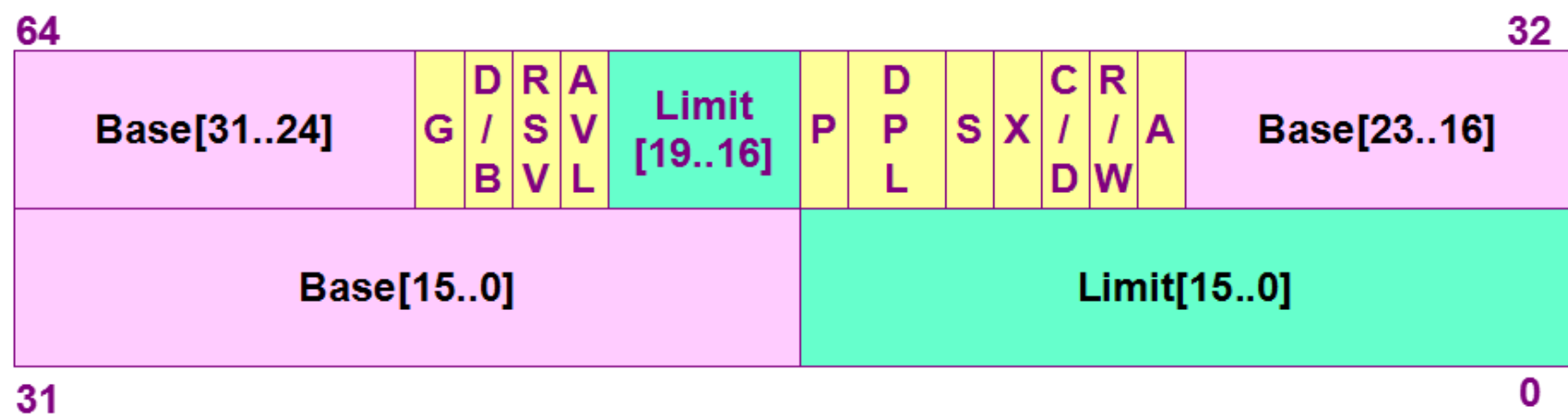
• 物理地址





分段机制-- 段描述符

使用段描述符描述段的位置信息、属性信息



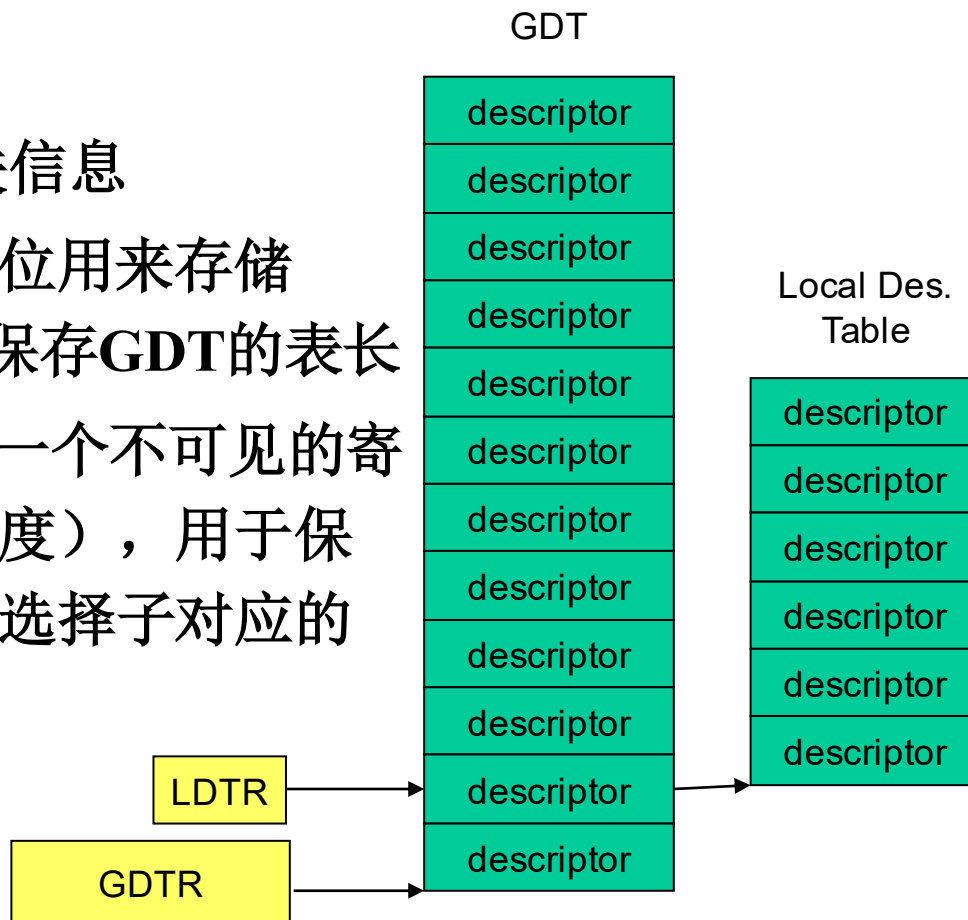
- G:** **1**—段长以页(4K)为单位分配; **0**—段长以字节为单位分配
- X:** **1**—32位代码段; **0**—16位代码段
- P:** **0**—一段在内存中; **1**—一段不在内存中
- DPL:** 描述符特权级(0—3)
- S:** **0**—系统段描述符或门描述符; **1**—代码或数据段描述符
- A:** **0**—未访问过; **1**—访问过
- R/W:** 段的类型或保护

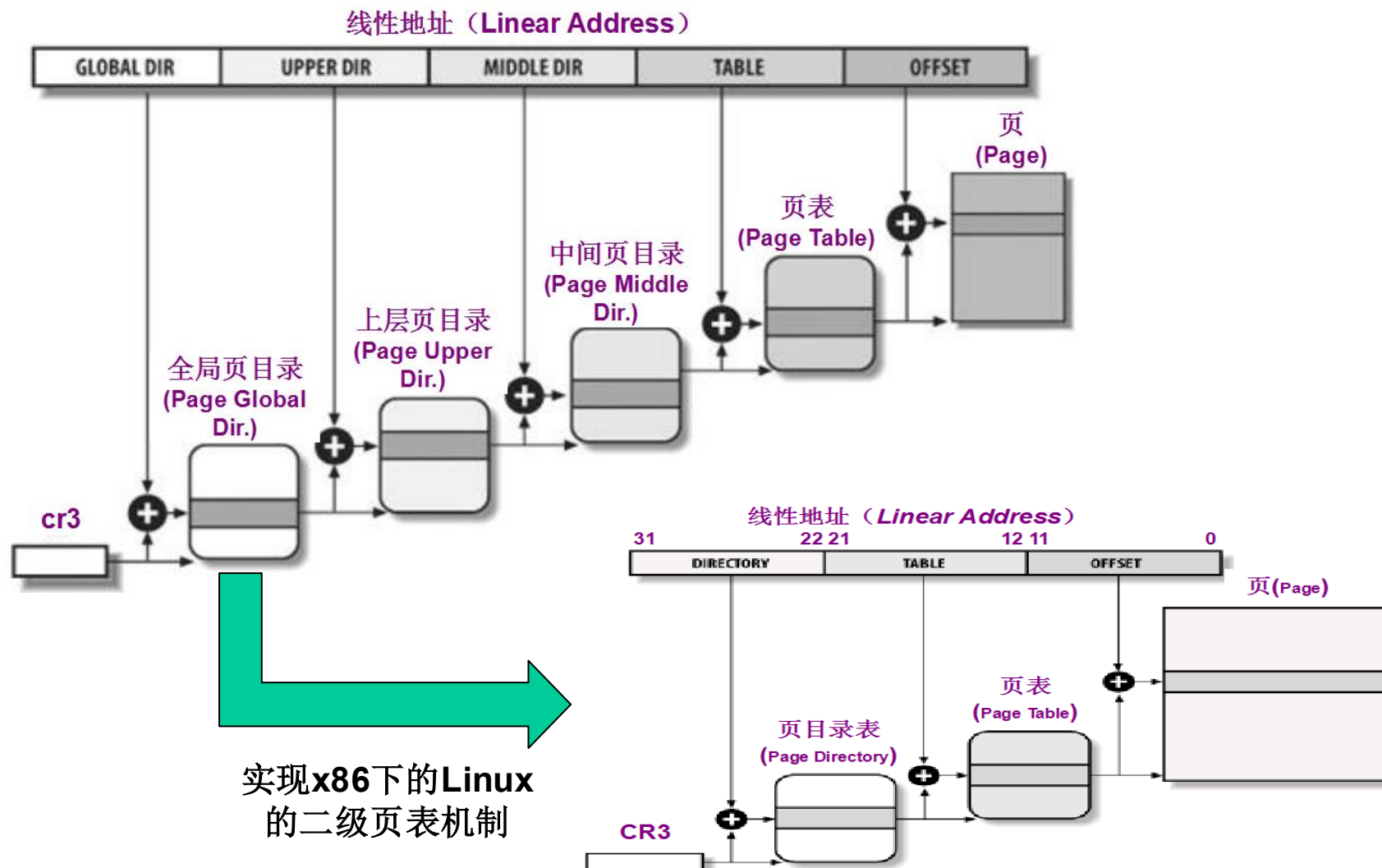


• GDT和LDT

• gdtr和ldtr

- 保存GDT和LDT的相关信息
- **GDTR**: 48bits, 其中32位用来存储GDT的基址; 另16bits保存GDT的表长
- **LDTR**: 16bits, (包含一个不可见的寄存器, 用于加快访问速度), 用于保存一个段选择子, 该段选择子对应的段中保存了LDT表



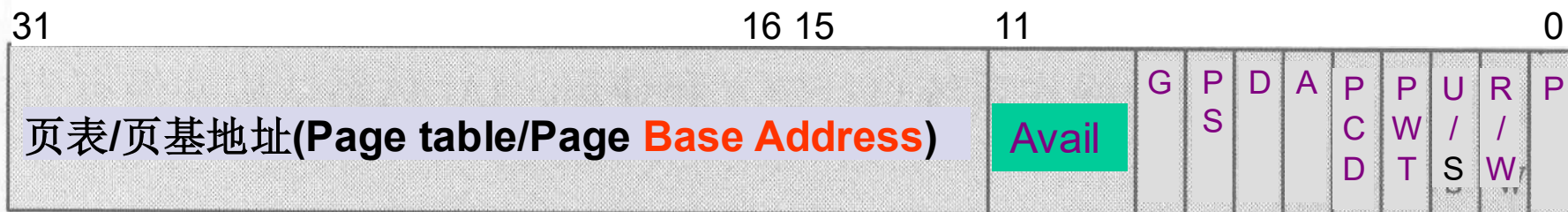




页表项结构



- 页目录项和页表项的结构一样，占4B



系统程序员使用

全局页 (Global Page)

页大小 (Page Size, 0表示4KB)

脏页 (Dirty)

访问过 (Accessed)

Cache不允许 (Cache disabled)

直写 (Write through)

用户/监督者 (User/Supervisor)

读/写 (Read/Write)

存在位 (Present)



8.6.2 内存管理模型及数据结构



提纲:

1. 内存架构
2. Linux物理内存管理模型及数据结构



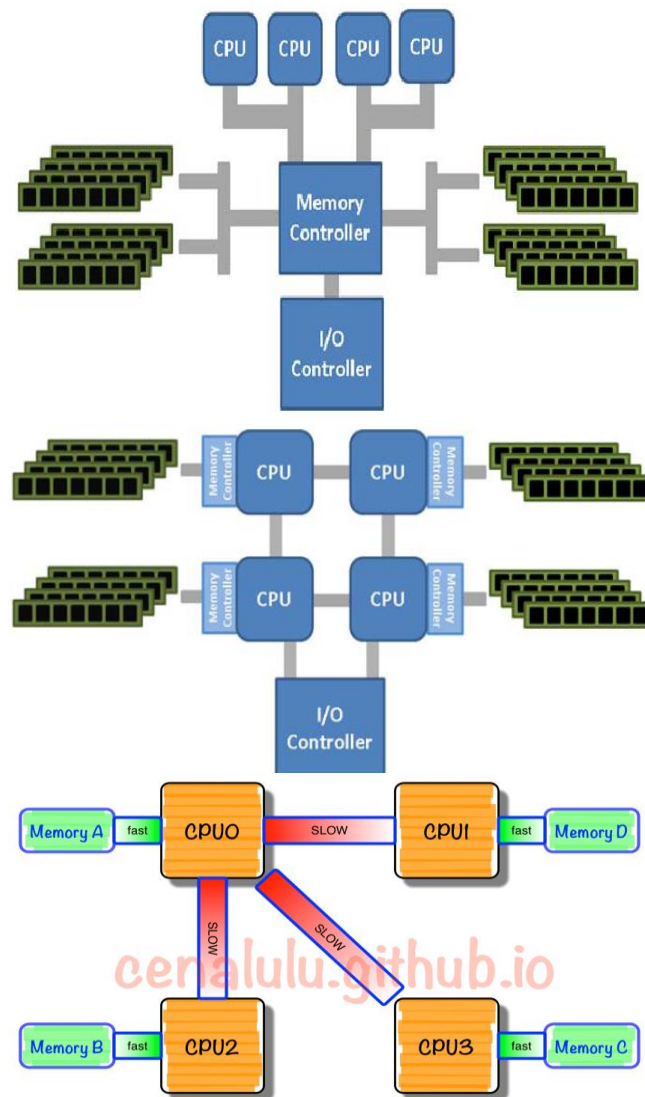
- 两种架构

- 一致内存架构UMA (Uniform Memory Access)

：这种结构对于CPU和其他外围设备访问的内存存在一块内存条上，没有任何不同，PC都是用的这种结构。

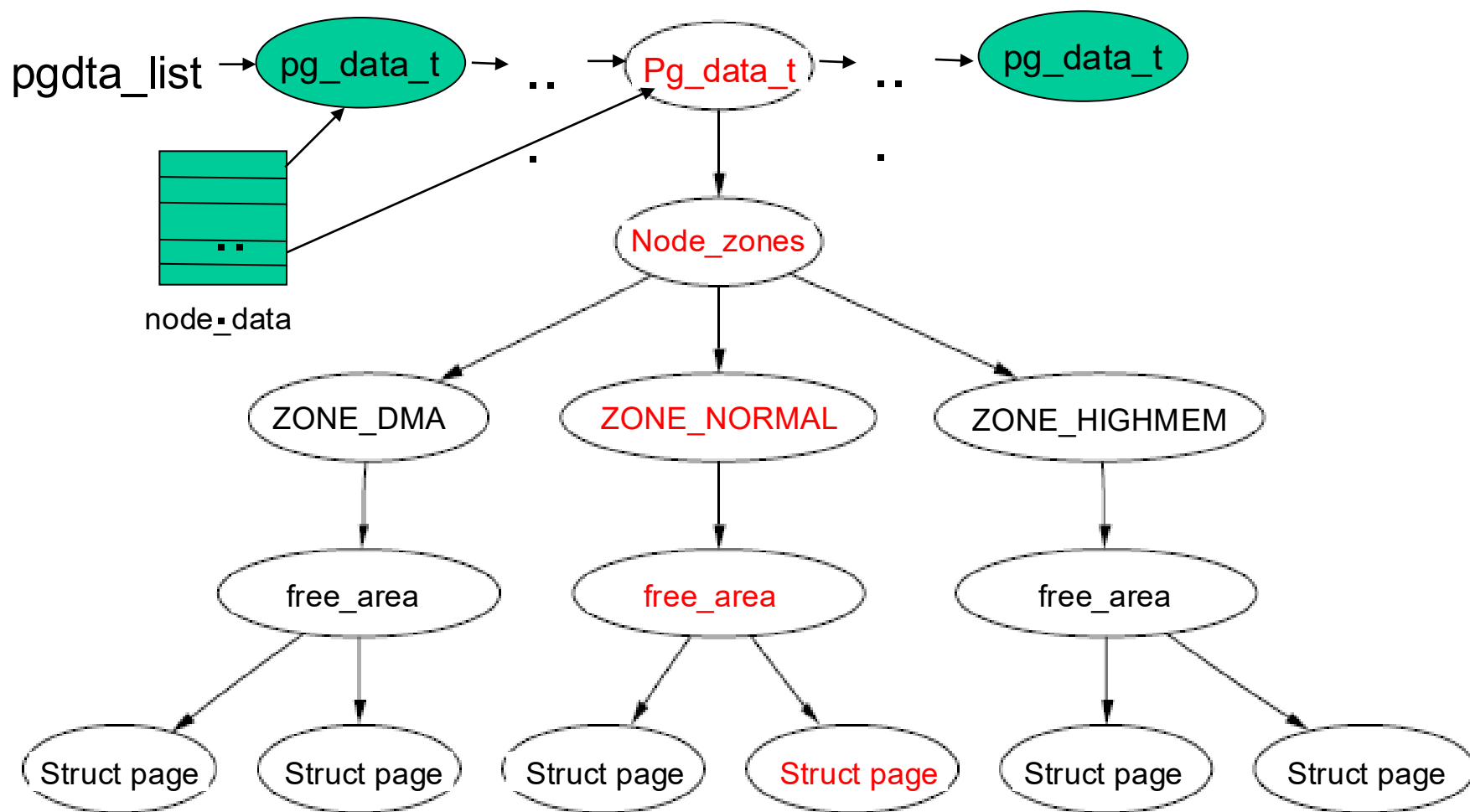
- 不一致内存架构NUMA (Non-Uniform Memory Access)：共享存储器物理上是分布在所有CPU的本地存储器上。所有本地存储器的集合组成了全局地址空间，可被所有的CPU访问。访问本地存储器是比较快的，但访问属于另一台CPU的远程存储器则比较慢。

- Linux的MM管理，采取了与具体系统架构无关的设计模型，实现了良好的可伸缩性





(2) 系统中NODE的组织





- Linux使用著名的**伙伴算法**来解决碎片问题
 - 把所有空闲页框, 按照空闲页块中页框数目, 分为11组 (`MAX_ORDER`), 形成11个空闲**块链表**, 每个块链表分别包含大小为1, 2, 4, 8, ..., 256, 512, 1024个连续的页框
 - 每个块的第一个页框的物理地址是该块大小的整数倍
 - 例如: 大小为16个页框的块, 其起址是 $16 \times 4\text{KB}$ 的倍数

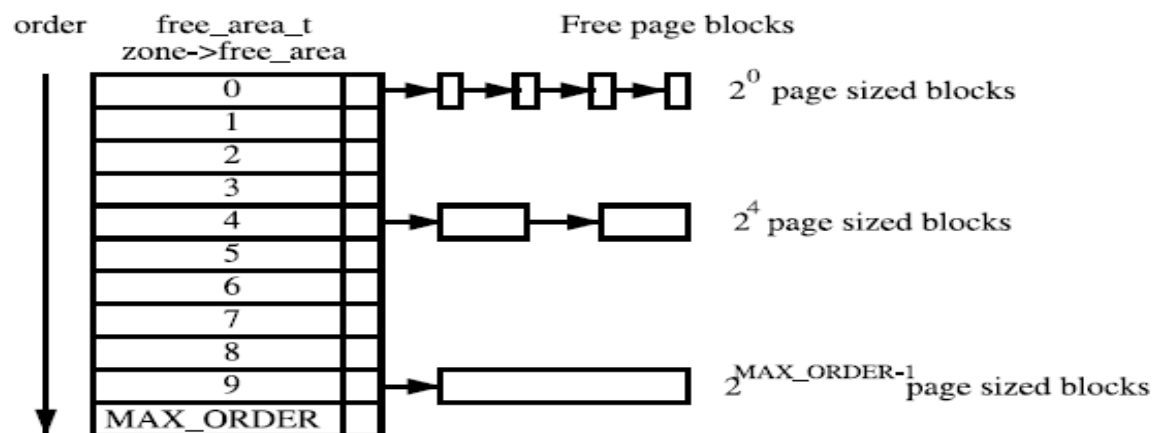


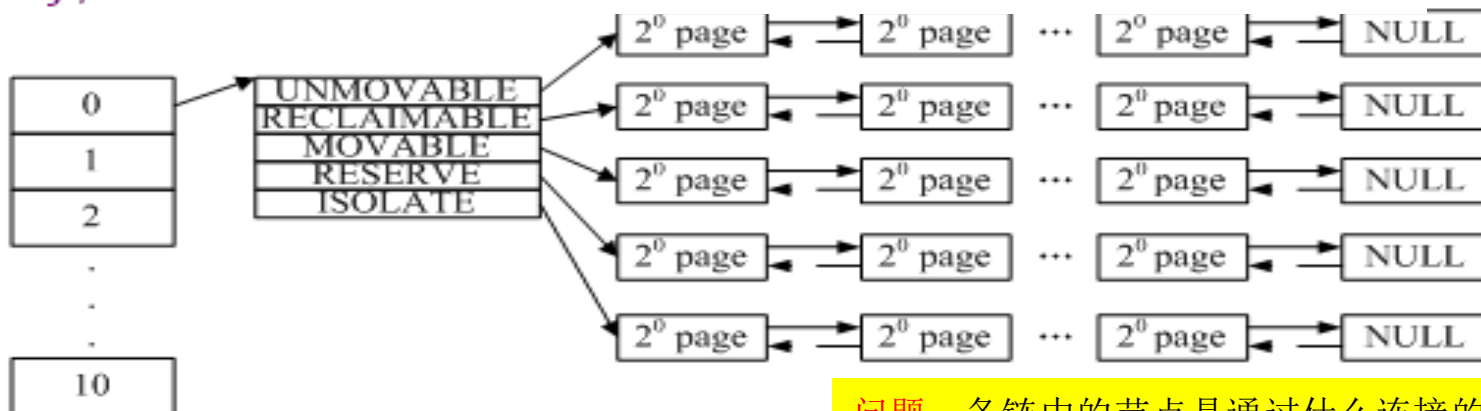
Figure 6.1. Free Page Block Management



数据结构—Buddy实现机制

- ❖ Linux为每个zone使用各自独立的伙伴系统
- ❖ 每个伙伴系统使用的主要数据结构为：
 - ❖ 在struct zone中的空闲内存管理数组free_area,
 - 空闲内存按照伙伴管理的方法进行组织

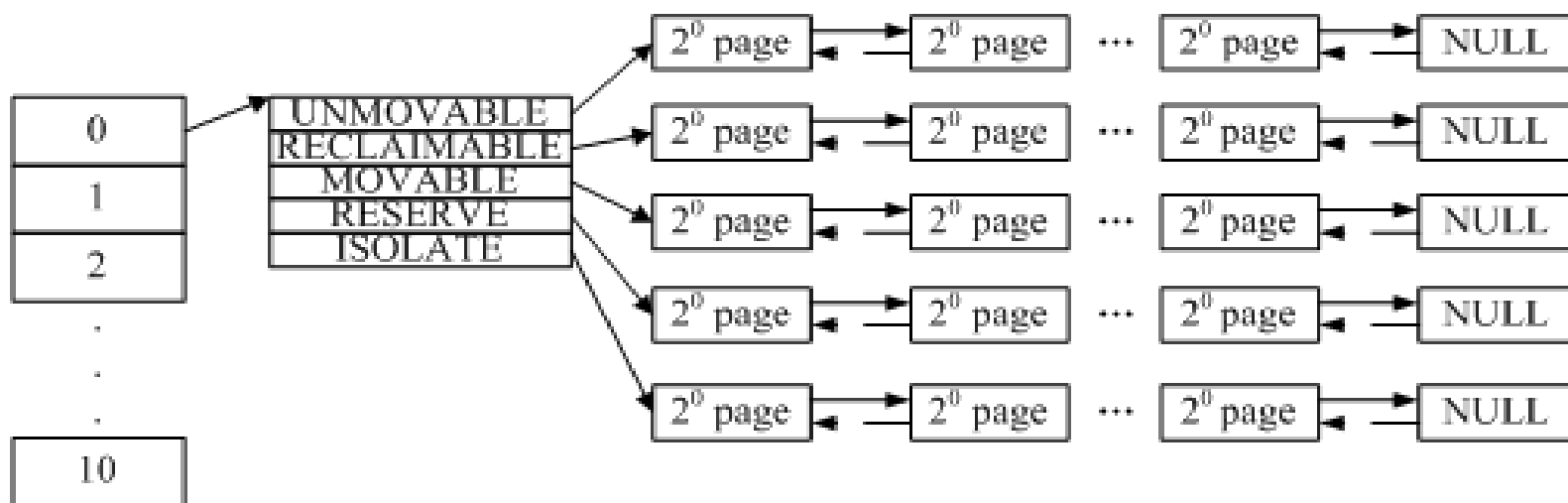
```
struct free_area    free_area[MAX_ORDER];  
  
struct free_area {  
    struct list_head    free_list[MIGRATE_TYPES];  
    unsigned long        nr_free;  
};  
include/linux/mmzone.h
```

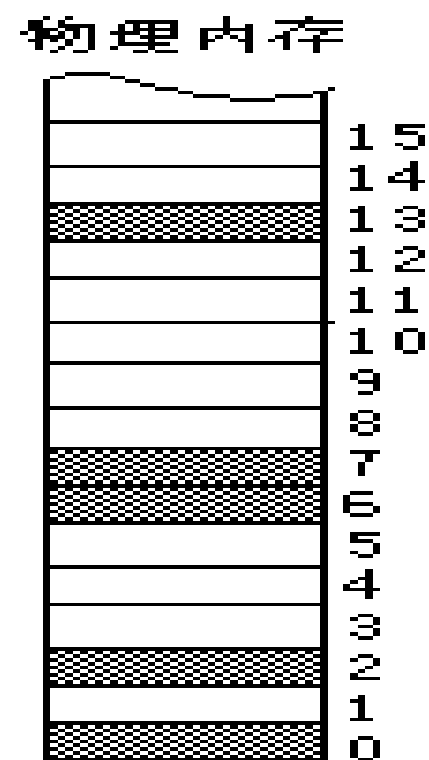
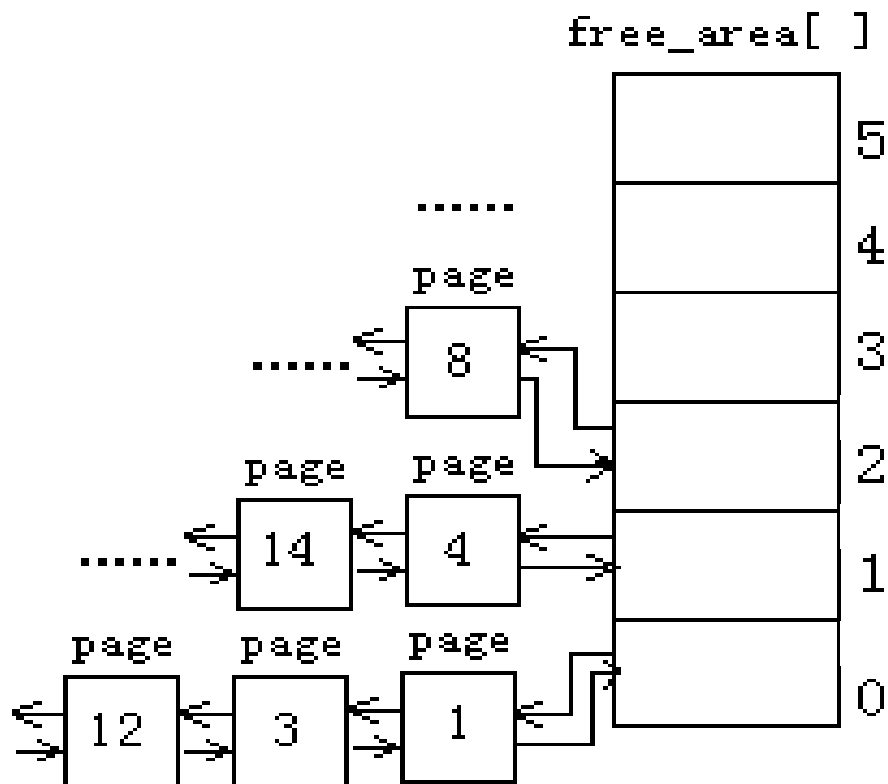


问题：各链中的节点是通过什么连接的？

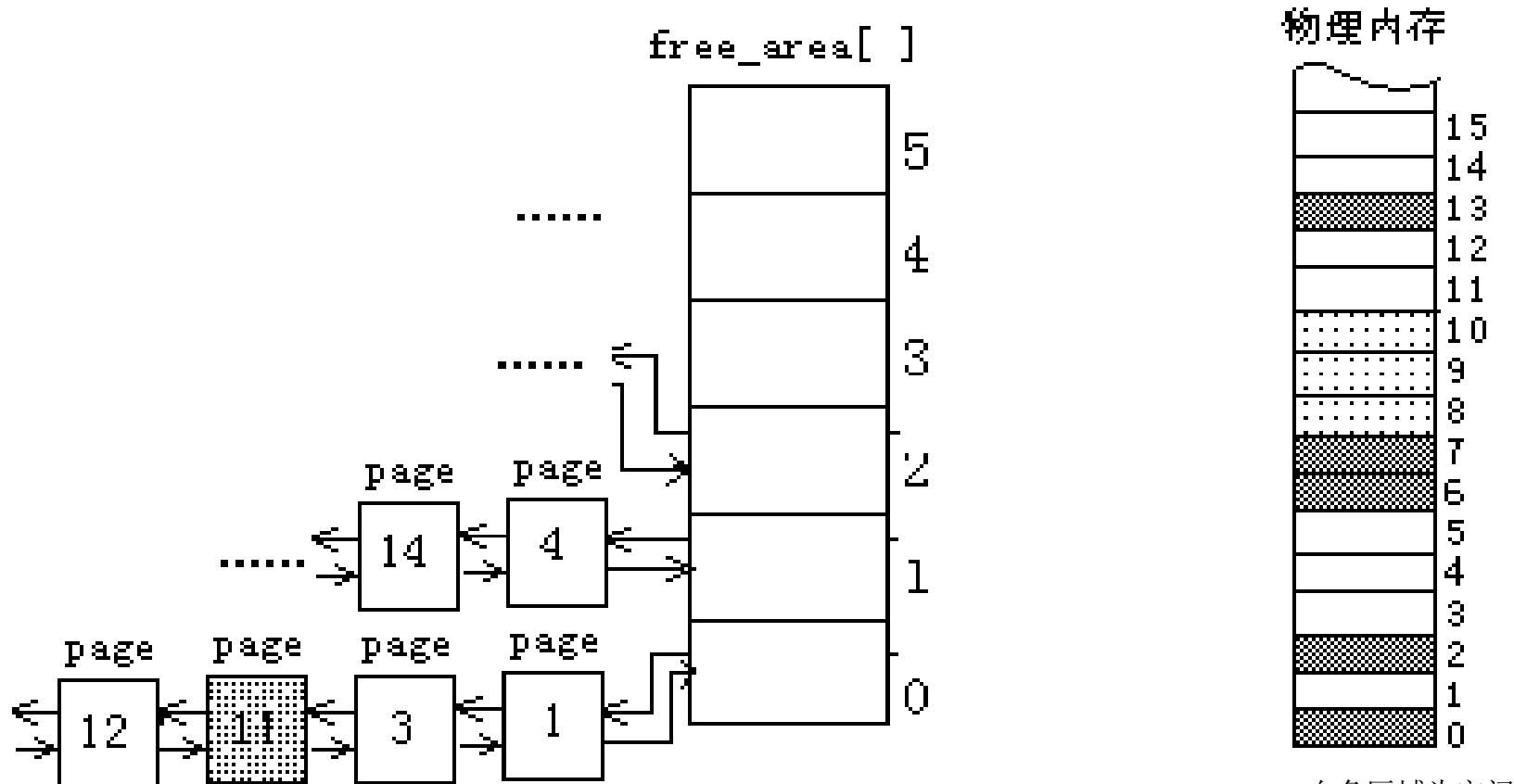


- ❖ `free_area`是一个有11个元素的数组，是各种块尺寸链表的表头
- ❖ 链中的节点是一个连续块的**第一个页框的struct page类型的描述符**，它们通过struct page中的**`lru`**字段连接起来





- 请求：申请3个页面



8、9、10页面分配后的示意

释放13号页面后，如何回收？请画出回收后的示意图



需求与问题

- 内核中大量使用各种数据结构，大小从几个字节到几十上百k不等，都取整到2的幂次个页面那是完全不现实的。只分配页面的分配器肯定是不能满足要求的
- 如何在一个页面内分配小的存储空间？
小存储空间的分配、释放导致的内碎片如何解决？
- 早期内核的解决方法是：提供大小为2, 4, 8, 16,...,132056字节的内存区域。需要新的内存区域时，内核从伙伴系统申请页面，把它们划分成一个个区域，取一个来满足需求。
- 如果某个页面中的内存区域都释放了，页面就交回到伙伴系统。
- 似乎解决问题了，还有没有可改进的地方呢？

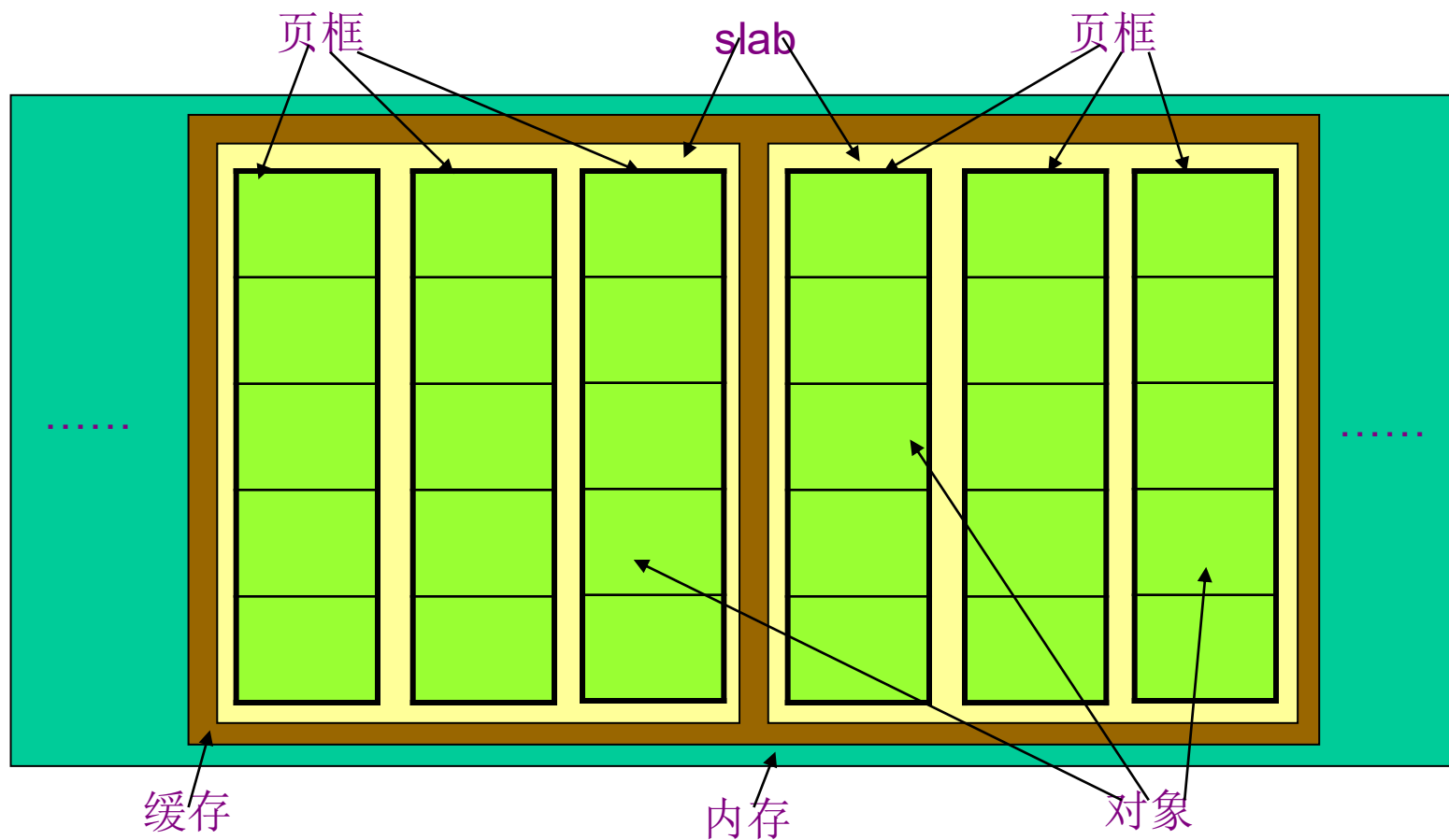


• Slab的核心思想

- 将回收和分配的数据结构称为“对象”
- 通过预先建立这些对象的缓冲池，来减少为内存对象分配空间的时间开销
- Slab分配器由许多缓存组成，不同的缓存存储大小不同的内存区
- 缓存分专用的和通用的两种（后面展开讲）
- `cat /proc/slabinfo`命令可以列出系统现有的slab分配器缓存

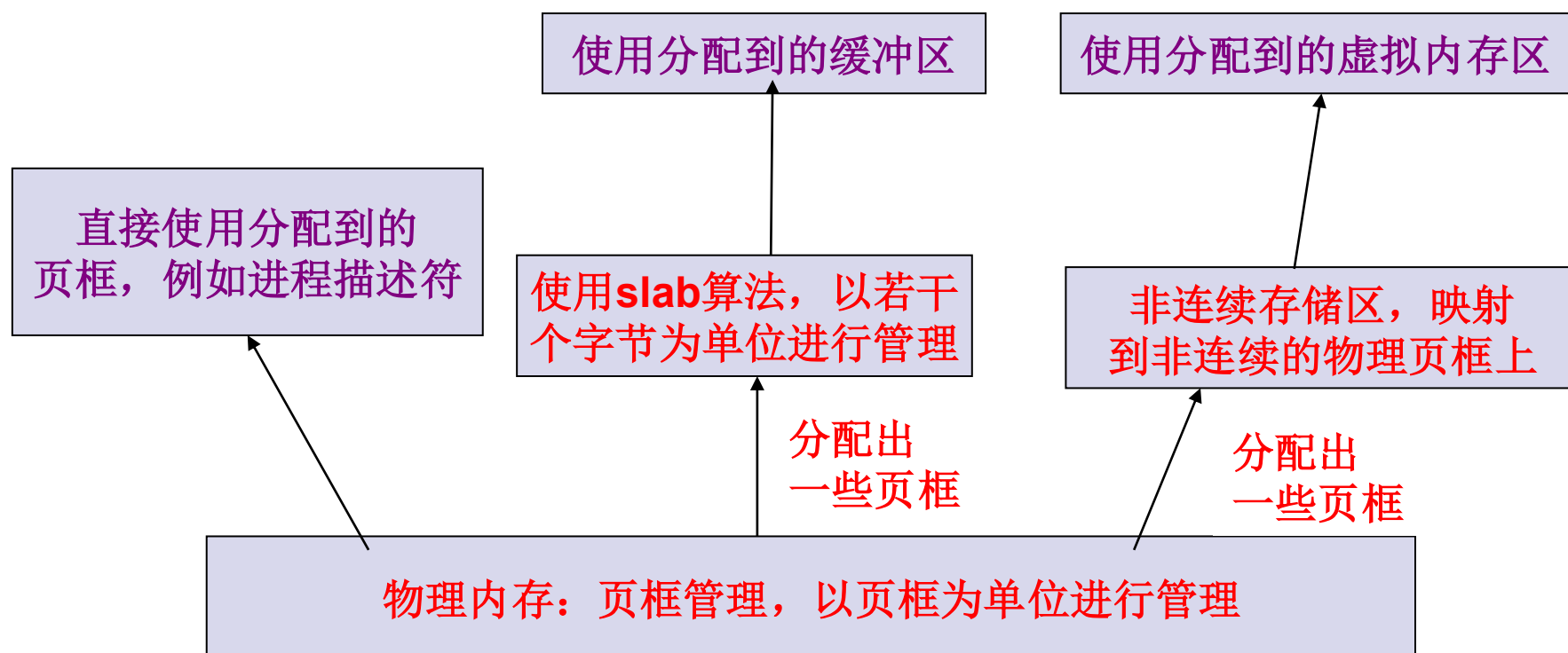


高速缓存就是主存中的一块区域，把它划分成多个块，每块就是一个 **Slab**，每个slab由一个或多个连续的页框组成，其中包含一定数目的对象



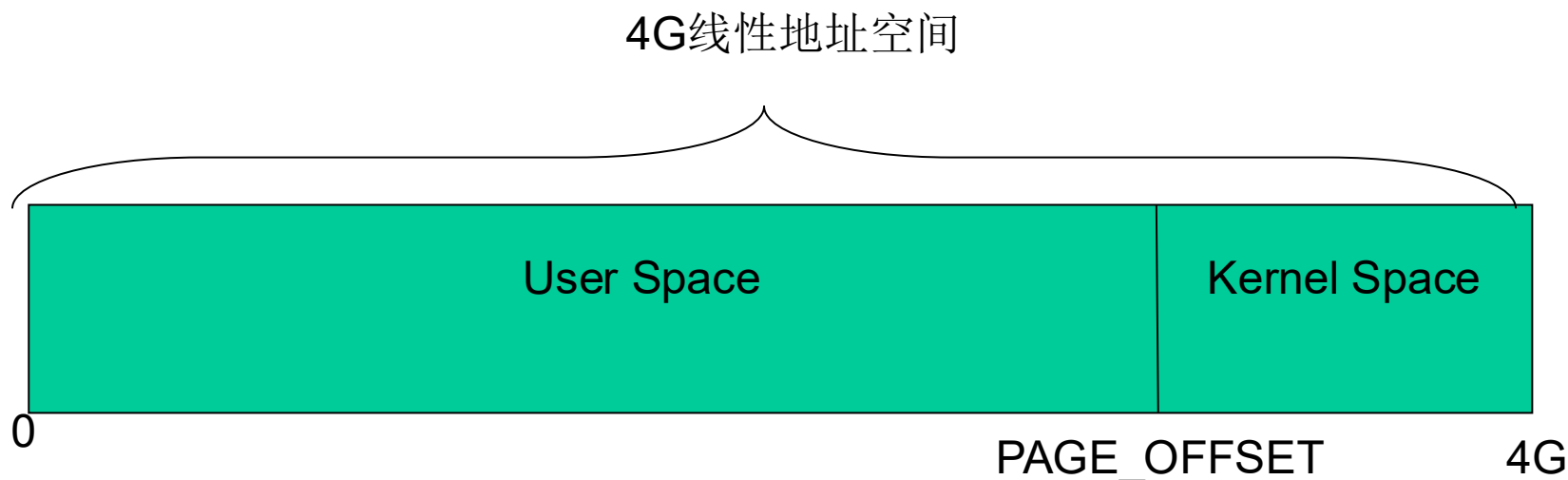


页框管理、内存区管理、 非连续存储区管理之间的关系





- Linux内核中对线性空间的划分和利用
 - 4GB的线性空间被分成用户地址空间和内核地址空间两部分，两部分的分界由PAGE_OFFSET定义，在include/asm-x86/page.h中，值0xC0000000





(1) Linux线性空间划分策略



- Linux将4GB的线性地址空间分成两部分：
 - 0到0XBFFFFFFF(3GB)是用户空间，由用户程序使用；
 - 0xC0000000到0xFFFFFFFF为内核空间，由内核使用
- 编译程序将内核代码定位在0xC0000000+0x100000处，即内核程序在系统代码段的偏移量从0xC0000000+0x100000开始



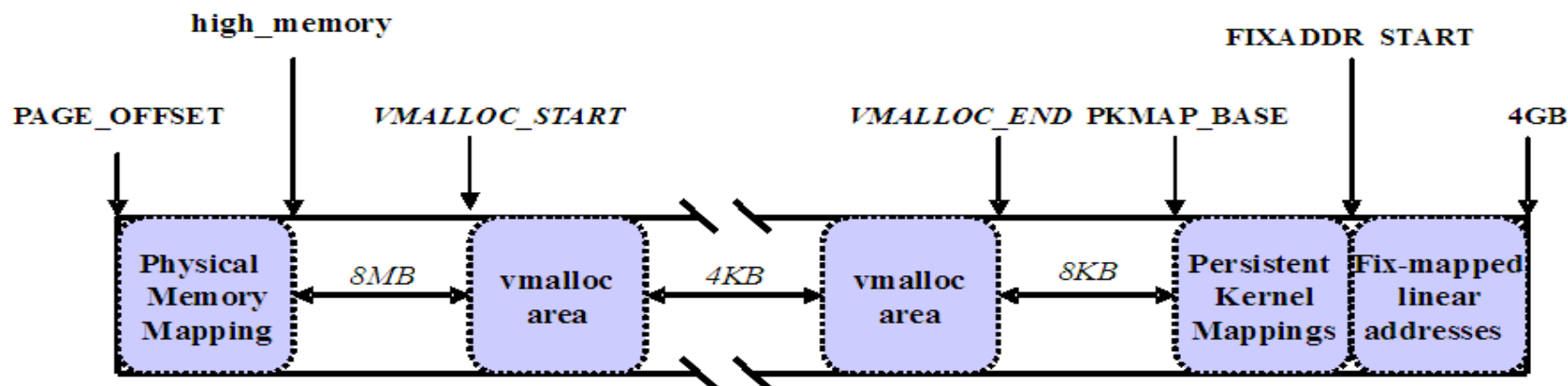
问题：如何实现这种规定？



(2) 1G内核线性地址空间的划分



- ❖ **1G**内核线性地址空间划分为**常规映射**、**非连续内存映射**、**持久映射**、**固定映射**等预留一些虚拟地址空间



- ❖ **常规映射**: 内核线性地址开始的部分是对前**896MB** RAM进行映射的线性地址，本区域的结尾保存在**high_memory**变量中
- ❖ **非连续内存映射**: 从**VMALLOC_START**到**VMALLOC_END** 是用于映射**非连续内存页帧**的内核线性地址空间，使用**vmalloc()**分配
- ❖ **持久映射**: 从**PKMAP_BASE**到**FIXADDR_START**用来由**kmap()**函数建立**持久映射**高端内存，用于查找高端内存页框
- ❖ **固定映射**: 内存区的结尾部分是**固定映射**的线性地址，用于**应急处理**

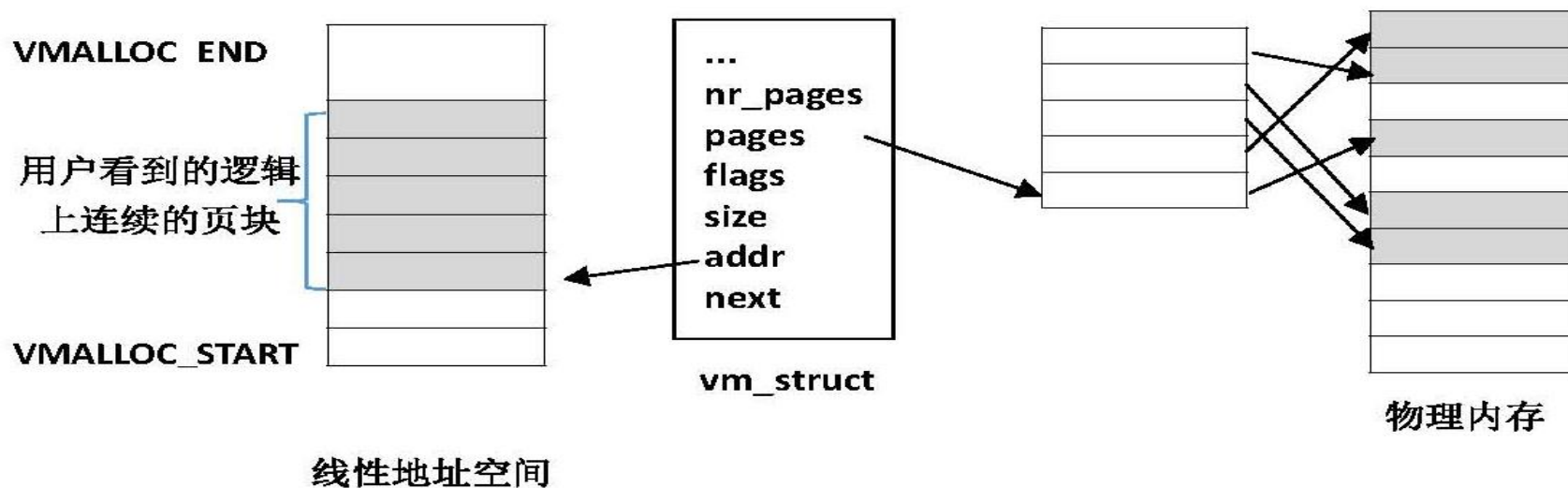


图 `vm_struct` 结构描述的逻辑内存块

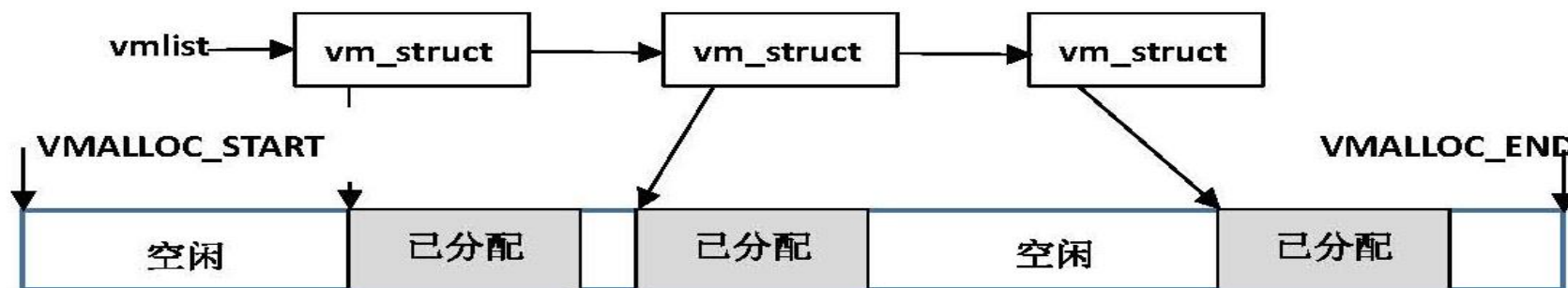
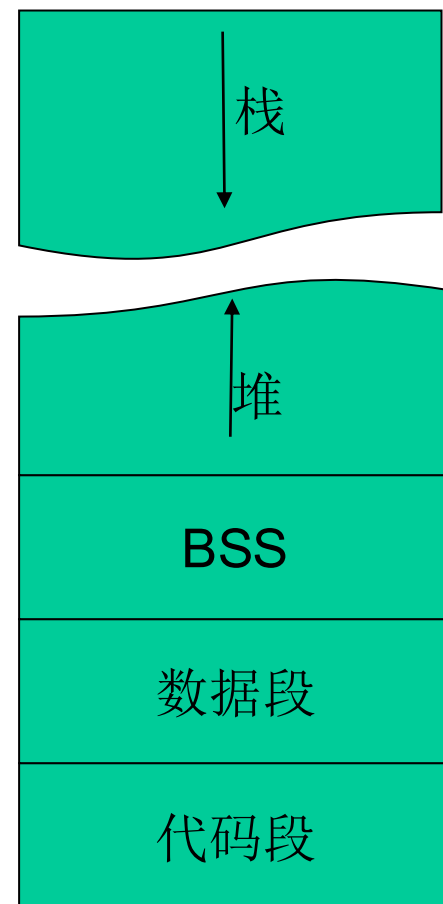


图 `vmlist` 队列与逻辑页块



- ❖ 进程的地址空间分成**3G私有空间+1G内核空间**
- ❖ 经**编译、链接后二进制映像文件**有一个代码段、数据段。代码段在下，数据段在上
- ❖ 进程对其**占用内存的管理方式**因内存用途不一而不尽相同，有些内存是**事先静态分配和统一回收的**，而有些却是**按需要动态分配和回收的**。
- ❖ 问题：**各个段存放什么内容？什么时候放？**
 - 全局变量、静态变量、局部变量、参数





❖ 进程在访问某个线性空间之前，必须获得该**线性空间的许可**

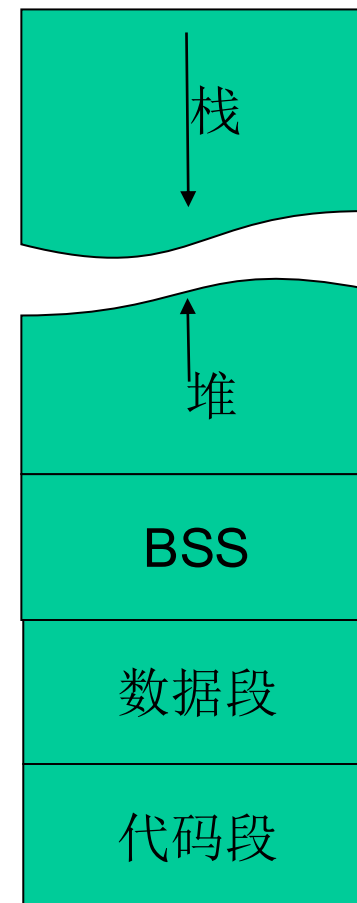
- 比如0x08048000—0x0804C000这段线性地址空间被分配给了一个进程，进程就可以访问这段地址空间
- 进程只能访问某个有效的VMA, 并且这个区域可以被标识为**只读或者不可执行(nonexecutable)**
- 如果进程试图访问一个**有效的area之外**的地址或者用不正确的方式访问一个有效的area，内核将通过段故障**杀死这个进程**

❖ 一个进程的地址空间是**由允许该进程访问的全部线性地址组成**

❖ 内核使用**VMA**来描述一个线性地址空间，**为了效率**，VMA的开始和结束都必须**4KB对齐**

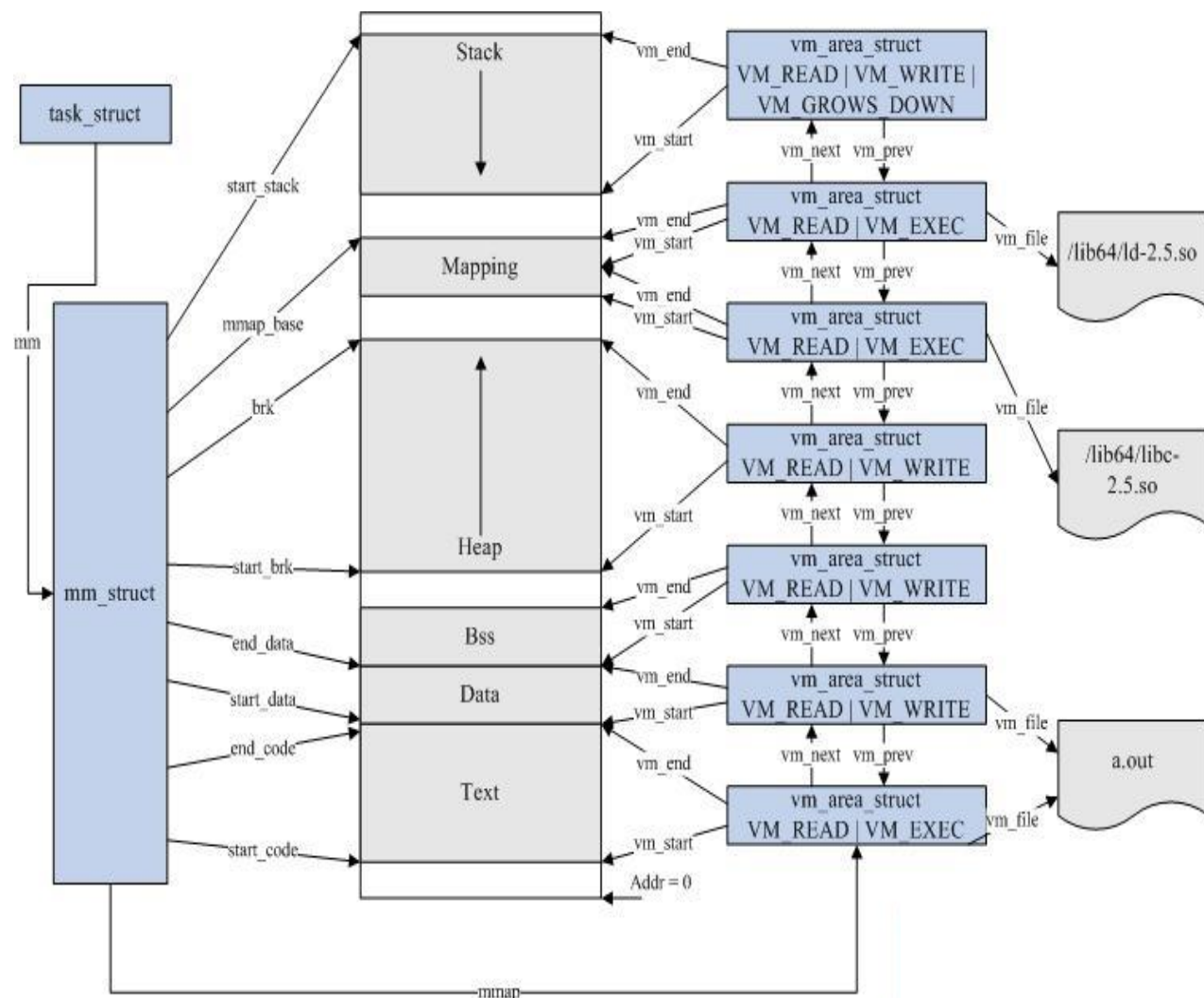
❖ 每个VMA由**起始线性地址**、**长度**和一些**存取权限**描述

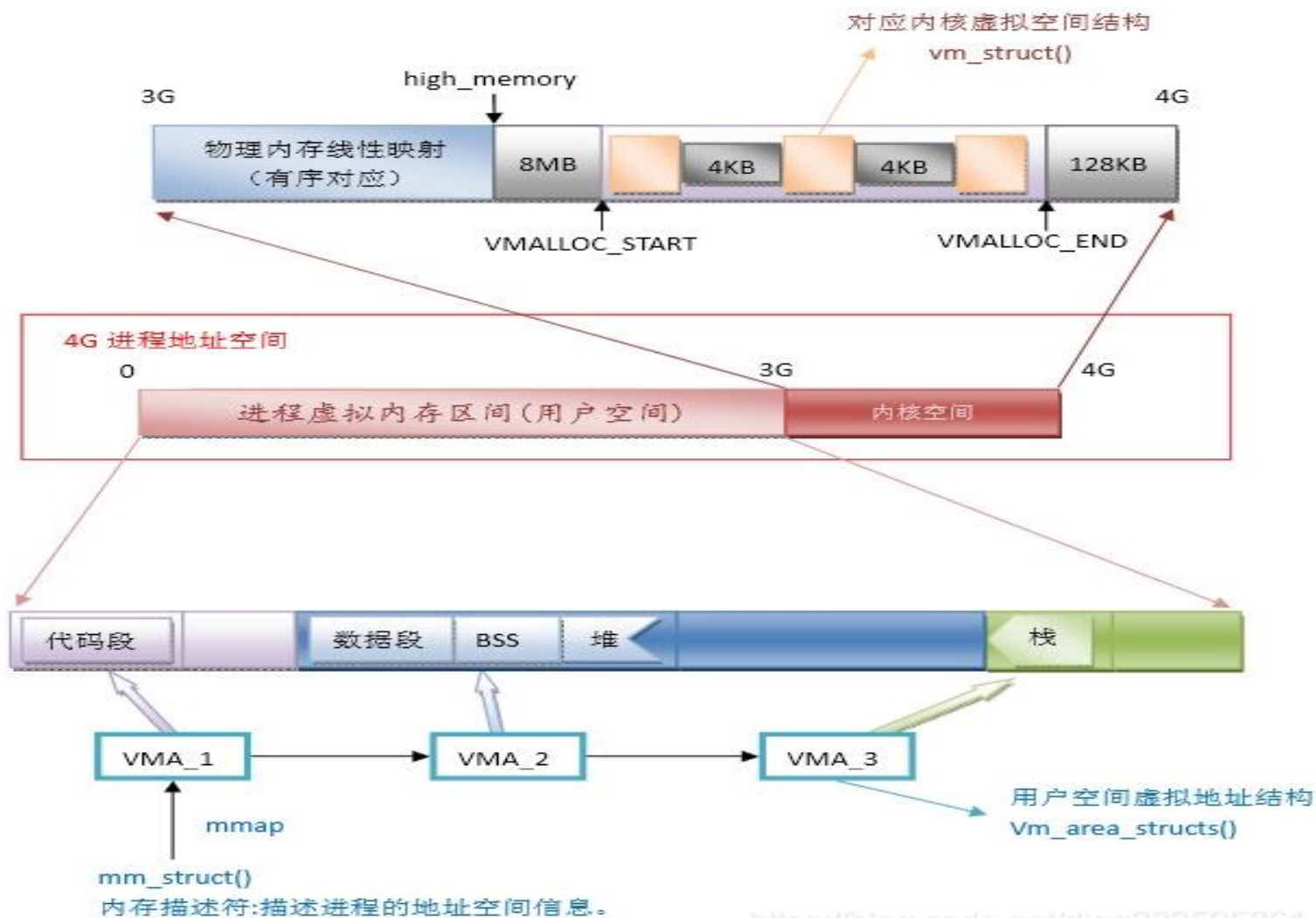
❖ **想象一下：**如何组织这些虚拟地址空间？





- **vm_area_struct**结构描述进程的一个虚拟内存地址区域(VMA)。一个VMA是对页处理有**相同规则**的进程虚拟内存空间的部分，如**共享库、运行区域**等，代表进程空间的一块单独连续的地址空间。
- 每个进程都有一个**struct mm_struct**结构体，用来描述进程的虚拟内存。在结构体中包含了进程的页表和其他的信息，其中**mmap**指向进程的VMA链。





<https://blog.csdn.net/ding283595861>

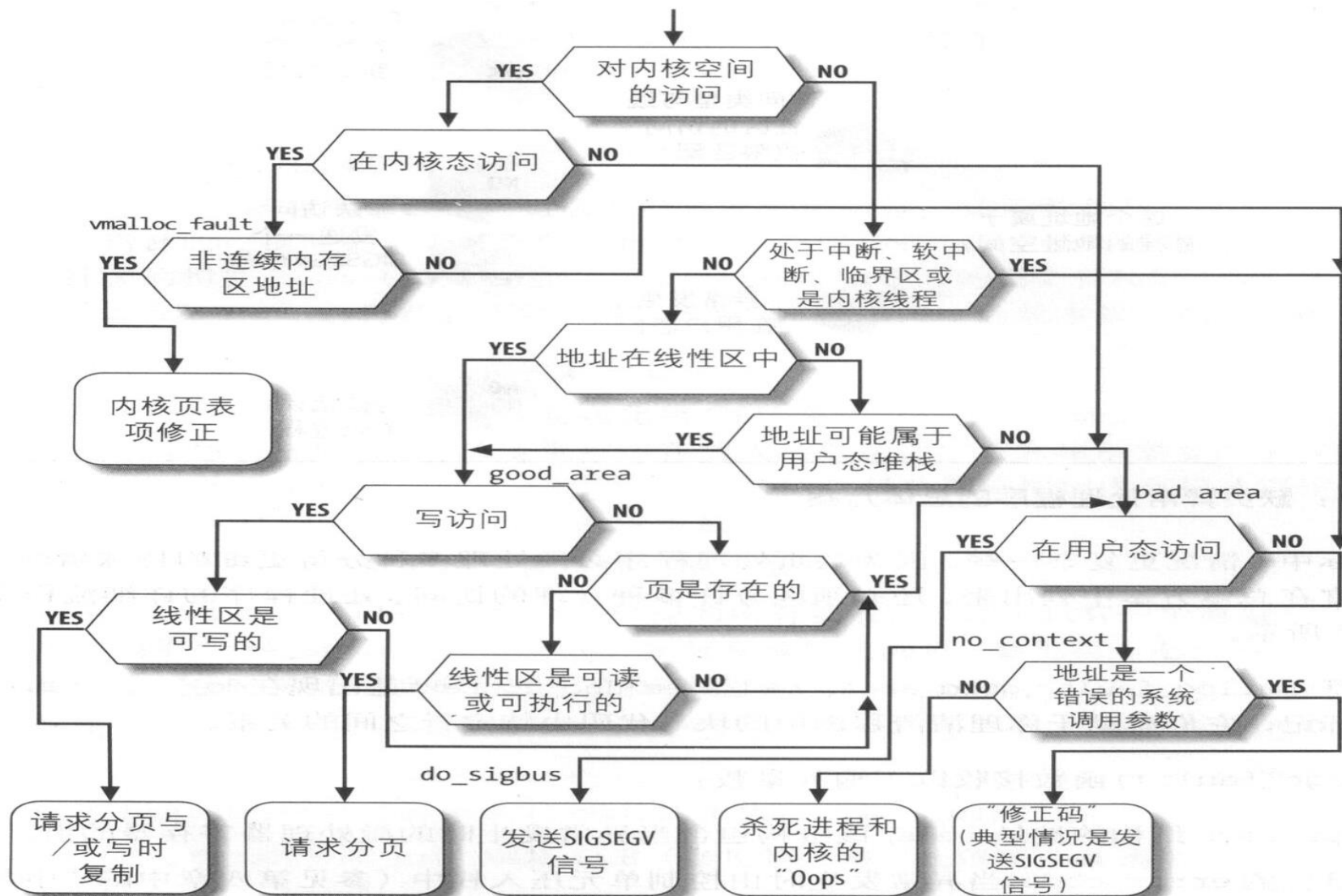


8.6.6 页面失效处理--缺页异常

- 如前所述，内核只是通过 **mmap()** 等调用分配了一些线性地址空间给进程，并没有真正的把实际的物理页框分配给进程
- 当进程试图访问这些分配给它的地址空间时，比如一段线性地址空间映射的是二进制代码，则进程被调度执行的时候会跳转到这个地址上去执行。
- 此时，并没有物理页框对应于这些线性地址，从而会引发一个 **缺页故障 (Page Fault)**
- 产生缺页故障时，先将异常的虚地址存入 **CR2** 寄存器中，然后调用缺页异常处理程序 **do_page_fault()**



8.6.6 页面失效处理--缺页处理程序流程图





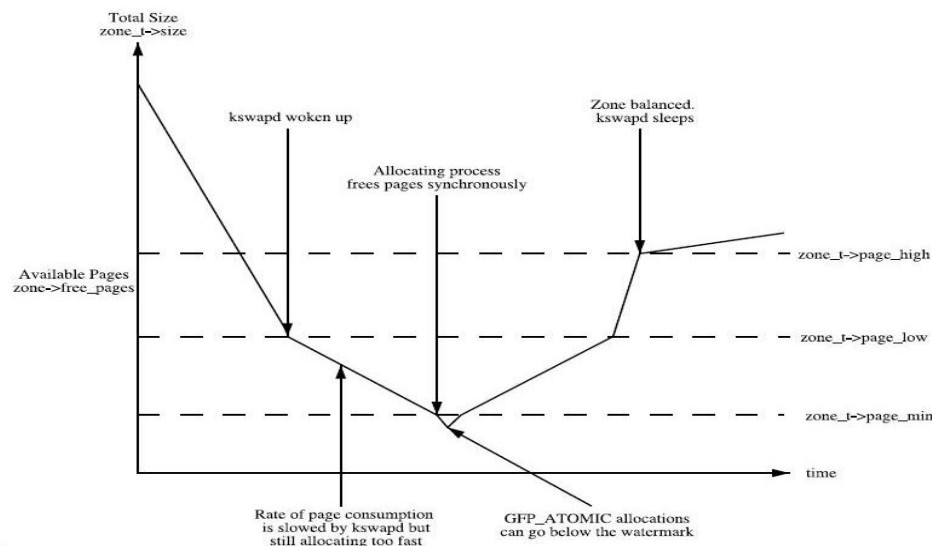
- 当内存物理页面不足时，Linux的MM子系统需要释放一些物理内存页框
- 由内核的交换守护线程kswapd完成，
 - 它是内核线程，在内核初始化时启动，并周期地运行
 - 任务：保证系统中有足够的空闲页面
- Linux中交换的单位是页面而不是进程
- 交换机制是一种以时间换空间的机制
- 交换要考虑的主要问题
 - 哪种页面要交换
 - 如何在交换区中存放页面
 - 如何选择被交换出的页面



- 内核中的kswapd和pdflush线程是跟内存管理相关的线程
- **kswapd用于物理内存回收**，以确保系统的空闲物理内存的数量在一个适当的范围。
 - 内存页分配中，如果zone的内存数量低于pages_low，就会通过wakeup_kswapd()唤醒kswapd，进行内存回收，一直到内存大于zone的pages_high的时候，kswapd进入休眠状态。

❖ pages_min, pages_low, pages_high, 表示持有内存区的水位值:

- ✓ if #空闲页 > pages_high 则内存域的状态是理想的;
- ✓ if #空闲页 < pages_low, 则内核开始将页换出到硬盘;
- ✓ if #空闲页 < pages_min, 则页回收工作压力大, 因为内核中急需空闲页





- **pdflush**负责内存的脏页面的回写。太多的脏页面意味着风险，内容易丢失
 - 当空闲内存低于阈值**dirty_background_ratio**时，内核调用**wakeup_pdflush()**唤醒pdflush将脏页写回磁盘，直到写完一定数量的脏页或者空闲内存大于阈值。
 - 内核中有一个定时器**wb_timer**，周期性地唤醒pdflush用于清理在内存中驻留时间超过**dirty_expire_centisecs**个百分之一秒的脏页，以确保脏页不会无限期地驻留在内存中。
 - 内存脏页回写会造成大量的磁盘IO。Linux初始化时创建了2个pdflush线程，当回写压力大时，可以创建新的pdflush线程分担回写工作，最多可分配8个pdflush线程，当会写需求降低时，可以删除空闲的pdflush线程



2008.11.13



第八章 小结

如何解决存储器容量、速度、造价之间的矛盾？

无

●如何在有限的内存中执行大程序？

- ✚覆盖与交换
- ✚简单段、页式管理
- ✚虚拟段、页式管理

●存储管理的具体问题：

- ✚地址映射与变换
- ✚程序链接
- ✚内存分区管理
- ✚内存分配与回收
- ✚内存共享与保护
- ✚页面调入与置换

Secondary Storage)

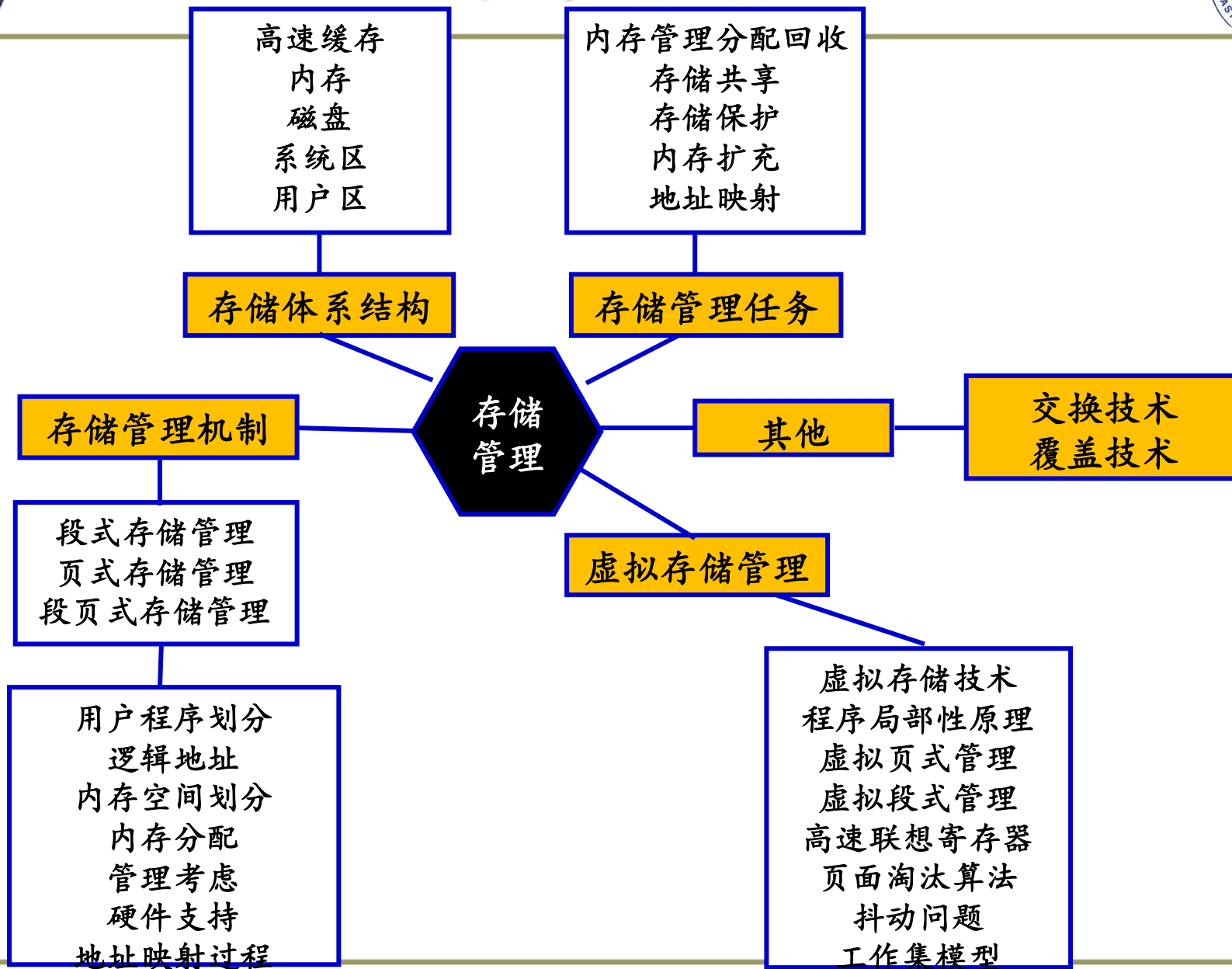
●相关硬件、数据结构

- ✚寄存器、栈
- ✚页表、段表
- ✚快表
- ✚分区说明表

联系



本章总结





继续学习第9章： 文件系统

